

Deep Learning

Note di Studio

Basato sulle slide del Prof. **Vito Walter Anelli**
Politecnico di Bari – SisInf Lab

Prefazione

Queste note sono redatte come supporto allo studio del corso di **Deep Learning** del Prof. Vito Walter Anelli (Politecnico di Bari). Il testo integra le slide ufficiali del corso con due risorse fondamentali: *NYU-DLSP21* ([LeCun and Canziani, 2021](#)) di LeCun e Canziani e *Deep Learning: Foundations and Concepts* ([Bishop and Bishop, 2024](#)) di Bishop & Bishop.

Nel testo, le etichette ^[BX] rimandano al capitolo X del libro Bishop, mentre ^[NYU Y] alla settimana Y del corso NYU.

Indice

Prefazione	1
1 Introduzione al Deep Learning	8
1.1 Machine Learning vs Deep Learning	8
1.2 Modello Parametrico e Funzione di Costo	9
1.2.1 Notazione con grafi a blocchi	10
1.2.2 Funzione di loss e loss media	10
1.3 Reti Neurali Tradizionali	11
1.3.1 Implementazione in PyTorch	12
1.4 Grafi Computazionali e Backpropagation	12
1.4.1 Regola della catena – caso scalare	12
1.4.2 Caso vettoriale e matrice Jacobiana	13
1.4.3 Backprop in un grafo multi-stadio	14
1.5 Riepilogo	15
2 Architetture di Base	16
2.1 Moduli Moltiplicativi	16
2.1.1 Unità Sigma-Pi e layer quadratico	16
2.1.2 Modulo di attenzione	17
2.1.3 Proprietà di differenziabilità	17
2.2 Mixture of Experts	18
2.3 Trasformazioni dei Parametri	19
2.3.1 Weight sharing – condivisione dei pesi	20
2.3.2 Shared weights per la rilevazione di motif	20
2.4 Riepilogo	21
3 Learning Representations	22
3.1 Limiti dei Classificatori Lineari	22
3.2 La Soluzione: Rappresentazioni (Feature)	23
3.2.1 Idee per l'estrazione generica di feature	24
3.2.2 Feature monomiali e limiti	24

3.3	Reti Shallow vs Reti Profonde	25
3.3.1	Le reti shallow sono approssimatori universali	25
3.3.2	Perché servono architetture profonde?	25
3.4	Invariant Feature Learning	26
3.5	L'Ipotesi del Manifold	27
3.6	Disentangling e Apprendimento delle Rappresentazioni	28
3.7	Architetture Multilayer = Struttura Compositiva	28
3.8	Riepilogo	30
4	Activation Functions	31
4.1	Il Problema del Vanishing Gradient	31
4.2	Panoramica: Saturating vs Non-Saturating	32
4.3	Famiglia ReLU	33
4.3.1	ReLU – <code>nn.ReLU()</code>	33
4.3.2	Leaky ReLU – <code>nn.LeakyReLU()</code>	33
4.3.3	PReLU – <code>nn.PReLU()</code>	34
4.3.4	RReLU – <code>nn.RReLU()</code>	35
4.3.5	ReLU6 – <code>nn.ReLU6()</code>	35
4.4	Famiglia ELU	36
4.4.1	ELU – <code>nn.ELU()</code>	36
4.4.2	CELU – <code>nn.CELU()</code>	37
4.4.3	SELU – <code>nn.SELU()</code>	37
4.5	Softplus e GELU	38
4.5.1	Softplus – <code>nn.Softplus()</code>	38
4.5.2	GELU – <code>nn.GELU()</code>	38
4.6	Funzioni Saturanti	39
4.6.1	Sigmoid – <code>nn.Sigmoid()</code>	39
4.6.2	Tanh – <code>nn.Tanh()</code>	40
4.6.3	Hardtanh – <code>nn.Hardtanh()</code>	41
4.6.4	Softsign – <code>nn.Softsign()</code>	42
4.7	Funzioni Shrink e Threshold	43
4.7.1	Threshold – <code>nn.Threshold()</code>	43
4.7.2	Tanhshrink – <code>nn.Tanhshrink()</code>	43
4.7.3	Softshrink – <code>nn.Softshrink()</code>	44
4.7.4	Hardshrink – <code>nn.Hardshrink()</code>	44
4.8	LogSigmoid – <code>nn.LogSigmoid()</code>	45
4.9	Funzioni Vettoriali: Softmin, Softmax, LogSoftmax	46
4.9.1	Softmin – <code>nn.Softmin()</code>	46

4.9.2	Soft(arg)max – <code>nn.Softmax()</code>	46
4.9.3	LogSoft(arg)max – <code>nn.LogSoftmax()</code>	47
4.10	Softmax e Cross-Entropy Loss	48
4.11	Guida Pratica alla Scelta	49
4.12	Riepilogo	50
5	Loss Functions	51
5.1	Loss per Regressione	51
5.1.1	MSE Loss – <code>nn.MSELoss()</code>	51
5.1.2	L1 Loss – <code>nn.L1Loss()</code>	52
5.1.3	Smooth L1 Loss (Huber Loss) – <code>nn.SmoothL1Loss()</code>	52
5.1.4	L1 vs. L2 in Computer Vision	52
5.2	Loss per Classificazione	53
5.2.1	NLL Loss – <code>nn.NLLLoss()</code>	53
5.2.2	Gestione dei Dataset Sbilanciati	53
5.2.3	Cross-Entropy Loss – <code>nn.CrossEntropyLoss()</code>	54
5.2.4	Binary Cross Entropy – <code>nn.BCELoss()</code>	55
5.2.5	BCE with Logits – <code>nn.BCEWithLogitsLoss()</code>	55
5.2.6	NLL Loss Adattiva – <code>nn.AdaptiveLogSoftmaxWithLoss()</code>	55
5.2.7	KL Divergence Loss – <code>nn.KLDivLoss()</code>	55
5.3	Loss Basate su Margine	55
5.3.1	Hinge Embedding Loss – <code>nn.HingeEmbeddingLoss()</code>	56
5.3.2	Margin Ranking Loss – <code>nn.MarginRankingLoss()</code>	56
5.3.3	Triplet Margin Loss – <code>nn.TripletMarginLoss()</code>	56
5.3.4	Soft Margin Loss – <code>nn.SoftMarginLoss()</code>	56
5.3.5	Multi-Class Hinge Loss – <code>nn.MultiLabelMarginLoss()</code>	57
5.4	Loss per Embedding: Cosine Embedding Loss	57
5.4.1	Formulazione	57
5.4.2	Interpretazione Geometrica	57
5.5	Guida Pratica alla Scelta della Loss	58
5.6	Riepilogo	59
6	Ottimizzazione	60
6.1	La Superficie di Errore	60
6.1.1	Neuroni Lineari e Ciotola Quadratica	60
6.1.2	Il Problema della Direzione del Gradiente	61
6.2	Algoritmi di Apprendimento: Batch, Mini-Batch, Online	62
6.2.1	Ridondanza nel Dataset	62
6.3	Accorgimenti per il Mini-Batch Gradient Descent	62

6.3.1	Learning Rate e Quando Ridurlo	62
6.3.2	Inizializzazione dei Pesi	63
6.3.3	Pre-elaborazione degli Input	64
6.4	Il Metodo del Momentum	64
6.4.1	Intuizione Fisica	64
6.4.2	Equazioni del Momentum (SGD con Momentum)	65
6.4.3	Momentum di Nesterov (NAG)	65
6.5	Learning Rate Adattativi Separati	66
6.5.1	Motivazione	66
6.5.2	Determinazione dei Guadagni Locali	66
6.6	rprop e rmsprop	67
6.6.1	rprop: Solo il Segno del Gradiente	67
6.6.2	rmsprop: Versione Mini-Batch di rprop	67
6.7	Metodi Adattativi Avanzati	68
6.7.1	Adagrad	68
6.7.2	Adadelta	68
6.7.3	Adam: Adaptive Moment Estimation	69
6.7.4	AdamW: Adam con Weight Decay Corretto	70
6.7.5	Lion: EvoLved Sign Momentum	71
6.8	Metodi di Secondo Ordine	71
6.8.1	Riduzione dell'Errore e Rapporto Gradiente/Curvatura	71
6.8.2	Metodo di Newton	71
6.8.3	Hessian-Free Optimization e Gradiente Coniugato	72
6.9	Riepilogo dei Metodi di Ottimizzazione	72
7	Normalization Layers	74
7.1	Covariate Shift e Internal Covariate Shift	74
7.2	Batch Normalization	74
7.2.1	Idea di Base	74
7.2.2	Perché l'Approccio Naïf Non Funziona	75
7.2.3	La Soluzione: un Layer di Normalizzazione Differenziabile	75
7.2.4	Parametri Apprendibili γ e β	76
7.3	Vantaggi della Batch Normalization	76
7.3.1	Benefici Teorici (Motivazione Originale)	76
7.3.2	Benefici Pratici	77
7.4	Posizionamento del Layer BN	77
7.5	Varianti di Normalizzazione	78
7.5.1	Batch Normalization (BN)	79

7.5.2	Layer Normalization (LN)	79
7.5.3	Instance Normalization (IN)	79
7.5.4	Group Normalization (GN)	79
7.6	Perché la Normalizzazione Funziona Davvero?	80
7.7	Riepilogo e Linee Guida Pratiche	80
8	Reti Neurali Convoluzionali	82
8.1	Limiti delle Reti Completamente Connesse sulle Immagini	82
8.2	Volumi 3D e Connettività Locale	82
8.2.1	Organizzazione tridimensionale	82
8.2.2	Connettività locale	83
8.3	Il Layer di Convoluzione	83
8.3.1	Operazione di convoluzione	83
8.3.2	Mappe di attivazione	84
8.3.3	Dimensioni spaziali: stride e zero-padding	85
8.3.4	Convoluzione 1×1	86
8.4	Campi Recettivi	86
8.5	Cosa Imparano i Filtri Convoluzionali?	87
8.6	Il Layer di Pooling	87
8.6.1	Max Pooling	87
8.6.2	Altre varianti di pooling	88
8.7	Il Layer Fully Connected	88
8.8	Architettura Tipica di una ConvNet	88
8.8.1	Esempio di stack convoluzionale	89
8.9	Proprietà Fondamentali delle ConvNet	89
8.9.1	Parameter sharing	89
8.9.2	Equivarianza traslazionale	89
8.9.3	Connessioni sparse	90
8.10	Riepilogo dei Componenti Principali	90
9	Reti Ricorrenti	91
9.1	Motivazione: Modellare Sequenze	91
9.1.1	Il Problema della Modellazione Sequenziale	91
9.1.2	Rappresentazione Numerica del Linguaggio	92
9.2	Reti Neurali Ricorrenti (RNN)	92
9.2.1	Neuroni con Ricorrenza	92
9.2.2	Grafo Computazionale nel Tempo	93
9.2.3	Architetture Sequenziali	94
9.3	Backpropagation Through Time (BPTT)	94

9.3.1	Algoritmo	94
9.3.2	Il Problema dei Gradienti che Svaniscono o Esplodono	95
9.4	Long Short-Term Memory (LSTM)	96
9.4.1	Motivazione: La Memoria Analogica	96
9.4.2	Architettura della Cella LSTM	96
9.4.3	Flusso del Gradiente nelle LSTM	98
9.5	Gated Recurrent Unit (GRU)	99
9.5.1	Architettura	99
9.5.2	Confronto LSTM vs GRU	100
9.6	Bi-LSTM e Architetture Avanzate	100
9.6.1	LSTM Bidirezionale	100
9.6.2	Encoder–Decoder per la Traduzione Automatica	100
9.7	Applicazioni delle RNN	101
9.8	Riepilogo e Linee Guida Pratiche	101

Capitolo 1

Introduzione al Deep Learning

La veloce crescita odierna del Machine Learning ha portato alla nascita di nuove tecnologie e campi di applicazione. Il **Deep Learning** nasce con l'intento di apprendere rappresentazioni gerarchiche direttamente dai dati, tramite reti neurali profonde. I campi di applicazione spaziano dalla diagnostica medica fino ai Large Language Models (LLM) (Bishop and Bishop, 2024).

1.1 Machine Learning vs Deep Learning

Nel **Machine Learning classico** la pipeline si divide in due fasi nettamente separate: un'estrazione di feature progettata manualmente dall'ingegnere, seguita da un classificatore appreso automaticamente dai dati. Nel **Deep Learning** entrambe le fasi sono apprese congiuntamente e in modo gerarchico: i layer bassi catturano edge e texture, quelli intermedi parti di oggetti, quelli alti concetti semantici (LeCun and Canziani, 2021).

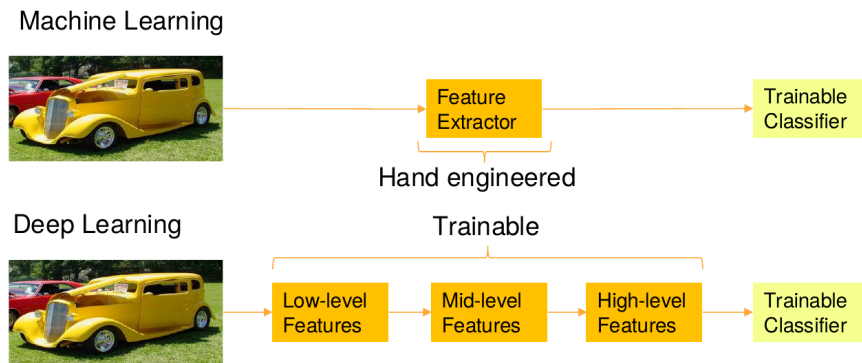


Figura 1.1: Confronto tra la pipeline di ML classico (feature extractor hand-engineered + classificatore) e quella di Deep Learning (tutta la catena è trainable end-to-end).

Il Deep Learning si colloca all'interno della gerarchia $AI \supset Machine\ Learning \supset Representation\ Learning \supset Deep\ Learning$. La figura seguente mostra questa gerarchia e il corrispondente spettro di paradigmi, dal rule-based al deep learning.

Learning Multiple Components

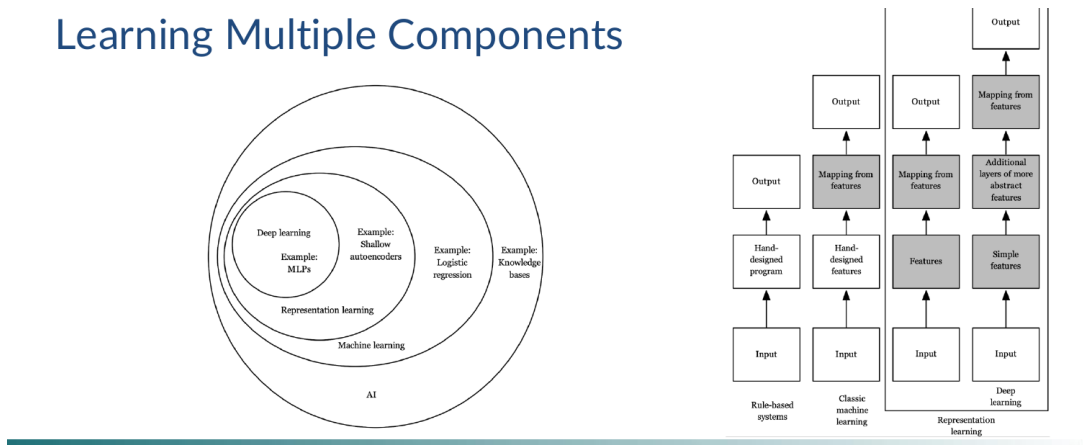


Figura 1.2: Gerarchia AI/ML/Repr. Learning/DL (sinistra) e i quattro paradigmi principali con il loro grado di automazione (destra).

Un esempio storico di benchmark è il dataset **MNIST** (LeCun et al., 1998): 70 000 immagini 28×28 pixel di cifre scritte a mano (0–9). Un MLP semplice raggiunge circa il 98% di accuratezza; la CNN LeNet-5 arriva al 99.3%.

1.2 Modello Parametrico e Funzione di Costo

Un **modello parametrico** è una funzione deterministica della forma

$$\bar{y} = G(x, w) \quad (1.1)$$

dove x è l'ingresso osservato, w il vettore dei parametri addestrabili e $\bar{y}$ l'uscita predetta. I parametri w sono condivisi tra tutti i campioni di addestramento, mentre x varia da campione a campione.

Parameterized Model

- Parameterized model

$$\bar{y} = G(x, w)$$

- Example: linear regression

$$\bar{y} = \sum_i w_i x_i \quad C(y, \bar{y}) = \|y - \bar{y}\|^2$$

- Example: Nearest neighbor:

$$\bar{y} = \arg \min_k \|x - w_k, \dots\|^2$$

Computing function G may involve complicated algorithms

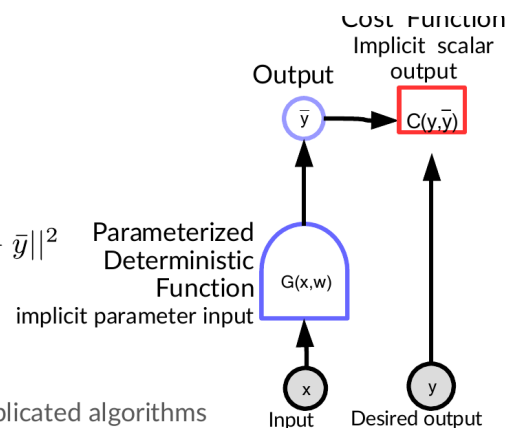


Figura 1.3: Schema del modello parametrico $\bar{y} = G(x, w)$. Il blocco $G(x, w)$ (rettangolo blu) prende in input x e produce $\bar{y}$; la funzione di costo $C(y, \bar{y})$ (rettangolo rosso) ha output scalare implicito.

Due esempi fondamentali sono la **regressione lineare**, $\bar{y} = \sum_i w_i x_i$ con costo $C(y, \bar{y}) = \|y - \bar{y}\|^2$, e il **k-Nearest Neighbor**, $\bar{y} = \arg \min_k \|x - w_k, \dots\|^2$, dove i parametri w_k coincidono con i prototipi del dataset di training.

1.2.1 Notazione con grafi a blocchi

Le slide del corso adottano una notazione grafica precisa per i grafi computazionali [NYU W1], illustrata nella figura seguente.

Block diagram notations for computation graphs

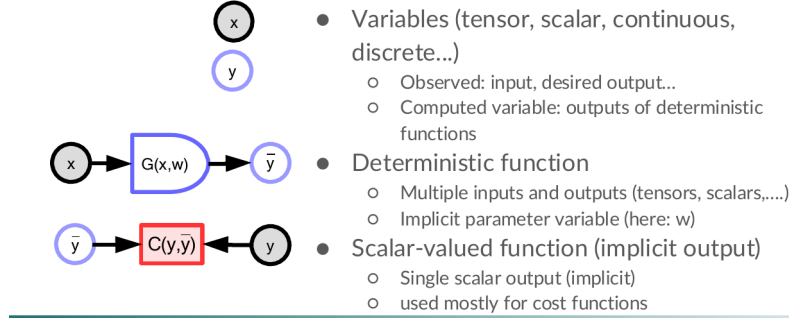


Figura 1.4: Notazione a blocchi per grafi computazionali: cerchi a bordo spesso = variabili osservate; cerchi a bordo sottile = variabili calcolate; rettangoli blu = funzioni deterministiche; rettangoli rossi = funzioni di costo con output scalare implicito.

1.2.2 Funzione di loss e loss media

La **loss per campione** è $L(x, y, w) = C(y, G(x, w))$. Dato il dataset $S = \{(x[p], y[p]) \mid p = 0, \dots, P-1\}$, la **loss media** è:

$$\mathcal{L}(S, w) = \frac{1}{P} \sum_{p=0}^{P-1} L(x[p], y[p], w). \quad (1.2)$$

L'obiettivo dell'apprendimento è $w^* = \arg \min_w \mathcal{L}(S, w)$.

Loss function, average loss

- Simple per-sample loss function

$$L(x, y, w) = C(y, G(x, w))$$
- A set of samples

$$S = \{(x[p], y[p]) / p = 0 \dots P-1\}$$
- Average loss over the set

$$\mathcal{L}(S, w) = \frac{1}{P} \sum_{(x,y)} L(x, y, w) = \frac{1}{P} \sum_{p=0}^{P-1} L(x[p], y[p], w)$$

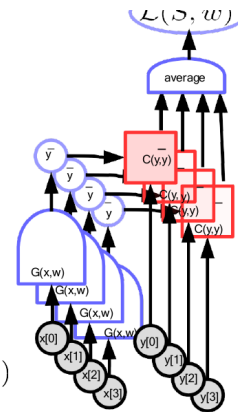


Figura 1.5: Grafo computazionale della loss media: il blocco $G(x, w)$ è replicato per ogni campione (con pesi condivisi) e le loss individuali $C(y, \bar{y})$ vengono mediate.

1.3 Reti Neurali Tradizionali

Una rete neurale tradizionale è una composizione alternata di blocchi **lineari** e **non-lineari** [B6]:

$$s_{k+1} = w_k z_k \quad (\text{blocco lineare}) \quad (1.3)$$

$$z_k = h(s_k) \quad (\text{blocco non-lineare}) \quad (1.4)$$

dove $h(\cdot)$ è una funzione di attivazione punto per punto. Per un neurone i , indicando con $UP(i)$ i neuroni del livello precedente:

$$s[i] = \sum_{j \in UP(i)} w[i, j] \cdot z[j], \quad z[i] = f(s[i]). \quad (1.5)$$

Traditional Neural Net

- Stacked linear and non-linear functional blocks

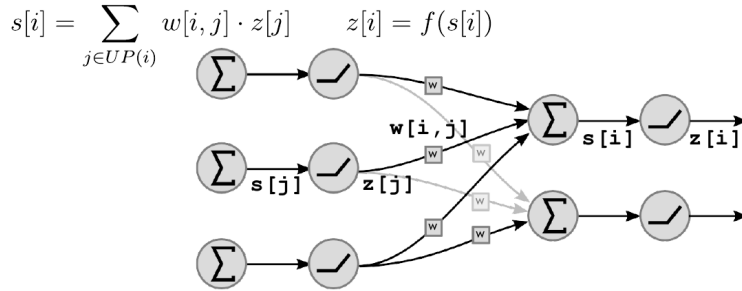


Figura 1.6: Struttura di una rete neurale tradizionale: i nodi Σ calcolano la somma pesata $s[i]$; i nodi con la curva applicano la non-linearità $z[i] = f(s[i])$; i riquadri w rappresentano i pesi $w[i, j]$.

La catena lineare a blocchi per una rete a 3 layer è: $x \xrightarrow{w_0} s_1 \xrightarrow{h} z_1 \xrightarrow{w_1} s_2 \xrightarrow{h} z_2 \xrightarrow{w_2} s_3$.

Block Diagram of a Traditional Neural Net

- Linear blocks:

$$s_{k+1} = w_k z_k$$

- Non-linear blocks:

$$z_k = h(s_k)$$

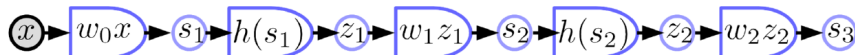


Figura 1.7: Diagramma a blocchi di una rete a 3 layer: blocchi lineari (rettangoli) e non-lineari (forme arrotondate) si alternano.

1.3.1 Implementazione in PyTorch

Listing 1.1: MLP a 3 layer in PyTorch

```

1 import torch
2 from torch import nn
3
4 class MyNet(nn.Module):
5     def __init__(self, d0, d1, d2, d3):
6         super().__init__()
7         self.m0 = nn.Linear(d0, d1)    # include bias
8         self.m1 = nn.Linear(d1, d2)
9         self.m2 = nn.Linear(d2, d3)
10
11     def forward(self, x):
12         z0 = x.view(-1)                # flatten
13         z1 = torch.relu(self.m0(z0))
14         z2 = torch.relu(self.m1(z1))
15         return self.m2(z2)             # logit di output
16
17 model = MyNet(600, 60, 40, 10)
18 out = model(torch.randn(3, 10, 20))

```

1.4 Grafi Computazionali e Backpropagation

Un **grafo computazionale** è un DAG in cui i nodi foglia sono le variabili di ingresso e i parametri, mentre i nodi interni rappresentano operazioni. L'ordinamento topologico definisce l'ordine del forward e del backward pass ^[B7]. Se il grafo contiene cicli (reti ricorrenti), è necessario srotolarlo nel tempo (**BPTT**).

1.4.1 Regola della catena – caso scalare

La backpropagation (Rumelhart et al., 1986) si fonda sulla **regola della catena**: per $c = g(z)$, $z = h(s)$:

$$\frac{dc}{ds} = \frac{dc}{dz} \cdot h'(s). \quad (1.6)$$

Perturbando s di ds si perturba z di $dz = h'(s) ds$, e quindi c di $dc = (dc/dz) dz$.

Backprop through a non-linear function

- Chain rule:
 $g(h(s))' = g'(h(s)) \cdot h'(s)$
 $dc/ds = dc/dz \cdot dz/ds$
 $dc/ds = dc/dz \cdot h'(s)$
- Perturbations:
 Perturbing s by ds will perturb z by:
 $dz = ds \cdot h'(s)$
 This will perturb c by
 $dc = dz \cdot dc/dz = ds \cdot h'(s) \cdot dc/dz$
 Hence:
 $dc/ds = dc/dz \cdot h'(s)$

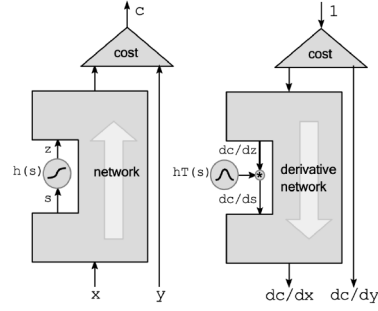


Figura 1.8: Backprop attraverso una funzione non-lineare $h(s)$: la rete duale (destra) moltiplica il gradiente entrante dc/dz per la derivata trasposta $h^T(s)$, producendo dc/ds .

Per un neurone z che alimenta più neuroni $s[0], s[1], s[2]$ tramite pesi $w[0], w[1], w[2]$, il gradiente rispetto all'ingresso è la somma pesata:

$$\frac{dc}{dz} = \sum_i \frac{dc}{ds[i]} \cdot w[i]. \quad (1.7)$$

1.4.2 Caso vettoriale e matrice Jacobiana

Per funzioni vettoriali con $z_f : [d_f \times 1]$ e $z_g : [d_g \times 1]$:

$$\underbrace{\frac{\partial c}{\partial z_f}}_{[1 \times d_f]} = \underbrace{\frac{\partial c}{\partial z_g}}_{[1 \times d_g]} \cdot \underbrace{\frac{\partial z_g}{\partial z_f}}_{[d_g \times d_f]}. \quad (1.8)$$

La **matrice Jacobiana** ha elemento (i, j) uguale alla derivata parziale della i -esima uscita rispetto alla j -esima entrata: $(\partial z_g / \partial z_f)_{ij} = (\partial z_g)_i / (\partial z_f)_j$.

Backprop through a functional module

- Using chain rule for vector functions
 $z_g : [d_g \times 1] \quad z_f : [d_f \times 1]$
 $\frac{\partial c}{\partial z_f} = \frac{\partial c}{\partial z_g} \frac{\partial z_g}{\partial z_f}$
 $[1 \times d_f] = [1 \times d_g] * [d_g \times d_f]$
- Jacobian matrix
 Partial derivative of i -th output w.r.t. j -th input
 $\left(\frac{\partial z_g}{\partial z_f} \right)_{ij} = \frac{(\partial z_g)_i}{(\partial z_f)_j}$

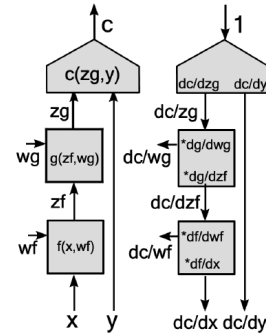


Figura 1.9: Backprop attraverso un modulo vettoriale: ogni modulo espone due Jacobiane, una per i pesi ($*dg/dwg$) e una per l'ingresso ($*dg/dzf$), corrispondenti alle due frecce della rete duale.

1.4.3 Backprop in un grafo multi-stadio

Per una rete a K stadi con $z_{k+1} = f_k(z_k, w_k)$:

$$\frac{\partial c}{\partial z_k} = \frac{\partial c}{\partial z_{k+1}} \cdot \frac{\partial f_k(z_k, w_k)}{\partial z_k} \quad (\text{gradiente di stato}) \quad (1.9)$$

$$\frac{\partial c}{\partial w_k} = \frac{\partial c}{\partial z_{k+1}} \cdot \frac{\partial f_k(z_k, w_k)}{\partial w_k} \quad (\text{gradiente dei pesi}) \quad (1.10)$$

Backprop through a multi-stage graph

- Using chain rule for vector functions

$$\frac{\partial c}{\partial z_k} = \frac{\partial c}{\partial z_{k+1}} \frac{\partial z_{k+1}}{\partial z_k} = \frac{\partial c}{\partial z_{k+1}} \frac{\partial f_k(z_k, w_k)}{\partial z_k}$$

$$\frac{\partial c}{\partial w_k} = \frac{\partial c}{\partial z_{k+1}} \frac{\partial z_{k+1}}{\partial w_k} = \frac{\partial c}{\partial z_{k+1}} \frac{\partial f_k(z_k, w_k)}{\partial w_k}$$

- Two Jacobian matrices for the module:
 - One with respect to $z[k]$
 - One with respect to $w[k]$

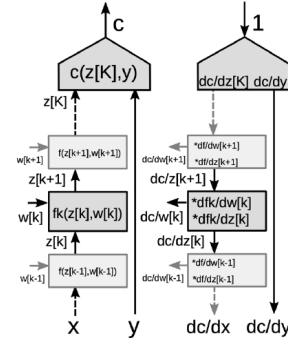


Figura 1.10: Backprop in un grafo multi-stadio: la rete duale (destra) esegue il forward pass “al contrario”, con ogni modulo che produce due Jacobiane ($*dfk/dw[k]$ e $*dfk/dz[k]$).

La backpropagation è quindi una *propagazione sul grafo duale*: ogni nodo è sostituito dal suo modulo derivata, e il gradiente scende dall’output verso l’input. Con $y, \bar{y} : [M \times 1]$ e $w : [N \times 1]$:

$$\underbrace{\frac{\partial C}{\partial w}}_{[1 \times N]} = \underbrace{\frac{\partial C}{\partial \bar{y}}}_{[1 \times M]} \cdot \underbrace{\frac{\partial \bar{y}}{\partial w}}_{[M \times N]} \quad (1.11)$$

Backprop = propagation through a transformed graph

- Derivative of composed functions

$$C(G(w))' = C'(G(w))G'(w)$$

$$\frac{\partial C(y, \bar{y})}{\partial w} = \frac{\partial C(y, \bar{y})}{\partial \bar{y}} \frac{\partial \bar{y}}{\partial w}$$

$$\frac{\partial C(y, \bar{y})}{\partial w} = \frac{\partial C(y, \bar{y})}{\partial \bar{y}} \frac{\partial G(x, w)}{\partial w}$$

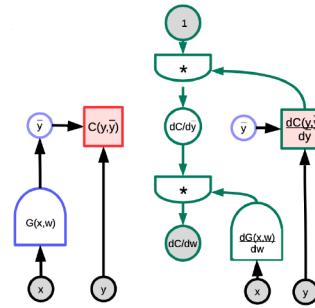


Figura 1.11: Backprop come propagazione sul grafo duale: $\partial C/\partial w = (\partial C/\partial \bar{y}) \cdot (\partial G/\partial w)$.

La backpropagation è applicabile a qualsiasi DAG. Se il grafo ha cicli (reti ricorrenti) occorre srotolarlo nel tempo (BPTT).

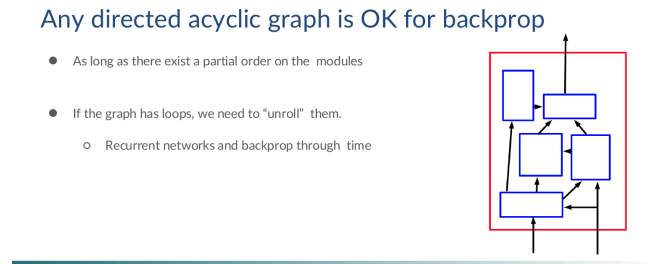


Figura 1.12: Un esempio di DAG con flusso di dati in avanti (freccie nere) e backprop (freccie rosse) che lo percorre in senso inverso.

1.5 Riepilogo

Concetto	Formula	Ruolo
Modello param.	$\bar{y} = G(x, w)$	Base di ogni sistema ML
Loss media	$\mathcal{L} = \frac{1}{P} \sum_p C$	Obiettivo
Blocco lineare	$s_{k+1} = w_k z_k$	Trasformazione affine
Blocco non-lin.	$z_k = h(s_k)$	Capacità non-lineare
Chain rule	$dc/ds = (dc/dz)h'(s)$	Motore backprop
Jacobiano	$(\partial z_g / \partial z_f)_{ij}$	General. vettoriale
DAG + backprop	$\partial c / \partial z_k = \partial c / \partial z_{k+1} \cdot \partial f_k / \partial z_k$	Algoritmo generale

Capitolo 2

Architetture di Base

Questo capitolo introduce i moduli fondamentali che estendono le reti neurali standard con operazioni moltiplicative e meccanismi di condivisione dei parametri. Questi concetti sono alla base di architetture moderne quali le reti convoluzionali, i meccanismi di attenzione e i modelli Mixture of Experts (Bishop and Bishop, 2024; LeCun and Canziani, 2021).

2.1 Moduli Moltiplicativi

Le reti neurali classiche calcolano combinazioni lineari degli ingressi seguite da una non-linearità. I **moduli moltiplicativi** generalizzano questa idea permettendo che i pesi stessi dipendano da un secondo ingresso, introducendo interazioni di ordine superiore tra feature.

2.1.1 Unità Sigma-Pi e layer quadratico

Definizione (Unità Sigma-Pi). In una rete standard il pre-input è $s_i = \sum_j w_{ij} x_j$. Nelle **unità Sigma-Pi** i pesi dipendono linearmente da un secondo vettore di ingresso $\mathbf{z}$:

$$w_{ij} = \sum_k u_{ijk} z_k \implies s_i = \sum_{j,k} u_{ijk} z_k x_j \quad (2.1)$$

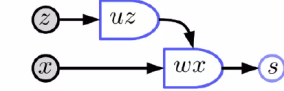
dove u_{ijk} sono i parametri effettivi del modello.

L'operazione è bilineare: lineare in $\mathbf{z}$ e in $\mathbf{x}$, ma quadratica nel vettore congiunto $(\mathbf{z}, \mathbf{x})$. La figura seguente mostra il diagramma computazionale dell'unità Sigma-Pi e, a destra, il modulo di attenzione come caso speciale [B6].

Multiplicative Modules

► Quadratic layer, product units, Sigma-Pi units

$$s_i = \sum_j w_{ij} x_j \text{ with } w_{ij} = \sum_k u_{ijk} z_k ; \text{equiv} : s_i = \sum_{jk} u_{ijk} z_k x_j$$



► Attention module

$$s_i = \sum_j w_j x_{ij} \quad w_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

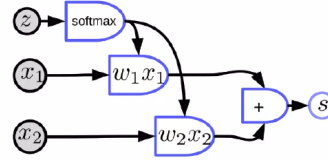


Figura 2.1: Sinistra: unità Sigma-Pi – il blocco uz calcola i pesi dinamici da z , che vengono applicati all'ingresso x nel blocco wx . Destra: modulo di attenzione – z passa per una softmax che produce i pesi w_1, w_2 per sommare le uscite pesate degli ingressi x_1, x_2 .

2.1.2 Modulo di attenzione

Il **meccanismo di attenzione** è il caso speciale in cui i pesi sono normalizzati tramite softmax, formando una distribuzione di probabilità:

$$s_i = \sum_j w_j x_{ij}, \quad w_j = \frac{e^{z_j}}{\sum_k e^{z_k}} = \text{softmax}(\mathbf{z})_j. \quad (2.2)$$

La rete sceglie dinamicamente su quale ingresso concentrarsi in base al contesto $\mathbf{z}$. Nel corso NYU ([LeCun and Canziani, 2021](#)), l'attenzione è interpretata come un meccanismo di *recupero soft* da una memoria associativa, che anticipa direttamente l'architettura Transformer ([Vaswani et al., 2017](#)).

2.1.3 Proprietà di differenziabilità

Il modulo moltiplicativo con selezione discreta ($z_1 \in \{0, 1\}$) non è differenziabile rispetto ai parametri della porta di controllo, ma rimane differenziabile rispetto a ingressi e uscite. La corrispondenza con la selezione matriciale è:

$$z_1=1, z_2=0 : \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x_1, \quad z_1=0, z_2=1 : \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x_2. \quad (2.3)$$

Multiplicative Modules

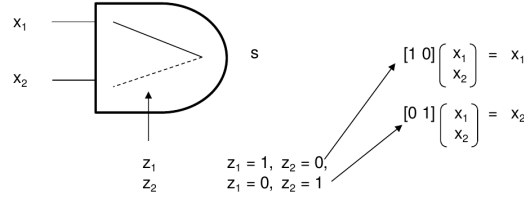
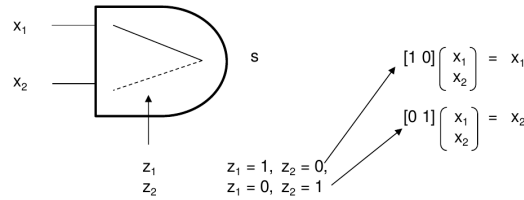


Figura 2.2: Selezione discreta: quando $z_1 = 1, z_2 = 0$ il modulo seleziona x_1 ; quando $z_1 = 0, z_2 = 1$ seleziona x_2 .

Multiplicative Modules



Not differentiable as it is, but it is differentiable with respect to inputs and outputs

Figura 2.3: Il modulo con selezione discreta non è differenziabile come tale, ma lo è rispetto agli ingressi e alle uscite: il rilassamento tramite softmax rende tutto addestrabile con backpropagation.

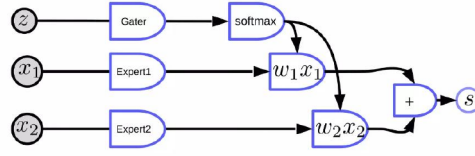
2.2 Mixture of Experts

Il **Mixture of Experts** (MoE) generalizza il modulo di attenzione applicandolo a intere reti esperto ([Jacobs et al., 1991](#)):

$$s_i = \sum_j w_j x_{ij}, \quad w_j = \text{softmax}(\mathbf{z})_j. \quad (2.4)$$

Mixture of Experts

► Attention module “Switches” expert networks



$$s_i = \sum_j w_j x_{ij} \quad w_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

● Example: <https://towardsdatascience.com/mixtral-8x7b-understanding-and-running-the-sparse-mixture-of-experts-0e3fc7fde818>

Figura 2.4: Mixture of Experts: la rete Gater produce i pesi softmax a partire da z ; Expert1 e Expert2 elaborano x_1 e x_2 ; le loro uscite pesate vengono sommate per produrre s .

Nelle architetture moderne come **Mixtral 8×7B** (AI, 2023), il gating è *sparse*: per ogni token vengono attivati solo k esperti su n (tipicamente $k=2$ su $n=8$), con *top-k routing*. Questo garantisce alta capacità parametrica a basso costo computazionale, ma introduce la sfida del *load balancing*.

2.3 Trasformazioni dei Parametri

Dato il vettore di parametri interno $\mathbf{u}$, i pesi operativi $\mathbf{w}$ sono ottenuti tramite una funzione differenziabile H :

$$\mathbf{w} = H(\mathbf{u}). \quad (2.5)$$

Applicando la regola della catena con learning rate η :

$$\mathbf{u} \leftarrow \mathbf{u} - \eta \frac{\partial H^T}{\partial \mathbf{u}} \frac{\partial C^T}{\partial \mathbf{w}} \quad (2.6)$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial H}{\partial \mathbf{u}} \frac{\partial H^T}{\partial \mathbf{u}} \frac{\partial C^T}{\partial \mathbf{w}} \quad (2.7)$$

con Jacobiane di dimensioni $[N_u \times N_w]$, $[N_w \times N_u]$, $[N_w \times 1]$.

Parameter Transformations

► When the parameter vector is the output of a function

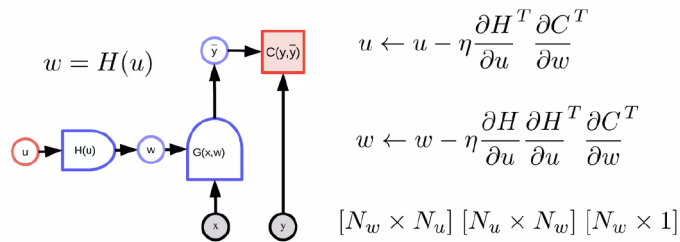


Figura 2.5: Schema della trasformazione parametrica: u passa per $H(u)$ per produrre w , che viene usato in $G(x, w)$. Il backward richiede la Jacobiana di H sia rispetto a u sia rispetto a w .

2.3.1 Weight sharing – condivisione dei pesi

Il caso più importante è il **weight sharing**: $H(\mathbf{u})$ replica alcune componenti di $\mathbf{u}$ in più componenti di $\mathbf{w}$, ad esempio $w_1 = w_2 = u_1$, $w_3 = w_4 = u_2$. In backpropagation i gradienti di tutte le copie vengono **sommati**:

$$\frac{\partial C}{\partial u_k} = \sum_{j=1}^m \frac{\partial C}{\partial w_{ij}}. \quad (2.8)$$

Simple parameter transform: weight sharing

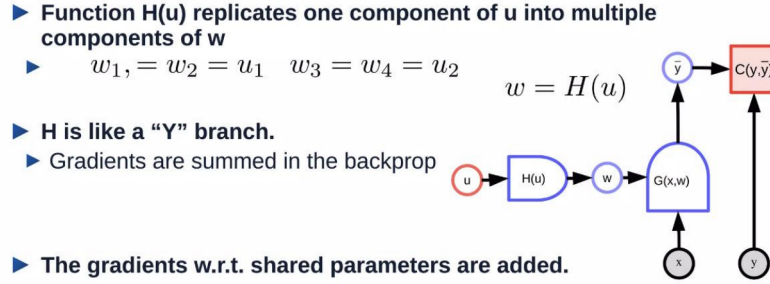


Figura 2.6: Weight sharing: $H(u)$ si comporta come un “ramo Y”, replicando il parametro u_1 nei pesi $w_1 = w_2$ e u_2 nei pesi $w_3 = w_4$. I gradienti w.r.t. i parametri condivisi vengono sommati.

2.3.2 Shared weights per la rilevazione di motif

Applicando lo stesso $\mathbf{w}$ a finestre diverse dell’input e raccogliendo con MAX pooling, si ottiene un rilevatore di motif invariante per traslazione. Questo schema è esattamente una *convoluzione 1D* seguita da global max-pooling (LeCun et al., 1998).

Shared Weights for Motif Detection

- ▶ **Detecting motifs anywhere on an input**

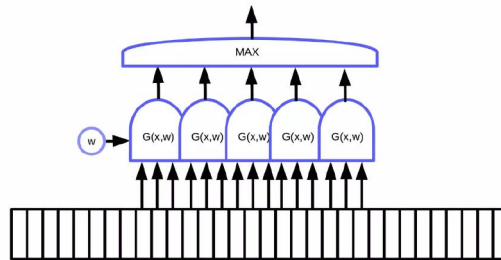


Figura 2.7: Rilevazione di motif con shared weights: i moduli $G(\mathbf{x}, \mathbf{w})$ identici (stesso filtro $\mathbf{w}$) scansionano l’input; il MAX pooling finale segnala la presenza del motif indipendentemente dalla posizione.

Le CNN (LeCun et al., 1998) estendono questa idea a 2D, con K filtri in parallelo e più layer per gerarchie di feature. LeCun e Canziani (LeCun and Canziani, 2021)

sottolineano come il weight sharing riduca drasticamente i parametri, costituendo il principale *inductive bias* delle CNN.

2.4 Riepilogo

Modulo	Formula chiave	Applicazione
Unità Sigma-Pi	$s_i = \sum_{jk} u_{ijk} z_k x_j$	Interaz. ordine 2
Attention module	$w_j = \text{softmax}(\mathbf{z})_j$	Selezione dinamica
Mixture of Experts	attention su reti esperto	Routing per token
Param. Transform	$\mathbf{w} = H(\mathbf{u})$	Vincoli strutturali
Weight Sharing	$w_i = w_j = u_k$	CNN, RNN
Motif Detection	$G(\mathbf{x}, \mathbf{w}) + \text{MAX}$	Invarianza trasl.

I moduli moltiplicativi introducono interazioni di secondo ordine tra ingresso e contesto. Il meccanismo di attenzione ne è il caso differenziabile per eccellenza, e costituisce il mattone fondamentale dei Transformer (Vaswani et al., 2017). La condivisione dei pesi è la chiave per apprendere da dataset anche limitati.

Capitolo 3

Learning Representations

Questo capitolo affronta una domanda fondamentale: *perché le reti profonde funzionano meglio di quelle shallow?* La risposta passa per il concetto di **rappresentazione**: trasformare i dati grezzi in uno spazio in cui il problema diventi più facile da risolvere (Bishop and Bishop, 2024; LeCun and Canziani, 2021).

3.1 Limiti dei Classificatori Lineari

Un **classificatore lineare** assegna la classe in base al segno di una combinazione lineare delle feature:

$$\bar{y} = \text{sign} \left(\sum_{i=1}^N w_i x_i + b \right). \quad (3.1)$$

Geometricamente, il classificatore partiziona lo spazio in due semispazi separati dall'iperpiano $\sum_i w_i x_i + b = 0$. Il vettore $\mathbf{w}$ è la normale all'iperpiano e ne determina l'orientamento; $-b/w_1$ ne fissa l'intercetta.

Linear Classifiers and their limitations

- Linear classifier $\bar{y} = \text{sign}(\sum_{i=1}^N w_i x_i + b)$
- Partitions the space into two half spaces separated by the hyperplane: $\sum_{i=1}^N w_i x_i + b = 0$

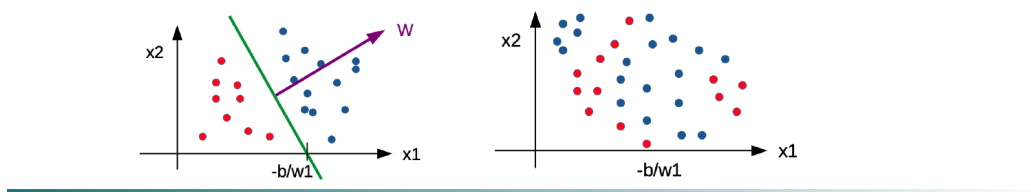


Figura 3.1: Classificatore lineare in 2D. Sinistra: caso separabile linearmente – l'iperpiano (retta verde) divide correttamente le due classi. Destra: caso non separabile linearmente – nessuna retta può separare i punti rossi da quelli blu.

Il limite fondamentale dei classificatori lineari è formalizzato dal **Teorema di Cover (1966)**: la probabilità che una dicotomia casuale di P punti in N dimensioni sia

linearmente separabile tende a zero quando $P \gg N$.

Number of linearly separable dichotomies

- The probability that a dichotomy over P points in N dimensions is linearly separable goes to zero as P gets larger than N
[Cover's theorem 1966]

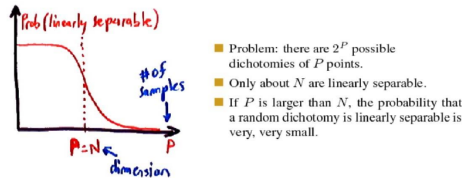


Figura 3.2: Teorema di Cover: la probabilità di separabilità lineare crolla bruscamente non appena il numero di campioni P supera la dimensionalità N . Esistono 2^P possibili dicotomie, ma solo circa N sono linearmente separabili.

In pratica: con $P \gg N$, quasi nessuna dicotomia casuale è linearmente separabile. Per classificare dati reali complessi, occorre una **trasformazione non lineare** che renda il problema separabile.

3.2 La Soluzione: Rappresentazioni (Feature)

La soluzione è costruire un **estrattore di feature** che trasformi l'input grezzo in una rappresentazione in cui un classificatore lineare possa operare efficacemente. L'estrattore deve essere *non lineare*: una trasformazione lineare non cambia la separabilità del problema.

Solution: representations (a.k.a. features)

- Extracting relevant features from the raw input
- Computing good representations of the input
- The feature extractor must be non-linear
- Simple solution: expand the dimension non-linearly
But how?

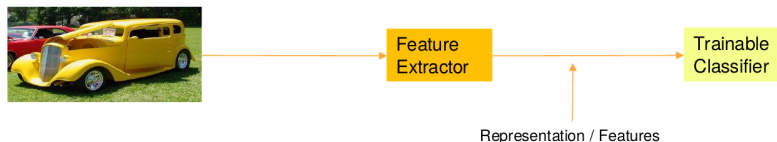


Figura 3.3: Soluzione con rappresentazioni: il blocco Feature Extractor trasforma l'input grezzo in una rappresentazione/feature; su queste opera un classificatore trainable. Nel DL anche l'estrattore è addestrato end-to-end.

Il principio di base è **espandere la dimensionalità** in modo non lineare: in uno spazio di dimensione maggiore, punti che erano non separabili diventano separabili con alta probabilità (per il Teorema di Cover, la probabilità di separabilità cresce con N).

3.2.1 Idee per l'estrazione generica di feature

Esistono diverse strategie “generiche” per costruire feature non lineari [NYUW2]:

- **Space tiling:** si suddivide lo spazio di input in regioni e si usano indicatori di appartenenza come feature.
- **Random projections:** si proietta l'input su vettori casuali seguiti da non-linearità (es. ReLU). Alla base dei *reservoir computing* e delle *random features* per kernel approximation.
- **Feature monomiali** (cross-products): il feature extractor calcola prodotti di coppie (o triple, ecc.) di variabili di input; un classificatore lineare su queste feature calcola un polinomio dell'input.
- **Radial Basis Functions (RBF):** feature del tipo $\phi_k(x) = \exp(-\|x - c_k\|^2 / 2\sigma^2)$, con centri c_k fissati o appresi.
- **Kernel machines:** generalizzano le RBF attraverso funzioni kernel $K(x^j, x)$; le SVM ne sono l'esempio principale.

3.2.2 Feature monomiali e limiti

Monomial features

- Feature extractor computes cross products of input variables
- A linear classifier on top computes a polynomial of input variables
- generalizable to degree d
- Unfortunately impractical for large d
- Number of features is d choose N , which grows like N^d
- But $d=2$ is used a lot in “attention” circuits.



Figura 3.4: Feature monomiali: aggiungendo il termine quadratico x_1x_2 (o più in generale prodotti di grado d) come feature, un classificatore lineare può separare pattern che erano non lineari nell'input originale – ad esempio boundary ellittici o a forma di C.

Le feature monomiali di grado d su N variabili producono $\binom{N+d}{d} \sim N^d$ feature: questo numero cresce esponenzialmente con d , rendendo il metodo impraticabile per grandi d . Tuttavia, il caso $d = 2$ è molto usato nei circuiti di *attention* (si veda il Cap. 2), in quanto corrisponde esattamente alle unità Sigma-Pi.

3.3 Reti Shallow vs Reti Profonde

3.3.1 Le reti shallow sono approssimatori universali

Un risultato teorico fondamentale afferma che una rete neurale a 2 layer con numero sufficiente di unità nascoste può approssimare qualsiasi funzione continua con precisione arbitraria ^[B6]. Lo stesso vale per le **Support Vector Machine** con kernel:

$$G(X, \alpha) = \sum_j \alpha_j K(X^j, X), \quad (3.2)$$

dove il primo layer calcola i kernel $K(X^j, X)$ usando i campioni di training come template, e il secondo layer è una combinazione lineare con pesi α_j .

Shallow networks are universal approximators!

- SVMs and Kernel methods
 - Layer1: kernels; layer2: linear
 - The first layer is "trained" with the simplest unsupervised method ever devised: using the samples as templates for the kernel functions.
- 2-layer neural nets
 - Layer1: dot products + non-linear function;
 - Layer2: linear
- But few useful functions can be efficiently represented with only two layers of reasonable size.

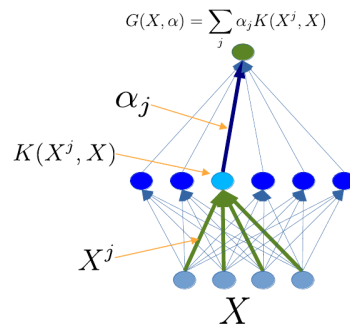


Figura 3.5: Architettura di una kernel machine (SVM): il primo layer calcola $K(X^j, X)$ per ogni campione di training X^j ; il secondo layer combina linearmente le attivazioni con pesi α_j .

3.3.2 Perché servono architetture profonde?

Se le reti shallow sono universali, perché usare reti profonde? Il **dilemma del teorico** pone esattamente questa domanda.

Do we really need deep architectures?

- **Theoretician's dilemma:** "We can approximate any function as close as we want with shallow architecture. Why would we need deep ones?"

$$y = \sum_{i=1}^P \alpha_i K(X, X^i) \quad y = F(W^{-1} \cdot F(W^0 \cdot X))$$
 - kernel machines (and 2-layer neural nets) are "universal".
- Deep learning machines

$$y = F(W^K \cdot F(W^{K-1} \cdot F(\dots F(W^0 \cdot X) \dots)))$$
- **Deep machines are more efficient for representing certain classes of functions**, particularly those involved in visual recognition
 - they can represent more complex functions with less "hardware"
- **We need an efficient parameterization of the class of functions that are useful for "AI" tasks (vision, audition, NLP...)**

Figura 3.6: Confronto tra reti shallow e profonde: una rete shallow (kernel machine o 2-layer net) può rappresentare qualunque funzione, ma richiede un numero esponenziale di unità per funzioni complesse; una rete profonda $y = F(W^K \cdot F(\dots F(W^0 \cdot X) \dots))$ le rappresenta in modo molto più efficiente.

La risposta è nell'**efficienza della rappresentazione**: le reti profonde possono rappresentare certe classi di funzioni – in particolare quelle coinvolte nel riconoscimento visivo – con molti meno parametri rispetto alle reti shallow (Bishop and Bishop, 2024). Come argomenta LeCun and Canziani (2021), una rete profonda *scambia spazio con tempo* (o ampiezza con profondità): più layer sequenziali ma meno hardware parallelo.

3.4 Invariant Feature Learning

L'idea di base per imparare feature invarianti è la seguente [NYU W2]:

1. **Espandere** l'input non linearmente in uno spazio ad alta dimensionalità (feature instabili ma discriminative).
2. **Aggregare** regioni del nuovo spazio (pooling, proiezione, riduzione di dimensionalità) per ottenere feature stabili e invarianti.

Basic Idea for Invariant Feature Learning

- **Embed the input non-linearly into a high(er) dimensional space**
 - In the new space, things that were non separable may become separable
- **Pool regions of the new space together**
 - Bringing together things that are semantically similar. Like pooling.

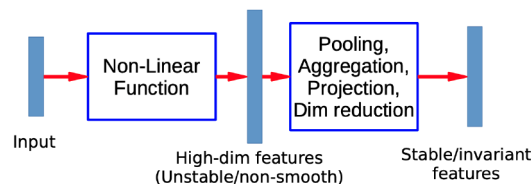


Figura 3.7: Schema dell'invariant feature learning: la funzione non lineare espande l'input in feature ad alta dimensione (instabili), che vengono poi aggregate tramite pooling/aggregazione/dim. reduction per produrre feature stabili e invarianti.

3.5 L'Ipotesi del Manifold

Definizione (Manifold Hypothesis). *I dati naturali vivono in un **manifold non lineare a bassa dimensione** immerso nello spazio ad alta dimensione. Le variabili nei dati naturali sono mutuamente dipendenti, per cui la variabilità intrinseca è molto più bassa della dimensione nominale.*

Discovering the Hidden Structure in High-Dimensional Data: The manifold hypothesis

- Learning Representations of Data:
 - Discovering & disentangling the independent explanatory factors
- The Manifold Hypothesis:
 - Natural data lives in a low-dimensional (non-linear) manifold
 - Because variables in natural data are mutually dependent

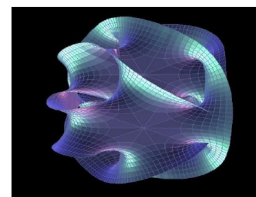


Figura 3.8: L'ipotesi del manifold: le immagini di volti (sinistra) variano in espressione, posa, illuminazione – ma questa variabilità appartiene a un manifold a bassa dimensione (destra), non a tutto lo spazio dei pixel.

Esempio: immagini di volti. Un'immagine 1000×1000 pixel ha 10^6 dimensioni. Tuttavia un volto è descritto da soli 3 coordinate cartesiane, 3 angoli di Eulero e meno di 50 muscoli facciali: il manifold delle immagini di un volto ha quindi meno di 56 dimensioni effettive.

Discovering the Hidden Structure in High-Dimensional Data

- Example: all face images of a person
 - 1000×1000 pixels = 1,000,000 dimensions
 - But the face has 3 Cartesian coordinates and 3 Euler angles And humans have less than about 50 muscles in the face
 - Hence the manifold of face images for a person has <56 dimensions
- The perfect representations of a face image:
 - Its coordinates on the face manifold
 - Its coordinates away from the manifold
- We do not have good and general methods to learn functions that turns an image into this kind of representation



$\begin{bmatrix} 1.2 \\ -3 \\ 0.2 \\ 2 \dots \end{bmatrix}$

Face/not face
 Pose
 Lighting
 Expression

Figura 3.9: La rappresentazione ideale di un'immagine facciale ne estrae le coordinate sul manifold (pose, lighting, expression...) come vettore compatto. Non disponiamo ancora di metodi generali per imparare questa funzione.

3.6 Disentangling e Apprendimento delle Rappresentazioni

L'obiettivo del **representation learning** è scoprire e *disintrecciare* (disentangle) i fattori di variazione indipendenti che generano i dati.

Disentangling factors of variation

- The Ideal Disentangling Feature Extractor

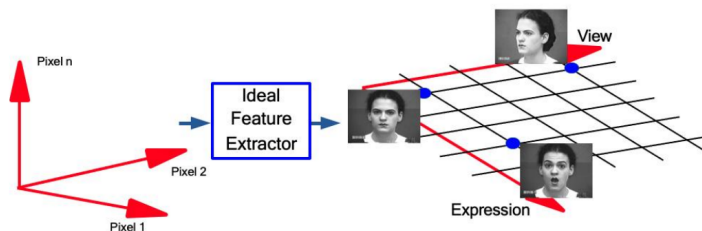


Figura 3.10: L'ideal feature extractor mappa lo spazio dei pixel (sinistra) nel manifold dei volti (destra), dove le coordinate corrispondono a fattori interpretabili come “view” (punto di vista) e “expression” (espressione). Ogni punto blu nel manifold corrisponde a un'immagine specifica.

Un esempio concreto di manifold appreso è mostrato nel lavoro di Hadsell et al. (CVPR 2006): immagini di aeroplani in diverse rotazioni sono disposte su una curva chiusa nell'embedding space, rispecchiando la struttura continua della rotazione.

Data Manifold

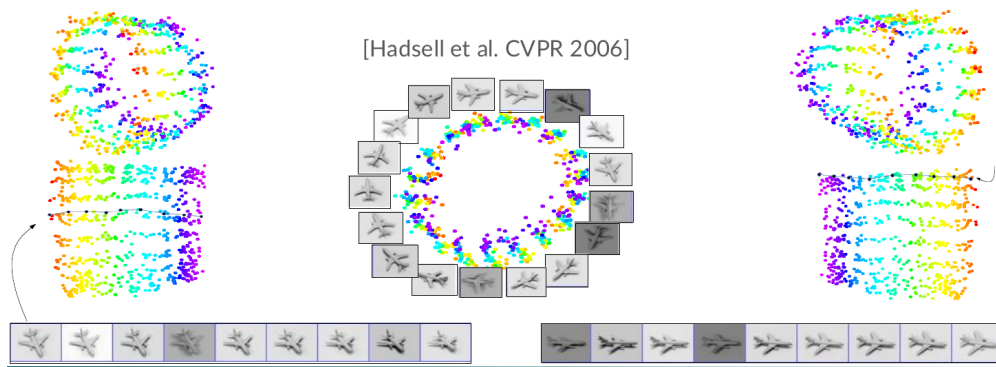


Figura 3.11: Data manifold appreso [Hadsell et al., CVPR 2006]: le immagini di aeroplani in diverse orientazioni formano una struttura ciclica coerente nello spazio di embedding – punti vicini corrispondono a immagini visivamente simili.

3.7 Architetture Multilayer = Struttura Compositiva

I dati naturali hanno una **struttura compositiva**: sono costruiti gerarchicamente da elementi via via più astratti. Le reti neurali multilayer sfruttano questa struttura, con

ogni layer che apprende un livello di astrazione (LeCun and Canziani, 2021).

Multilayer Architectures == Compositional Structure of Data

- Natural data is compositional => it is efficiently representable hierarchically

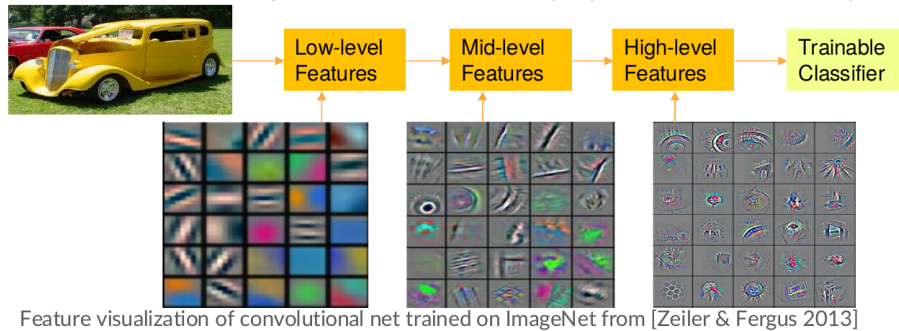


Figura 3.12: Le architetture multilayer corrispondono alla struttura compositiva dei dati. La visualizzazione delle feature di una CNN addestrata su ImageNet [Zeiler & Fergus, 2013] mostra: layer bassi = edge e texture, layer intermedi = parti di oggetti, layer alti = concetti semantici.

Multilayer Architecture == Hierarchical representation

- Hierarchy of representations with increasing level of abstraction Each stage is a kind of trainable feature transform
- Image recognition
 - Pixel → edge → texton → motif → part → object
- Text
 - Character → word → word group → clause → sentence → story
- Speech
 - Sample → spectral band → sound → ... → phone → phoneme → word

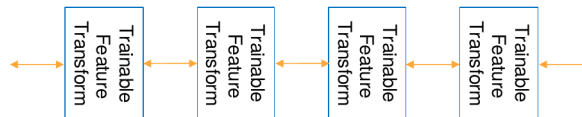


Figura 3.13: Architettura multilayer come rappresentazione gerarchica: ogni blocco “Trainable Feature Transform” corrisponde a un livello di astrazione. Le frecce bidirezionali indicano che ogni modulo può essere addestrato con backpropagation.

Gerarchia di rappresentazioni nei diversi domini. La gerarchia di astrazioni varia a seconda del dominio:

- **Immagini:** pixel → edge → texton → motif → parte → oggetto.
- **Testo:** carattere → parola → gruppo di parole → proposizione → frase → storia.
- **Audio:** campione → banda spettrale → suono → ... → fonema → parola.

L’efficienza delle architetture profonde è spiegata da LeCun and Canziani (2021) e da Bengio & LeCun (2007): una rete profonda *scambia ampiezza con profondità* – più layer sequenziali ma meno hardware parallelo – riuscendo a rappresentare funzioni complesse con molti meno parametri rispetto a una rete shallow.

3.8 Riepilogo

Concetto	Idea chiave	Implicazione
Classificatore lineare	$\bar{y} = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$	Limite: non sep. lineare
Teorema di Cover	Prob. sep. $\rightarrow 0$ se $P \gg N$	Serve non-linearità
Feature monomiali	Cross-products di grado d	$O(N^d)$ feature
Reti shallow	Approssimatori universali	Ma inefficienti
Reti profonde	$y = F(W^K \cdot F(\dots))$	Più efficienti
Manifold Hypothesis	Dati su manifold bassa-dim.	Motiva il DL
Disentangling	Separare i fattori di variazione	Obiettivo del repr. learning
Struttura compositiva	Pixel $\rightarrow$ edge $\rightarrow$ parte $\rightarrow$ obj	Giustifica i multilayer

Le reti neurali profonde sono giustificate non solo dall'approssimazione universale (che vale anche per reti shallow), ma dalla loro **efficienza parametrica** nel rappresentare le funzioni utili per i compiti AI. L'ipotesi del manifold e la struttura compositiva dei dati naturali forniscono le basi teoriche per questa efficienza.

Capitolo 4

Activation Functions

Le **funzioni di attivazione** sono il cuore della non-linearità nelle reti neurali. Senza di esse, una rete a più strati si ridurrebbe a una semplice trasformazione lineare, indipendentemente dalla profondità (Bishop and Bishop, 2024). Come osserva Canziani nelle note NYU-DL, ogni layer di una rete tradizionale alterna un blocco *lineare* $\mathbf{s}_{k+1} = W_k \mathbf{z}_k$ a un blocco *non lineare* $\mathbf{z}_k = h(\mathbf{s}_k)$, dove h è appunto la funzione di attivazione. La scelta di h influenza profondamente la velocità di convergenza, la stabilità del gradiente e le capacità espressive della rete.

4.1 Il Problema del Vanishing Gradient

Il principale limite delle funzioni di attivazione **saturanti** (es. Sigmoid, Tanh) è il cosiddetto **vanishing gradient problem**. La retropropagazione moltiplica gradienti attraverso ogni layer: se ogni fattore è $\ll 1$, il segnale si azzerava esponenzialmente procedendo verso i layer iniziali.

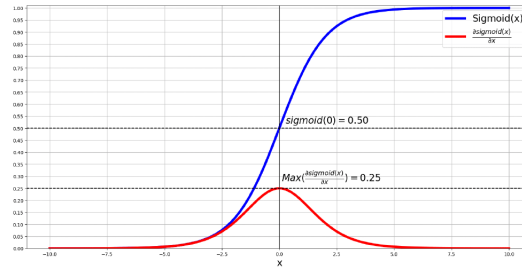
Per la Sigmoid, il gradiente massimo è:

$$\max_x \frac{\partial \sigma(x)}{\partial x} = \frac{1}{4} = 0,25 \quad (\text{raggiunto in } x = 0), \quad (4.1)$$

e si avvicina a zero sia per $x \rightarrow +\infty$ sia per $x \rightarrow -\infty$. In una rete con L layer, il gradiente che torna al primo layer scala come $(0,25)^L$: già per $L = 10$ si ottiene $\approx 10^{-6}$.



The problem of vanishing gradients



Deep Learning - Vito Walter Anelli

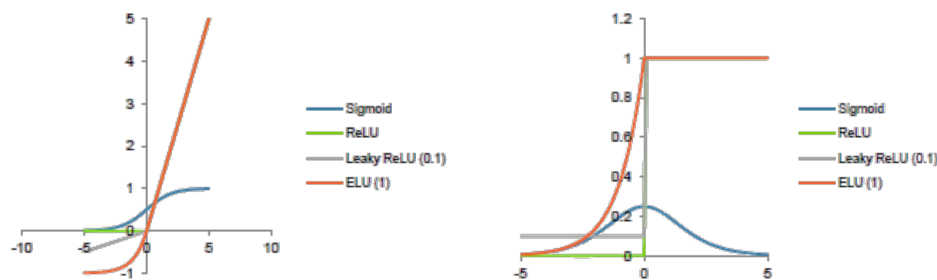
Figura 4.1: Sigmoid e sua derivata. Il valore $\sigma(0) = 0,5$ è il punto di inflessione e il gradiente massimo è 0,25. Per input molto grandi o molto piccoli la derivata tende a zero, bloccando il flusso del gradiente nei layer profondi.

La soluzione più efficace è passare a funzioni **non saturanti** per i layer interni, riservando funzioni saturanti solo all'output quando la struttura del problema lo richiede (es. classificazione binaria, gating nelle LSTM).

4.2 Panoramica: Saturating vs Non-Saturating



Non-saturating activations



Deep Learning - Vito Walter Anelli

Figura 4.2: Confronto visivo tra funzioni di attivazione saturanti e non saturanti (Sigmoid, ReLU, Leaky ReLU, ELU) nelle loro curve e nelle rispettive derivate. Le funzioni non saturanti mantengono un gradiente costante nella regione positiva, resolvendo il vanishing gradient.

4.3 Famiglia ReLU

4.3.1 ReLU – `nn.ReLU()`

La **Rectified Linear Unit** (ReLU) è oggi la funzione di attivazione di default per i layer nascosti delle reti profonde (Bishop and Bishop, 2024):

$$\text{ReLU}(x) = (x)^+ = \max(0, x). \quad (4.2)$$

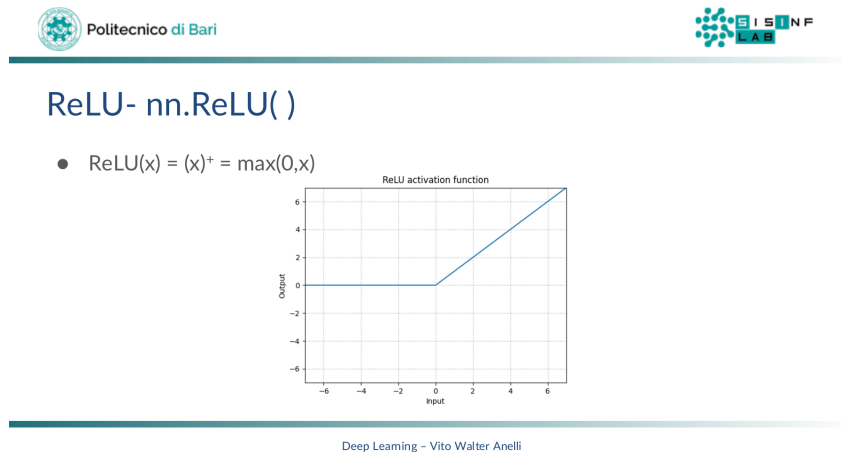


Figura 4.3: ReLU: uscita identità per $x \geq 0$, zero per $x < 0$. La derivata è discontinua in $x = 0$ ma il gradiente è 1 per tutti gli input positivi, eliminando completamente il vanishing gradient nel ramo attivo.

Proprietà chiave.

- **Efficienza computazionale:** riduce a un semplice confronto con zero.
- **Gradiente non saturante:** la derivata è 1 per $x > 0$, evitando il vanishing gradient.
- **Sparsità:** i neuroni con input negativo producono esattamente zero, inducendo rappresentazioni sparse che migliorano la generalizzazione.
- **Dead ReLU problem:** se un neurone riceve input sempre negativi (es. dopo un grande aggiornamento del gradiente), il suo gradiente sarà sempre zero e il neurone cessa di apprendere. Le varianti seguenti mitigano questo problema.

4.3.2 Leaky ReLU – `nn.LeakyReLU()`

$$\text{LeakyReLU}(x) = \begin{cases} x, & \text{se } x \geq 0 \\ a_{\text{negative_slope}} \times x, & \text{altrimenti} \end{cases} \quad (4.3)$$

dove $a_{\text{negative_slope}}$ è una piccola costante (tipicamente 0,01).

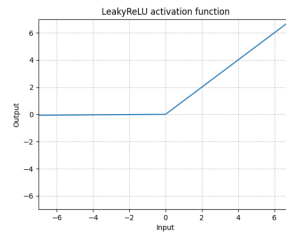


Politecnico di Bari



LeakyReLU - nn.LeakyReLU()

$$\text{LeakyReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ a_{\text{negative.slope}} \times x, & \text{otherwise} \end{cases}$$



Deep Learning - Vito Walter Anelli

Figura 4.4: Leaky ReLU: introduce una piccola pendenza nella regione negativa ($a_{\text{neg}} = 0,1$ nella figura), consentendo al gradiente di fluire anche per neuroni con input negativo e risolvendo il *dead neuron* problem.

Rispetto a ReLU, Leaky ReLU garantisce un gradiente non nullo anche per $x < 0$, mantenendo i neuroni attivi durante l'intero training. Il coefficiente a è fisso e scelto prima del training.

4.3.3 PReLU – nn.PReLU()

$$\text{PReLU}(x) = \begin{cases} x, & \text{se } x \geq 0 \\ ax, & \text{altrimenti} \end{cases} \quad (4.4)$$



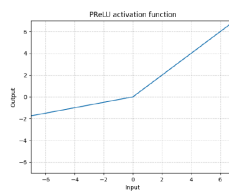
Politecnico di Bari



Parametric PReLU - nn.PReLU()

$$\text{PReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ ax, & \text{otherwise} \end{cases}$$

Here a is a learnable parameter. When called without arguments, `nn.PReLU()` uses a single parameter a across all input channels. If called with `nn.PReLU(Channels)`, a separate a is used for each input channel.



Deep Learning - Vito Walter Anelli

Figura 4.5: Parametric ReLU (PReLU): il coefficiente a è un **parametro apprendibile** con la retropropagazione. Chiamato senza argomenti, `nn.PReLU()` usa un unico parametro a condiviso; chiamato con `nn.PReLU(C)`, usa un a distinto per ognuno degli C canali in input.

La distinzione tra le tre varianti è riassumibile come segue: **LReLU** usa un a fisso, **PReLU** apprende a , **RReLU** campiona a casualmente durante il training.

4.3.4 RReLU – nn.RReLU()

$$\text{RReLU}(x) = \begin{cases} x, & \text{se } x \geq 0 \\ ax, & \text{altrimenti} \end{cases} \quad (4.5)$$

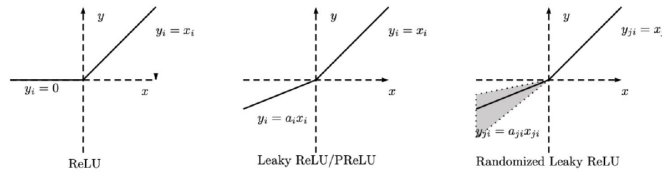


Politecnico di Bari



Randomized LeakyReLU - RReLU - nn.RReLU()

$$\text{RReLU}(x) = \begin{cases} x, & \text{if } x \geq 0 \\ ax, & \text{otherwise} \end{cases}$$



Note that for RReLU, a is a **random variable** that keeps samplings in a given range during training, and remains fixed during testing. For PReLU, a is also learned. For Leaky ReLU, a is fixed.

Deep Learning – Vito Walter Anelli

Figura 4.6: Confronto visivo tra ReLU, Leaky ReLU/PReLU e Randomized Leaky ReLU (RReLU). In RReLU, il parametro a_{ji} è campionato uniformemente da un intervallo $[l, u]$ durante il training; al test viene usato il valor medio $(l + u)/2$. Questo effetto di randomizzazione funge da regolarizzazione implicita.



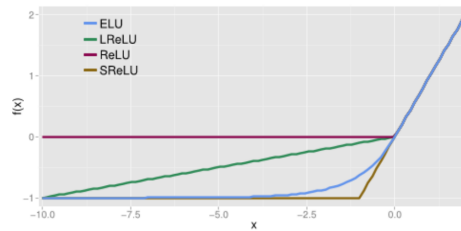
Politecnico di Bari



Differences between LReLU, PReLU and RReLU

LReLU, PReLU and RReLU provide with negative values in the negative part of the respective functions.

- LReLU is using a small tilted slope
- PReLU learns the steepness of this slope.
- RReLU sets this slope to be a random value between an upper and lower bound during the training and an average of these bounds during the testing.



Deep Learning – Vito Walter Anelli

Figura 4.7: Differenze nella regione negativa tra ELU, LReLU, ReLU e SReLU. LReLU usa una pendenza fissa piccola; PReLU la apprende; RReLU la randomizza durante il training e la fissa alla media al test.

4.3.5 ReLU6 – nn.ReLU6()

$$\text{ReLU6}(x) = \min(\max(0, x), 6). \quad (4.6)$$

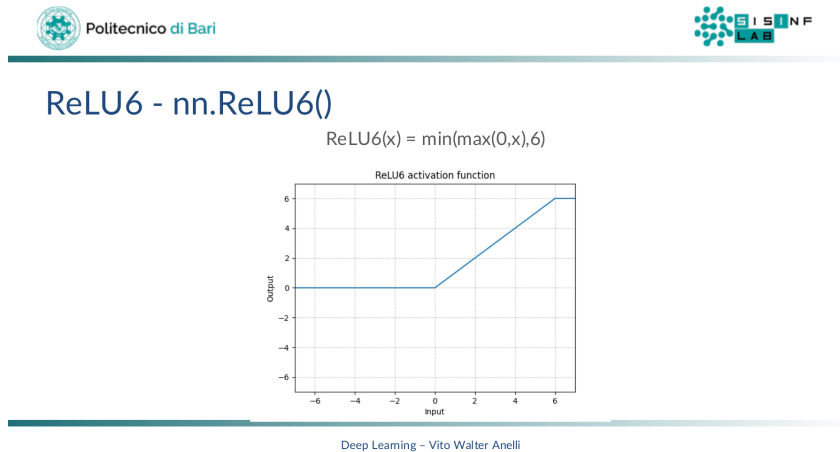


Figura 4.8: ReLU6: ReLU con saturazione superiore a 6. Progettata per ambienti a precisione ridotta (es. mobile/quantizzazione), previene attivazioni molto grandi che possono degradare la rappresentazione in virgola mobile a bassa precisione.

4.4 Famiglia ELU

4.4.1 ELU – `nn.ELU()`

La **Exponential Linear Unit** risolve sia il dead neuron problem sia il problema delle medie non centrate in zero (Bishop and Bishop, 2024):

$$\text{ELU}(x) = \max(0, x) + \min(0, \alpha \cdot (\exp(x) - 1)). \quad (4.7)$$

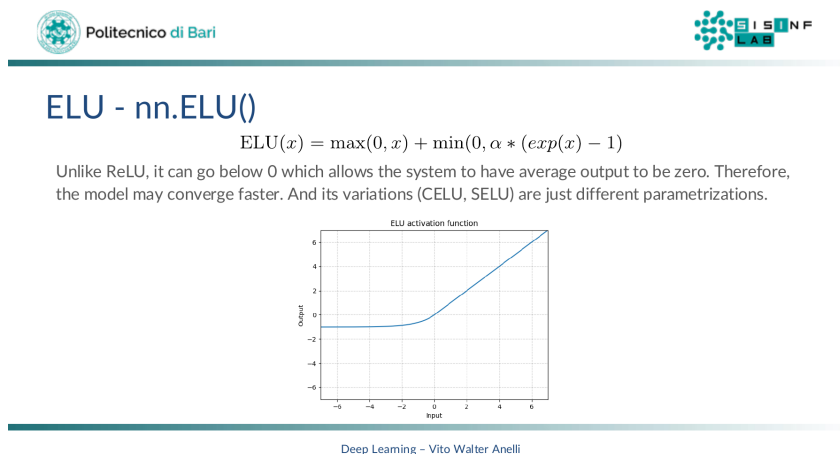


Figura 4.9: ELU: a differenza di ReLU, può produrre valori negativi, permettendo all'output medio del layer di essere vicino a zero. Questo centra le attivazioni e accelera la convergenza. Le varianti CELU e SELU sono reparametrizzazioni di ELU.

Il valore α controlla il limite inferiore della regione negativa (di default $\alpha = 1$). Avere output con media vicina a zero ha lo stesso beneficio del *Batch Normalization*: il gradiente non è sistematicamente distorto, e la convergenza è più rapida.

4.4.2 CELU – nn.CELU()

$$\text{CELU}(x) = \max(0, x) + \min\left(0, \alpha \cdot \left(\exp\left(\frac{x}{\alpha}\right) - 1\right)\right). \quad (4.8)$$

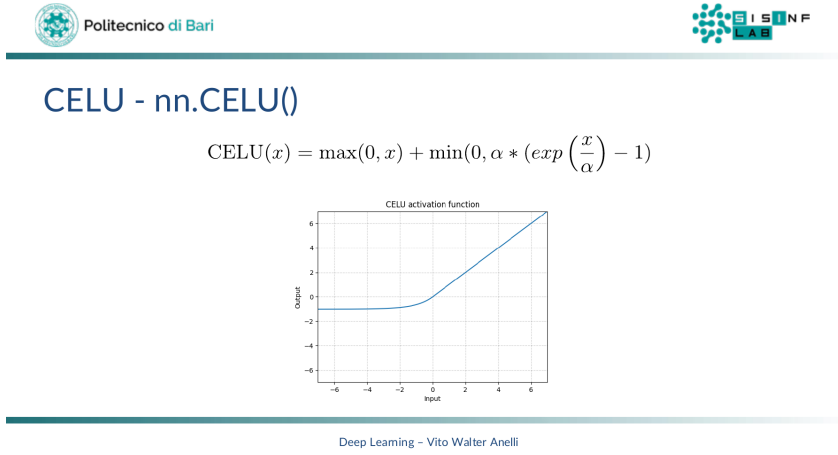


Figura 4.10: CELU (Continuously Differentiable ELU): reparametrizza ELU in modo da garantire la derivabilità continua in $x = 0$ per qualsiasi valore di α , migliorando la stabilità numerica del training.

4.4.3 SELU – nn.SELU()

$$\text{SELU}(x) = \text{scale} \cdot \left(\max(0, x) + \min\left(0, \alpha \cdot (\exp(x) - 1)\right)\right), \quad (4.9)$$

con $\text{scale} \approx 1,0507$ e $\alpha \approx 1,6733$ (valori calcolati analiticamente da Klambauer et al., 2017).

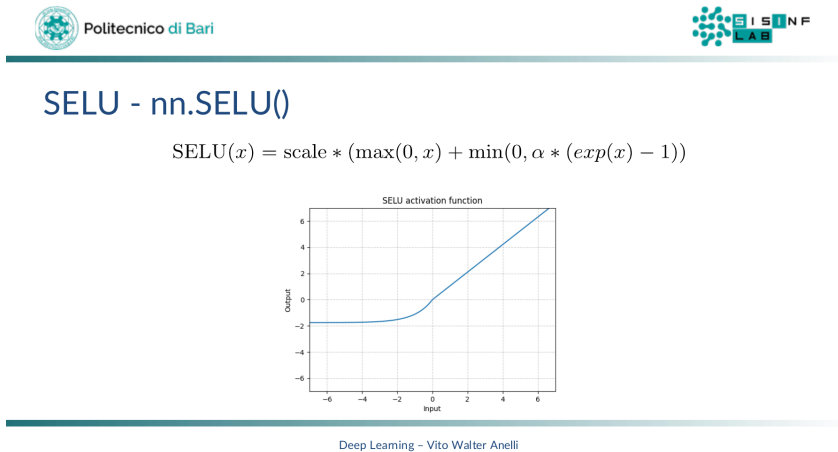


Figura 4.11: SELU (Scaled ELU): aggiunge un fattore di scala alla ELU. Con le costanti α e scale opportune, la SELU garantisce *self-normalizing* properties: attivazioni con media 0 e varianza 1 si propagano stabili attraverso layer arbitrariamente profondi, senza necessità di Batch Normalization.

4.5 Softplus e GELU

4.5.1 Softplus – `nn.Softplus()`

$$\text{Softplus}(x) = \frac{1}{\beta} \cdot \log(1 + \exp(\beta x)). \quad (4.10)$$

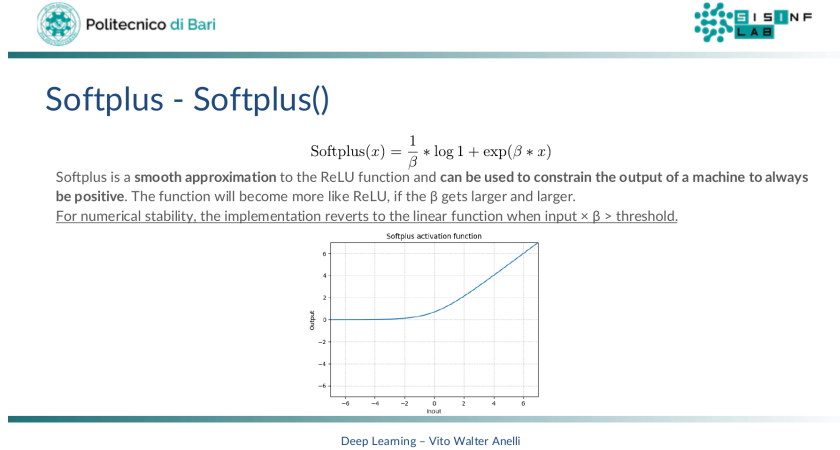


Figura 4.12: Softplus: approssimazione **smooth** di ReLU, con derivata continua in tutto $\mathbb{R}$. All’aumentare di β converge a ReLU. Può essere usata per *vincolare l’output a essere sempre positivo*. Per stabilità numerica, l’implementazione torna alla funzione lineare quando $x \cdot \beta > \text{threshold}$.

La Softplus è la primitiva della Sigmoid: $\frac{d}{dx} \text{Softplus}(x) = \sigma(x)$, il che ne chiarisce il ruolo teorico come “versione continua” di ReLU.

4.5.2 GELU – `nn.GELU()`

$$\text{GELU}(x) = x \cdot \Phi(x), \quad (4.11)$$

dove $\Phi(x)$ è la *Cumulative Distribution Function* della distribuzione gaussiana standard.



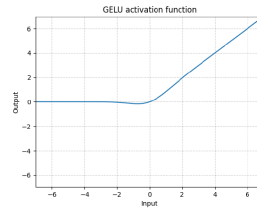
Politecnico di Bari



GELU - nn.GELU()

$$GELU(x) = x * \Phi(x)$$

where $\Phi(x)$ is the Cumulative Distribution Function for Gaussian Distribution. **Non-Monotonic!!** (e.g., $\sin()$)



Deep Learning - Vito Walter Anelli

Figura 4.13: GELU (Gaussian Error Linear Unit): funzione **non monotona**, a differenza di ReLU e delle sue varianti. Nella regione $x < 0$ l'output decresce, poi risale, imitando il comportamento del dropout stocastico. È la funzione di attivazione predefinita nei Transformer (BERT, GPT) per le sue proprietà di regolarizzazione implicita.

L'approssimazione pratica spesso utilizzata è:

$$GELU(x) \approx 0,5 x \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} (x + 0,044715 x^3) \right] \right). \quad (4.12)$$

4.6 Funzioni Saturanti

4.6.1 Sigmoid – nn.Sigmoid()

$$\text{Sigmoid}(x) = \sigma(x) = \frac{1}{1 + \exp(-x)}. \quad (4.13)$$



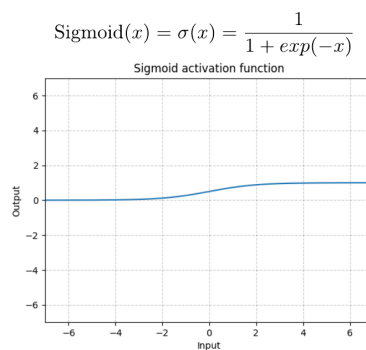
Sigmoid - nn.Sigmoid()

If we stack sigmoids in many layers, it may be inefficient for the system to learn and requires careful initialization.

This is because if the input is very large or small, the gradient of the sigmoid function is close to 0.

In this case, there is no gradient flowing back to update the parameters, known as saturating gradient problem. Therefore, for deep neural networks, a single kink function (such as ReLU) is preferred.

Good to activate or deactivate parts of the network because of the smooth (differentiable) transition.



Deep Learning - Vito Walter Anelli

Figura 4.14: Sigmoid: output in $(0, 1)$, il che la rende ideale come gate (LSTM, GRU) o per output probabilistici di classificatori binari. Impilare molti layer sigmoid causa il vanishing gradient: il gradiente della sigmoid è prossimo a zero per $|x| \gg 0$, bloccando la retropropagazione. Per reti profonde si preferisce ReLU.

Connessione con Softmax. Come mostrato nelle slide (Sez. 4.9), la Sigmoid è un caso speciale della Softmax a due classi: posto $x_2 = 0$,

$$z_1 = \frac{e^{x_1}}{e^{x_1} + e^{x_2}} = \frac{e^{x_1}}{e^{x_1} + 1} = \frac{1}{1 + e^{-x_1}} = \sigma(x_1). \quad (4.14)$$

4.6.2 Tanh - nn.Tanh()

$$\text{Tanh}(x) = \tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}. \quad (4.15)$$



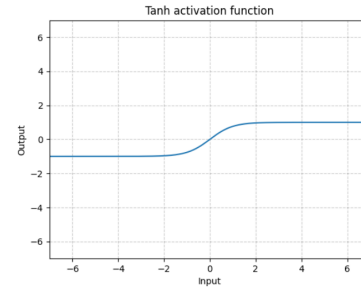
Tanh - nn.Tanh()

Tanh is basically identical to Sigmoid except it is centred, ranging from -1 to 1.

The output of the function will have roughly zero mean.

Therefore, the model will converge faster. Note that convergence is usually faster if the average of each input variable is close to zero. One example is Batch Normalization.

$$\text{Tanh}(x) = \tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$



Deep Learning - Vito Walter Anelli

Figura 4.15: Tanh: versione centrata della Sigmoid con output in $(-1, 1)$. L'output ha media approssimativamente zero, accelerando la convergenza rispetto a Sigmoid (nota fondamentale: la convergenza è più rapida quando la media degli input è vicina a zero, come avviene nel Batch Normalization). Soffre comunque del vanishing gradient.

Relazione con Sigmoid. $\tanh(x) = 2\sigma(2x) - 1$: Tanh è semplicemente una Sigmoid riscalata e traslata. Preferire Tanh a Sigmoid nei layer nascosti perché l'output centrato evita bias sistematici nel gradiente (Bishop and Bishop, 2024).

4.6.3 Hardtanh – nn.Hardtanh()

$$\text{Hardtanh}(x) = \begin{cases} 1, & \text{se } x > 1 \\ -1, & \text{se } x < -1 \\ x, & \text{altrimenti} \end{cases} \quad (4.16)$$



Politecnico di Bari

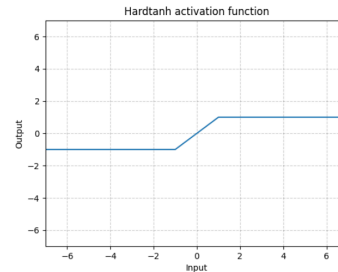


Hardtanh - nn.Hardtanh()

The range of the linear region $[-1, 1]$ can be adjusted using `min_val` and `max_val`.

It works surprisingly well especially when weights are kept within the small value range.

$$\text{Hardtanh}(x) = \begin{cases} 1, & \text{if } x > 1 \\ -1, & \text{if } x < -1 \\ x, & \text{otherwise} \end{cases}$$



Deep Learning - Vito Walter Anelli

Figura 4.16: Hardtanh: approssimazione lineare a tratti di Tanh. I valori di saturazione $[-1, 1]$ sono regolabili tramite `min_val` e `max_val`. Funziona sorprendentemente bene quando i pesi rimangono in un range di valori piccoli, ed è computazionalmente più leggera di Tanh.

4.6.4 Softsign – nn.Softsign()

$$\text{SoftSign}(x) = \frac{x}{1 + |x|}. \quad (4.17)$$

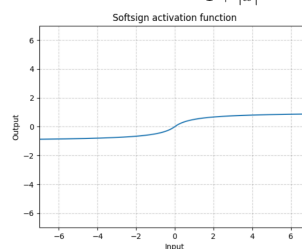


Politecnico di Bari



Softsign - nn.Softsign()

$$\text{SoftSign}(x) = \frac{x}{1 + |x|}$$



Deep Learning - Vito Walter Anelli

Figura 4.17: Softsign: alternativa a Tanh con saturazione più lenta (decadimento polinomiale anziché esponenziale). Le code più pesanti implicano gradienti meno soggetti al vanishing problem rispetto a Tanh, ma l'output rimane in $(-1, 1)$.

4.7 Funzioni Shrink e Threshold

4.7.1 Threshold – `nn.Threshold()`

$$\text{threshold}(x) = \begin{cases} x, & \text{se } x > \text{threshold} \\ v, & \text{altrimenti} \end{cases} \quad (4.18)$$




Threshold - `nn.Threshold()`

It is rarely used because we cannot propagate the gradient back.

$$\text{threshold}(x) = \begin{cases} x, & \text{if } x > \text{threshold} \\ v, & \text{otherwise} \end{cases}$$

And it is also the reason preventing people from using back-propagation in 60s and 70s when they were using binary neurons.

Deep Learning – Vito Walter Anelli

Figura 4.18: Threshold (neurone binario): il gradiente è zero nella regione subthreshold, rendendo impossibile la retropropagazione per quei neuroni. È la ragione storica per cui la backpropagation non venne adottata negli anni '60–'70, quando si usavano neuroni binari. Attualmente non è utilizzata in reti moderne.

4.7.2 Tanhshrink – `nn.Tanhshrink()`

$$\text{Tanhshrink}(x) = x - \tanh(x). \quad (4.19)$$

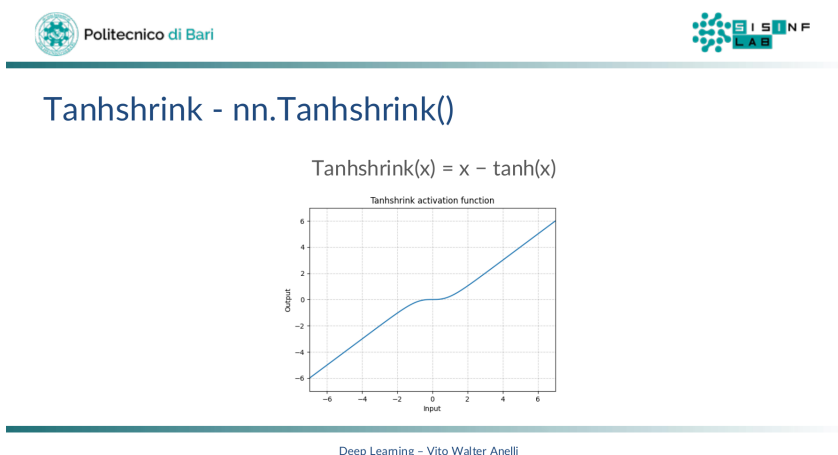


Figura 4.19: Tanhshrink: sottrae $\tanh(x)$ dall'input, producendo una funzione quasi lineare (con una lieve curvatura centrale). Usata in contesti di sparse coding per promuovere rappresentazioni sparse.

4.7.3 Softshrink – `nn.Softshrink()`

$$\text{Softshrinkage}(x) = \begin{cases} x - \lambda, & \text{se } x > \lambda \\ x + \lambda, & \text{se } x < -\lambda \\ 0, & \text{altrimenti} \end{cases} \quad (4.20)$$



Politecnico di Bari

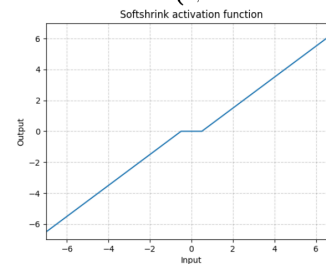


Softshrink - `nn.Softshrink()`

This basically **shrinks the variable by a constant towards 0**, and forces to 0 if the variable is close to 0.

You can think of it as a step of gradient for the L_1 criteria. It is also one of the step of the Iterative Shrinkage-Thresholding Algorithm (ISTA). But it is not commonly used in standard neural network as activations.

$$\text{Softshrinkage}(x) = \begin{cases} x - \lambda, & \text{if } x > \lambda \\ x + \lambda, & \text{if } x < -\lambda \\ 0, & \text{otherwise} \end{cases}$$



Deep Learning – Vito Walter Anelli

Figura 4.20: Softshrink: riduce il valore assoluto di x di λ verso zero, forzando a zero gli input nell'intervallo $[-\lambda, \lambda]$. Corrisponde a un passo del gradiente per la norma L_1 ed è uno step dell'algoritmo ISTA (*Iterative Shrinkage-Thresholding*). Non è comunemente usata come attivazione nelle reti standard.

4.7.4 Hardshrink – `nn.Hardshrink()`

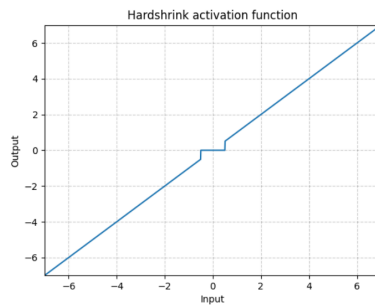
$$\text{Hardshrink}(x) = \begin{cases} x, & \text{se } x > \lambda \\ x, & \text{se } x < -\lambda \\ 0, & \text{altrimenti} \end{cases} \quad (4.21)$$



Politecnico di Bari



Hardshrink - nn.Hardshrink()



$$\text{Hardshrink}(x) = \begin{cases} x, & \text{if } x > \lambda \\ x, & \text{if } x < -\lambda \\ 0, & \text{otherwise} \end{cases}$$

Avoid using it with backprop because it is not continuous

Deep Learning - Vito Walter Anelli

Figura 4.21: Hardshrink: variante discontinua di Softshrink. La discontinuità in $x = \pm\lambda$ rende il gradiente non definito in quei punti, per cui è **sconsigliato l'uso con backpropagation**. Utilizzata in alcuni algoritmi di sparse coding fuori dal contesto delle reti neurali gradient-based.

4.8 LogSigmoid – nn.LogSigmoid()

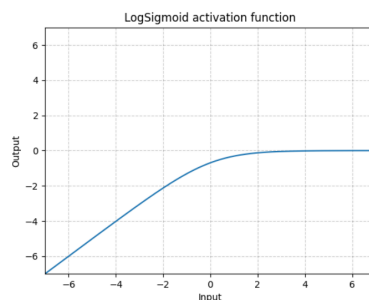
$$\text{LogSigmoid}(x) = \log\left(\frac{1}{1 + \exp(-x)}\right). \quad (4.22)$$



Politecnico di Bari



LogSigmoid - nn.LogSigmoid()



$$\text{LogSigmoid}(x) = \log\left(\frac{1}{1 + \exp(-x)}\right)$$

Designed for numerical instabilities (functions with exponentials lead to instabilities)

Deep Learning - Vito Walter Anelli

Figura 4.22: LogSigmoid: logaritmo della Sigmoid. Progettata per gestire le instabilità numeriche che emergono quando funzioni con esponenziali vengono composte (overflow/underflow). Usata prevalentemente nelle funzioni di costo, non come attivazione interna.

4.9 Funzioni Vettoriali: Softmin, Softmax, LogSoftmax

Le funzioni di questa sezione agiscono su **vettori interi**, trasformando un vettore di scores in una distribuzione di probabilità. Non sono usate come attivazioni interne, ma tipicamente nell'ultimo layer per compiti di classificazione multiclasse.

4.9.1 Softmin – `nn.Softmin()`

$$\text{Softmin}(x_i) = \frac{\exp(-x_i)}{\sum_j \exp(-x_j)}. \quad (4.23)$$

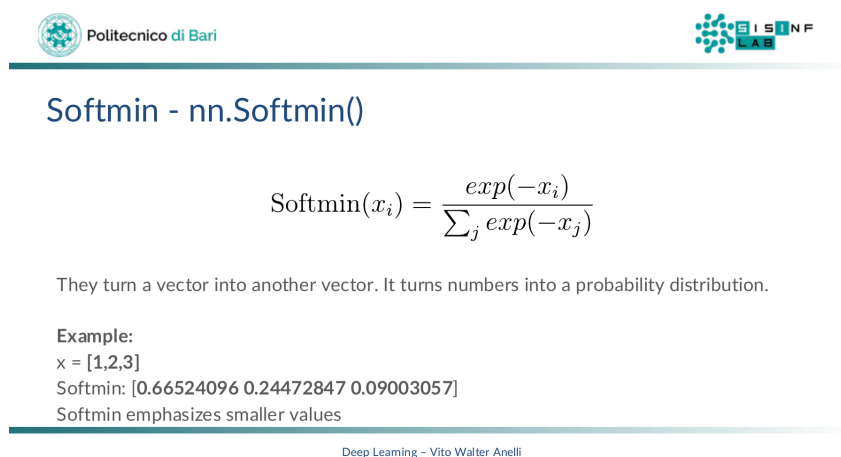


Figura 4.23: Softmin: enfatizza i valori più *piccoli* del vettore input. Per $x = [1, 2, 3]$ produce $[0,665, 0,245, 0,090]$: il valore più piccolo (1) riceve la probabilità maggiore. Raro in pratica; utile in contesti dove si vuole “selezionare” il minimo in modo differenziabile.

4.9.2 Soft(arg)max – `nn.Softmax()`

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}. \quad (4.24)$$



Soft(arg)max - nn.Softmax()

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

Example:

$x = [1, 2, 3]$

Softmax: [0.09003057 0.24472847 0.66524096]

Softmax tends to amplify the differences between the values

Deep Learning - Vito Walter Anelli

Figura 4.24: Softmax: trasforma uno score vettoriale in una distribuzione di probabilità sommande a 1. Enfatisza il valore più grande: per $x = [1, 2, 3]$ produce [0,090, 0,245, 0,665]. Amplifica le differenze tra i valori (*winner-takes-all*).

Softmax come generalizzazione di Sigmoid. La Sigmoid è un caso speciale della Softmax per due classi. Fissando $x_2 = 0$:

$$z_1 = \frac{e^{x_1}}{e^{x_1} + e^{x_2}} \xrightarrow{x_2=0} \frac{e^{x_1}}{e^{x_1} + 1} = \frac{1}{1 + e^{-x_1}} = \sigma(x_1). \quad (4.25)$$



What is a Softmax?

$$z_i = \frac{\exp(x_i)}{\sum_j \exp(x_j)} \quad z_1 = \frac{e^{x_1}}{e^{x_1} + e^{x_2}}$$

$$x_2 = 0 \quad z_1 = \frac{e^{x_1}}{e^{x_1} + 1} \quad z_1 = \frac{1}{1 + e^{-x_1}}$$

What is now?

Deep Learning - Vito Walter Anelli

Figura 4.25: Derivazione della Sigmoid come caso speciale della Softmax a 2 classi. Questa relazione spiega perché la regressione logistica binaria (che usa Sigmoid) è un caso speciale della Softmax multiclasse.

4.9.3 LogSoft(arg)max - nn.LogSoftmax()

$$\text{LogSoftmax}(x_i) = \log \left(\frac{\exp(x_i)}{\sum_j \exp(x_j)} \right). \quad (4.26)$$



LogSoft(arg)max - nn.LogSoftmax()

$$\text{LogSoftmax}(x_i) = \log \left(\frac{\exp(x_i)}{\sum_j \exp(x_j)} \right)$$

It is mostly used in the loss function but not common for activations. (to avoid numerical instabilities and saturation of ones and zeros) [it is used as a component in other loss functions]

Deep Learning - Vito Walter Anelli

Figura 4.26: LogSoftmax: usato principalmente nelle funzioni di costo (es. in combinazione con `nn.NLLLoss`) per evitare instabilità numeriche (overflow/underflow per esponenziali grandi) e la saturazione di valori in 0 e 1. Non è comune come attivazione interna.

4.10 Softmax e Cross-Entropy Loss

Nell'ultimo layer di un classificatore multiclasse, la Softmax viene accoppiata alla **Cross-Entropy Loss**:

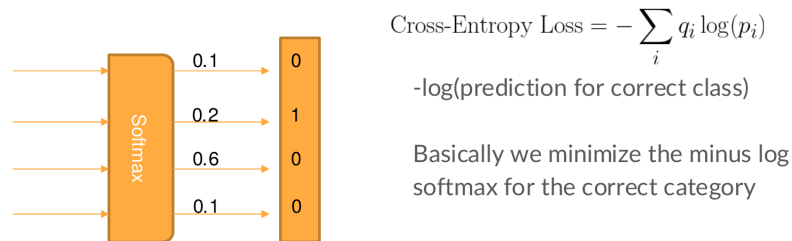
$$\mathcal{L}_{\text{CE}} = - \sum_i q_i \log(p_i), \quad (4.27)$$

dove $q_i \in \{0, 1\}$ è la label one-hot e $p_i = \text{Softmax}(x_i)$. Per un vettore one-hot, la somma collassa a un solo termine:

$$\mathcal{L}_{\text{CE}} = - \log(\text{Softmax}(x_{\text{corr}})), \quad (4.28)$$

ossia si *massimizza la probabilità assegnata alla classe corretta* (o equivalentemente si minimizza il suo logaritmo negativo).

What to use with Softmax?



Deep Learning - Vito Walter Anelli

Figura 4.27: Softmax + Cross-Entropy Loss: il layer Softmax produce una distribuzione $[0.1, 0.2, 0.6, 0.1]$ mentre il target one-hot seleziona la classe 2 ($[0, 1, 0, 0]$). La loss è $-\log(0.2)$. In pratica si usa `nn.CrossEntropyLoss` in PyTorch che fonde `LogSoftmax` e `NLLLoss` per stabilità numerica.

In PyTorch è raccomandato usare `nn.CrossEntropyLoss` anziché `nn.Softmax` + `nn.NLLLoss` separatamente, perché la formulazione unificata implementa il log-sum-exp trick che evita overflow numerici (Bishop and Bishop, 2024).

4.11 Guida Pratica alla Scelta

La scelta della funzione di attivazione dipende dal task, dalla profondità della rete e dai requisiti computazionali. Le linee guida di Bishop and Bishop (2024) e del corso NYU-DL possono essere sintetizzate nella Tabella 4.1.

Tabella 4.1: Guida pratica alla scelta della funzione di attivazione.

Funzione	Output range	Gradiente	Uso tipico
ReLU	$[0, +\infty)$	Non saturante (pos.)	Default hidden layer
Leaky ReLU	$(-\infty, +\infty)$	Non saturante	ReLU con dead neurons
PReLU	$(-\infty, +\infty)$	Apprendibile	Quando LReLU è insufficiente
ELU	$(-\alpha, +\infty)$	Non saturante	Hidden, mean ≈ 0
SELU	$(\approx -1.76, +\infty)$	Self-norm.	Reti senza BatchNorm
GELU	$\approx (-0.17, +\infty)$	Non mon.	Transformer (BERT, GPT)
Sigmoid	$(0, 1)$	Saturante	Output prob. binaria, gate
Tanh	$(-1, 1)$	Saturante	RNN, output centrato
Softmax	$(0, 1)^C$, sum = 1	Vettoriale	Output multiclasse

Raccomandazioni operative.

1. **Punto di partenza:** usare **ReLU** per i layer nascosti di MLP e CNN; è semplice, efficiente e funziona bene nella maggior parte dei casi.

2. **Dead neurons:** se si osservano molti neuroni inattivi (output sempre zero), passare a **Leaky ReLU** o **ELU**.
3. **Reti senza BatchNorm:** considerare **SELU**, che fornisce self-normalization automatica.
4. **Transformer:** preferire **GELU**, che è la scelta standard in BERT, GPT-2/3 e modelli derivati.
5. **Output layer:** usare **Sigmoid** per classificazione binaria, **Softmax** per multi-classe, nessuna attivazione (o ReLU/Softplus) per regressione con output positivo.
6. **Loss function:** abbinare sempre `nn.CrossEntropyLoss` alla Softmax (fuse internamente con `LogSoftmax`) per stabilità numerica.

4.12 Riepilogo

Problema	Causa	Soluzione
Vanishing gradient	Saturazione Sigmoid/Tanh	ReLU, ELU, GELU
Dead neurons	$\text{ReLU} = 0$ per input neg.	LReLU, PReLU, ELU
Output non centrato	$\text{ReLU} \geq 0$ sempre	ELU, SELU, BatchNorm
Instabilità num.	Exp. overflow in Softmax	LogSoftmax, CrossEntropyLoss
Parametri extra	PReLU ha a apprendibile	Beneficio: adattività
Non monotonicità	GELU ha un “dip”	Proprietà regolatrice

Le funzioni di attivazione rappresentano uno degli assi portanti dell'ingegneria delle reti neurali: la scelta sbagliata può precludere l'apprendimento nei layer profondi (vanishing gradient), mentre la scelta giusta velocizza la convergenza e migliora la generalizzazione. Il consenso attuale nella letteratura è di usare ReLU o le sue varianti non saturanti come default, riservando Sigmoid e Tanh ai casi in cui la struttura del problema lo impone (output probabilistici, meccanismi di gating, reti ricorrenti).

Capitolo 5

Loss Functions

La **funzione di perdita** (loss function, o cost function) misura la discrepanza tra la previsione della rete e il target desiderato. La scelta della loss è tanto importante quanto quella dell'architettura: determina *cosa* la rete impara ad ottimizzare, e influenza il comportamento del gradiente durante il training.

In PyTorch tutte le loss sono implementate nel modulo `torch.nn` e seguono una convenzione comune: accettano un input `x` (la predizione) e un target `y`, e supportano l'argomento `reduction` (`'none'`, `'mean'`, `'sum'`) per controllare come le loss elementari vengono aggregate sul batch (Bishop and Bishop, 2024).

5.1 Loss per Regressione

5.1.1 MSE Loss – `nn.MSELoss()`

La **Mean Squared Error** (errore quadratico medio, norma L2 al quadrato) è la loss di regressione più comune:

$$l(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = (x_n - y_n)^2, \quad (5.1)$$

dove N è la dimensione del batch. Con `reduction='none'` la loss viene restituita come vettore; altrimenti:

$$l(x, y) = \begin{cases} \text{mean}(L), & \text{se } \text{reduction} = \text{'mean'} \\ \text{sum}(L), & \text{se } \text{reduction} = \text{'sum'} \end{cases} \quad (5.2)$$

La somma opera su tutti gli elementi e, con `'mean'`, divide per n . Per evitare la divisione si usa `reduction='sum'`.

Pro e contro. MSE è differenziabile ovunque e penalizza fortemente gli errori grandi (il quadrato amplifica gli outlier). In task di regressione con output continui è la scelta predefinita, ma soffre di elevata sensibilità al rumore.

5.1.2 L1 Loss – `nn.L1Loss()`

La **Mean Absolute Error** (MAE) misura l'errore assoluto medio:

$$l(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = |x_n - y_n|. \quad (5.3)$$

Supporta le stesse opzioni di riduzione di `nn.MSELoss()`.

Vantaggio rispetto a L2. L1 è più robusta agli outlier: gli errori non vengono elevati al quadrato, quindi i punti rumorosi influenzano meno la loss totale. Il valore y che minimizza la distanza L1 è la **mediana** (anziché la media per L2), il che produce previsioni meno sfocate in task come la super-risoluzione d'immagine.

L1 ha gradienti costanti: anche quando la loss si avvicina a zero, il gradiente non si annulla, producendo immagini predette più nitide.

Limite. L1 **non è differenziabile** in $x = 0$ (dove la funzione valore assoluto ha un “angolo”). Questo richiede attenzione nella gestione del gradiente (cfr. `nn.Softshrink`). Per questo motivo è stata introdotta la `SmoothL1Loss`.

5.1.3 Smooth L1 Loss (Huber Loss) – `nn.SmoothL1Loss()`

Combina L2 per errori piccoli e L1 per errori grandi, eliminando il problema della non-differenziabilità in zero:

$$\text{loss}(x, y) = \frac{1}{n} \sum_i z_i, \quad z_i = \begin{cases} 0.5 (x_i - y_i)^2, & \text{se } |x_i - y_i| < 1 \\ |x_i - y_i| - 0.5, & \text{altrimenti} \end{cases} \quad (5.4)$$

Nota anche come **Huber Loss** o **Elastic Network** quando usata come funzione obiettivo. È stata resa popolare da Ross Girshick in Fast R-CNN.

Pro e contro. Meno sensibile agli outlier rispetto a MSE e smooth nel punto zero. Usata frequentemente in **computer vision** (object detection) per proteggere contro outlier nelle coordinate dei bounding box. Il limite è la presenza del fattore di scala 0.5 fisso nella regione quadratica.

5.1.4 L1 vs. L2 in Computer Vision

In task di predizione d'immagine dove esistono molte possibili output y :

- **MSE (L2):** la previsione converge alla *media* di tutte le possibili y . In visione artificiale questo produce immagini **sfocate** (blur), perché la media di immagini nitide con strutture leggermente diverse è un'immagine priva di dettagli.
- **L1:** la previsione converge alla *mediana*, che non produce blur ma è difficile da definire in spazi multidimensionali.

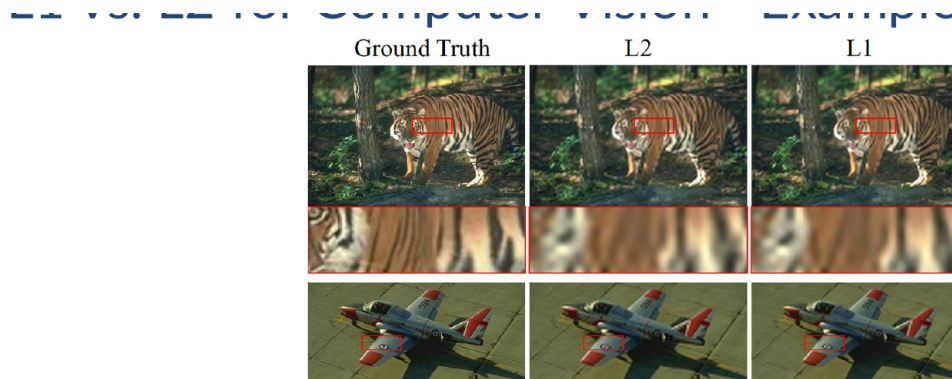


Figura 5.1: Confronto visivo tra Ground Truth, predizione con L2 (sfocata) e predizione con L1 (più nitida) su due esempi: una tigre e un aereo. Il dettaglio ingrandito mostra come L2 produca un'area blurred nel crop della tigre, mentre L1 preserva meglio le strutture. [Fonte: towardsdatascience.com/perceptual-losses-for-image-restoration]

5.2 Loss per Classificazione

5.2.1 NLL Loss – `nn.NLLLoss()`

La **Negative Log Likelihood Loss** è usata per problemi di classificazione con C classi. Matematicamente, l'input dovrebbe essere una log-likelihood (come l'output di `nn.LogSoftmax`), ma PyTorch non lo impone.

Loss non ridotta:

$$l(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = -w_{y_n} x_{n, y_n}, \quad w_c = \text{weight}[c] \cdot \mathbf{1}\{c \neq \text{ignore_index}\}. \quad (5.5)$$

Con riduzione:

$$l(x, y) = \begin{cases} \sum_{n=1}^N \frac{1}{\sum_{n=1}^N w_{y_n}} l_n, & \text{se reduction = 'mean'} \\ \sum_{n=1}^N l_n, & \text{se reduction = 'sum'} \end{cases} \quad (5.6)$$

L'argomento opzionale **weight** accetta un tensore 1D che assegna un peso a ciascuna classe, utile per gestire dataset sbilanciati.

5.2.2 Gestione dei Dataset Sbilanciati

Quando la frequenza delle classi differisce notevolmente (es. influenza comune vs cancro al polmone), si possono adottare due strategie:

Strategia 1 – Pesi nella loss. Aumentare il peso `weight[c]` per le classi minoritarie in `NLLLoss` o `CrossEntropyLoss`.

Strategia 2 – Equalizzazione della frequenza (preferita). Invece di assegnare pesi, è meglio **equalizzare la frequenza nel training** per sfruttare meglio i gradienti stocastici:

1. Raccogliere i campioni di ogni classe in buffer separati.
2. Generare ogni minibatch prendendo lo stesso numero di campioni da ciascun buffer.
3. Quando il buffer della classe minoritaria si esaurisce, ricominciare dall'inizio (buffer circolare) finché tutti i campioni della classe maggioritaria sono stati usati.

Attenzione: non ridurre artificialmente la classe maggioritaria scartando campioni — non sprecare dati. Se si equalizza la frequenza, il modello non conosce più la frequenza reale delle classi. Per correggere questo, si eseguono pochi epoch finali con la distribuzione reale delle classi, affinché il sistema adatti i bias dell'output layer.

5.2.3 Cross-Entropy Loss – `nn.CrossEntropyLoss()`

Combina `nn.LogSoftmax` e `nn.NLLLoss` in un'unica classe, massimizzando lo score della classe corretta:

$$\text{loss}(x, c) = -\log\left(\frac{\exp(x[c])}{\sum_j \exp(x[j])}\right) = -x[c] + \log\left(\sum_j \exp(x[j])\right). \quad (5.7)$$

Con peso per classe $w[c]$:

$$\text{loss}(x, c) = w[c] \left(-x[c] + \log\left(\sum_j \exp(x[j])\right) \right). \quad (5.8)$$

Perché unire `LogSoftmax` e `NLLLoss`? Quando il valore dopo Softmax è vicino a 0 o 1, il suo logaritmo tende a $-\infty$ e la derivata del log vicino a 0 tende a $+\infty$, causando instabilità numerica durante la retropropagazione. Unendo le due operazioni, i gradienti vengono saturati a valori ragionevoli tramite il **log-sum-exp trick**.

Interpretazione probabilistica: KL Divergence. La Cross-Entropy ha una lettura information-theoretic: misura la **divergenza di Kullback-Leibler** tra la distribuzione predetta q e quella target p :

$$H(p, q) = H(p) + \mathcal{D}_{KL}(p \parallel q) \quad (5.9)$$

$$H(p, q) = -\sum_i p(x_i) \log q(x_i) \quad (\text{cross-entropy}) \quad (5.10)$$

$$H(p) = -\sum_i p(x_i) \log p(x_i) \quad (\text{entropia}) \quad (5.11)$$

$$\mathcal{D}_{KL}(p \parallel q) = \sum_i p(x_i) \log \frac{p(x_i)}{q(x_i)} \quad (\text{KL divergence}) \quad (5.12)$$

Nel caso classificazione, p è il vettore one-hot (0 sulle classi errate, 1 sulla corretta) e q è il vettore Softmax delle predizioni. Minimizzare la Cross-Entropy equivale a minimizzare la KL Divergence tra predizioni e target.

5.2.4 Binary Cross Entropy – `nn.BCELoss()`

Caso speciale della Cross-Entropy per **due classi**:

$$l(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = -w_n [y_n \log x_n + (1 - y_n) \log(1 - x_n)]. \quad (5.13)$$

Usata tipicamente per misurare l'errore di ricostruzione in un auto-encoder. **Attenzione:** la formula assume che x e y siano probabilità, quindi strettamente in $(0, 1)$.

5.2.5 BCE with Logits – `nn.BCEWithLogitsLoss()`

Versione numericamente più stabile di `BCELoss`: accetta score grezzi (logit) non normalizzati e applica internamente la Sigmoid prima del log:

$$l_n = -w_n [y_n \log \sigma(x_n) + (1 - y_n) \log(1 - \sigma(x_n))]. \quad (5.14)$$

Non assume che $x \in (0, 1)$ e combina Sigmoid + BCE in un'unica operazione numericamente stabile. **È preferita a `nn.BCELoss`** nelle reti con output layer lineare.

5.2.6 NLL Loss Adattiva – `nn.AdaptiveLogSoftmaxWithLoss()`

Approssimazione efficiente di Softmax per un numero molto elevato di classi (nell'ordine dei milioni). Implementa il metodo descritto in *Efficient softmax approximation for GPUs* (Grave et al., 2017), che raggruppa le classi per frequenza e usa head di dimensioni diverse per classi rare vs frequenti.

5.2.7 KL Divergence Loss – `nn.KLDivLoss()`

$$l(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = y_n (\log y_n - \log x_n). \quad (5.15)$$

Utile quando il target è una distribuzione one-hot. Assume che x e y siano probabilità. **Svantaggio:** non è integrata con una Softmax o LogSoftmax, quindi può soffrire di instabilità numerica. Per questo motivo si preferisce `nn.CrossEntropyLoss` nella maggior parte dei casi.

5.3 Loss Basate su Margine

Le **margin losses** sono una categoria importante di loss functions: invece di misurare la distanza assoluta tra predizione e target, richiedono che la predizione corretta superi quella errata di almeno un certo margine.

5.3.1 Hinge Embedding Loss – `nn.HingeEmbeddingLoss()`

Usata in apprendimento semi-supervisionato per misurare la similarità/dissimilarità tra coppie di input:

$$l_n = \begin{cases} x_n, & y_n = 1 \\ \max\{0, \delta - x_n\}, & y_n = -1 \end{cases} \quad (5.16)$$

La variabile y indica se la coppia (x) è simile (+1) o dissimile (-1). Con la hinge loss, lo score è positivo se $y = 1$ e viene penalizzato se $y = -1$ ma non supera il margine δ . Di solito si usa con la distanza L1 a coppie come x .

5.3.2 Margin Ranking Loss – `nn.MarginRankingLoss()`

$$L(x, y) = \max(-y \cdot (x_1 - x_2) + \text{margin}, 0). \quad (5.17)$$

Richiede che un input sia maggiore dell'altro di almeno `margin`:

- x_1, x_2 : score assegnati dalla rete a due item (positivo e negativo).
- $y = +1$: si vuole $x_1 > x_2$ (item positivo deve avere score maggiore).
- $y = -1$: si vuole $x_2 > x_1$ (flip del confronto).
- `margin`: soglia minima della differenza desiderata tra gli score.

Se $y \cdot (x_1 - x_2) > \text{margin}$, il costo è 0; altrimenti cresce linearmente. In classificazione, x_1 è lo score della risposta corretta e x_2 è lo score più alto tra le risposte errate nel minibatch. Nei modelli energy-based (EBM), questa loss abbassa l'energia della risposta corretta x_1 e alza quella della risposta errata x_2 .

5.3.3 Triplet Margin Loss – `nn.TripletMarginLoss()`

$$L(a, p, n) = \max\{d(a_i, p_i) - d(a_i, n_i) + \text{margin}, 0\}, \quad (5.18)$$

dove a è l'anchor, p è il campione positivo (stessa classe), n è quello negativo (classe diversa) e $d(\cdot, \cdot)$ è una metrica di distanza.

Usata per misurare similarità relativa tra campioni:

- Due immagini della stessa categoria passate a una CNN devono produrre vettori **vicini**.
- Due immagini di categorie diverse devono produrre vettori **lontani**.

La loss spinge la distanza della coppia positiva verso 0 e quella della coppia negativa oltre il margine. Ciò che conta è che la distanza della coppia buona sia *minore* di quella della coppia cattiva, non i valori assoluti.

5.3.4 Soft Margin Loss – `nn.SoftMarginLoss()`

$$L(x, y) = \sum_i \frac{\log(1 + \exp(-y[i] \cdot x[i]))}{n.\text{nelement}()}. \quad (5.19)$$

Versione **soft** (continua) della margin loss: ottimizza una perdita logistica per classificazione a due classi con target $y \in \{-1, +1\}$. Vuole che $\exp(-y[i] \cdot x[i])$ per la risposta corretta sia più piccolo che per le altre: avvicina i valori positivi di $y[i] \cdot x[i]$ e allontana quelli negativi, ma con un effetto esponenzialmente decrescente (a differenza di un hard margin).

5.3.5 Multi-Class Hinge Loss – `nn.MultiLabelMarginLoss()`

$$L(x, y) = \sum_{ij} \frac{\max(0, 1 - (x[y[j]] - x[i]))}{x.size(0)}. \quad (5.20)$$

Estende la hinge loss al caso multiclasse, permettendo a diversi input di avere quantità variabili di target. Somma la hinge loss su tutte le categorie target. Nei modelli EBM, abbassa l'energia delle categorie desiderate e alza quella delle non-desiderate.

5.4 Loss per Embedding: Cosine Embedding Loss

5.4.1 Formulazione

`nn.CosineEmbeddingLoss()` misura se due input sono simili o dissimili usando la **distanza coseno**:

$$l_n = \begin{cases} 1 - \cos(x_1, x_2), & y = 1 \\ \max\{0, \cos(x_1, x_2) - \text{margin}\}, & y = -1 \end{cases} \quad (5.21)$$

Tipicamente usata per apprendere embedding non lineari o in apprendimento semi-supervisionato.

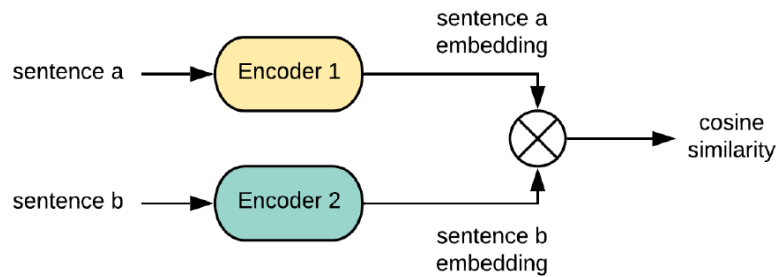


Figura 5.2: Schema della CosineEmbeddingLoss: due input (*sentence a* e *sentence b*) vengono proiettati attraverso due encoder e i rispettivi embedding vengono confrontati tramite cosine similarity. Il simbolo $\otimes$ indica il prodotto scalare normalizzato.

5.4.2 Interpretazione Geometrica

$1 - \cos(\theta)$ tra due vettori è equivalente alla **distanza euclidea normalizzata**: dopo aver portato i vettori sulla sfera unitaria, la loro distanza dipende solo dall'angolo θ tra loro, non dalla loro norma.

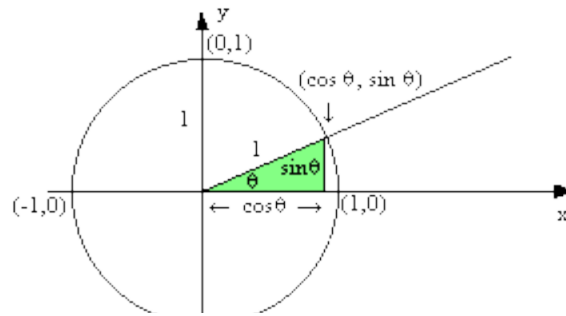


Figura 5.3: Cerchio unitario: il punto $(\cos \theta, \sin \theta)$ giace sulla sfera unitaria. La distanza $1 - \cos \theta$ misura quanto due vettori normalizzati differiscono in direzione, indipendentemente dalla loro magnitudine.

Perché normalizzare? Se si vuole massimizzare la distanza tra vettori dissimili, la rete potrebbe semplicemente allungare i vettori anziché ruotarli nella direzione corretta. La normalizzazione forza la rete a **ruotare** i vettori, non a scalarne la norma. In uno spazio ad alta dimensione, la maggior parte dell’“area” della sfera si trova vicino all’equatore: dopo la normalizzazione, campioni semanticamente simili devono stare vicini sulla sfera, quelli dissimili devono essere **ortogonali**.

Comportamento per le due casistiche:

- **Coppie positive** ($y = 1$): la loss cerca di allineare i vettori il più possibile ($\cos \theta \rightarrow 1$).
- **Coppie negative** ($y = -1$): la loss spinge il coseno sotto un margine positivo piccolo, rendendo i vettori ortogonali o opposti.

5.5 Guida Pratica alla Scelta della Loss

Tabella 5.1: Guida alla scelta della funzione di perdita.

Task	Loss consigliata	Note
Regressione standard	MSELoss	Sensibile agli outlier
Regressione robusta	SmoothL1Loss	Huber, usata in CV
Regressione con immagini	L1Loss	Predizioni più nitide
Classificazione multiclasse	CrossEntropyLoss	Input = logit grezzi
Classificazione binaria	BCEWithLogitsLoss	Numericamente stabile
Classi sbilanciate	NLLLoss + weight	O equalizzare frequenza
Classi $\gg 10^4$	AdaptiveLogSoftmax	Approssimazione efficiente
Ranking / retrieval	MarginRankingLoss	Coppia positivo/negativo
Metric learning	TripletMarginLoss	Tripla anchor/pos/neg
Embedding similarity	CosineEmbeddingLoss	Direzione vettoriale

Regole operative.

1. Usare sempre `nn.CrossEntropyLoss` (non `Softmax` + `NLLLoss` separatamente) per stabilità numerica del gradiente.

2. Per classificazione binaria preferire `BCEWithLogitsLoss` a `BCELoss`: non richiedere che l'output sia già in $(0, 1)$.
3. In computer vision generativa (super-risoluzione, inpainting) preferire L1 a L2 per evitare immagini sfocate.
4. Per dataset sbilanciati, privilegiare l'equalizzazione della frequenza sui buffer rispetto ai pesi nella loss, e fare fine-tuning finale con la distribuzione reale.
5. Nelle loss basate su margine, scegliere il valore del margine tramite validation set: un margine troppo piccolo produce scarsa separazione, uno troppo grande può impedire la convergenza.

5.6 Riepilogo

Categoria	Loss	Proprietà chiave
Regressione	MSE	Media, sensibile outlier
	L1	Mediana, robusta, non diff. in 0
	SmoothL1 (Huber)	L2 vicino a 0, L1 lontano
Classificazione	CrossEntropyLoss	LogSoftmax + NLL fusi
	BCEWithLogitsLoss	Sigmoid + BCE fusi
	NLLLoss	Con pesi per classi sbilanciate
	KLDivLoss	Distanza tra distribuzioni
Margine	HingeEmbedding	Similarità/dissimilarità
	MarginRanking	Ordinamento relativo
	TripletMargin	Anchor/Positivo/Negativo
	CosineEmbedding	Similarità angolare normalizzata

La scelta della loss non è separabile dall'architettura: `CrossEntropyLoss` richiede logit grezzi in input (non una Softmax esplicita), `BCEWithLogitsLoss` richiede logit (non probabilità), e le loss basate su margine operano su coppie o triple che devono essere costruite correttamente nel DataLoader. Come sottolinea [Bishop and Bishop \(2024\)](#), la loss function codifica l'assunzione probabilistica sul rumore del dato: MSE corrisponde a un modello gaussiano, L1 a un modello laplaciano, e la Cross-Entropy al massimo della verosimiglianza per distribuzioni categoriali.

Capitolo 6

Ottimizzazione

L'ottimizzazione è il processo che permette alla rete neurale di apprendere, aggiornando iterativamente i parametri $\mathbf{w}$ in modo da minimizzare la funzione di perdita $\mathcal{L}$. A differenza dell'ottimizzazione classica, in Deep Learning la funzione obiettivo è *non convessa*, ad alta dimensionalità e valutata su dati rumorosi campionati in mini-batch. La comprensione della geometria della superficie di errore e dei metodi per percorrerla efficientemente è fondamentale tanto quanto la scelta dell'architettura (Bishop and Bishop, 2024).

6.1 La Superficie di Errore

6.1.1 Neuroni Lineari e Ciotola Quadratica

La **superficie di errore** vive in uno spazio con un asse orizzontale per ogni peso e un asse verticale per il valore della loss. Per un neurone lineare con errore quadratico medio la superficie è una **ciotola quadratica** (quadratic bowl):

- Le sezioni verticali sono **parabole**.
- Le sezioni orizzontali sono **ellissi** (o cerchi se il problema è ben condizionato).

Per reti multi-layer non lineari la superficie è molto più complessa, ma *localmente* una ciotola quadratica rimane un'ottima approssimazione.

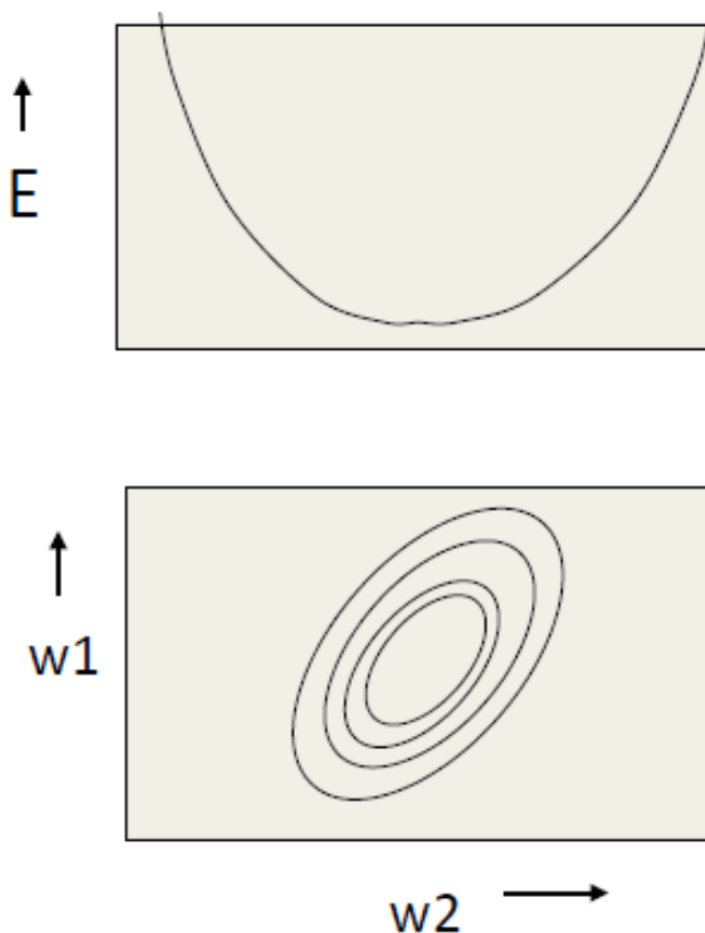


Figura 6.1: Sinistra: sezione verticale della superficie di errore per un neurone lineare (parabola). Destra: sezione orizzontale (ellissi concentriche) nello spazio dei pesi w_1 – w_2 .

6.1.2 Il Problema della Direzione del Gradiente

Nella discesa del gradiente si scende lungo la direzione di massima pendenza, ma questo non coincide con la direzione verso il minimo salvo che le ellissi siano cerchi. Il problema nasce perché:

- Il gradiente è **grande** nella direzione in cui vogliamo percorrere una piccola distanza (curvatura elevata).
- Il gradiente è **piccolo** nella direzione in cui vogliamo percorrere una grande distanza (curvatura bassa).

Questa asimmetria rende la convergenza lenta e produce i caratteristici *zig-zag* osservabili nelle traiettorie di ottimizzazione su superfici ellittiche.

6.2 Algoritmi di Apprendimento: Batch, Mini-Batch, Online

6.2.1 Ridondanza nel Dataset

Quando il dataset è **altamente ridondante**, il gradiente calcolato sulla prima metà dei dati è quasi identico a quello calcolato sulla seconda metà. In questo caso conviene aggiornare i pesi più volte per epoch, anziché attendere il calcolo del gradiente completo.

Full Batch Il gradiente viene calcolato sull'intero dataset. Sono disponibili metodi più sofisticati (gradiente coniugato non lineare, L-BFGS) ma il costo per iterazione è elevato.

Online (SGD puro) Il peso viene aggiornato dopo ogni singolo esempio. Noisy ma veloce nelle prime fasi.

Mini-Batch Compromesso ottimale per reti grandi: si calcola il gradiente su un sottoinsieme B di esempi. Sfrutta la moltiplicazione matrice-matrice accelerata da GPU e bilancia rumore e costo computazionale.

Regola pratica. Per reti neurali grandi su dataset grandi e ridondanti il **mini-batch** è quasi sempre la scelta migliore (Bishop and Bishop, 2024). I mini-batch devono essere bilanciati tra le classi per evitare gradienti sistematicamente distorti.

6.3 Accorgimenti per il Mini-Batch Gradient Descent

6.3.1 Learning Rate e Quando Ridurlo

Abbassare il learning rate η durante l'addestramento riduce le fluttuazioni casuali della loss dovute ai diversi gradienti sui diversi mini-batch:

- Si ottiene un *guadagno rapido* nel breve periodo (la loss scende più liscia).
- Si rallenta però l'apprendimento nel lungo periodo.

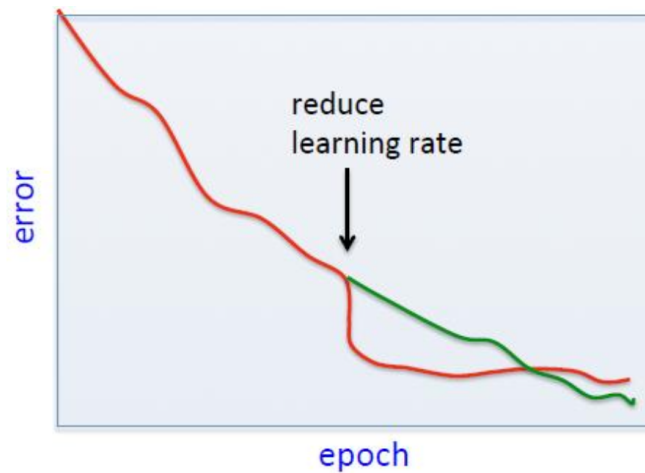


Figura 6.2: Effetto della riduzione del learning rate sulla curva di addestramento. La riduzione prematura (rosso) porta a convergenza lenta; quella ritardata (verde) sfrutta le oscillazioni iniziali per esplorare meglio lo spazio dei parametri.

Non ridurre il learning rate troppo presto!

6.3.2 Inizializzazione dei Pesi

Se due unità nascoste hanno esattamente lo stesso bias e gli stessi pesi in ingresso e in uscita, riceveranno sempre lo stesso gradiente e non potranno mai imparare feature diverse. Questo problema, detto *symmetry breaking*, si risolve inizializzando i pesi con valori casuali piccoli.

Fan-in e overshooting. Un'unità con fan-in elevato (molti ingressi) può subire un effetto di *overshoot*: piccole variazioni su molti pesi in ingresso si sommano, spostando eccessivamente l'uscita. La regola empirica è:

$$w \propto \frac{\mathcal{N}(0, 1)}{\sqrt{n_{\text{in}}}}, \quad (6.1)$$

dove n_{in} è il numero di connessioni in ingresso (fan-in).

Inizializzazione di Xavier / Glorot

? hanno mostrato che, per preservare la varianza dei segnali sia in forward che in backward pass, conviene tenere conto sia del fan-in che del fan-out. Per una distribuzione **uniforme** $\mathcal{U}(-a, a)$:

$$a = \text{gain} \times \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \quad (6.2)$$

mentre per una distribuzione **normale** $\mathcal{N}(0, \sigma^2)$:

$$\sigma = \text{gain} \times \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}. \quad (6.3)$$

Il parametro **gain** dipende dalla funzione di attivazione (es. 1 per la sigmoide, $\sqrt{5}$ per ReLU nella variante He). In PyTorch: `nn.init.xavier_uniform_` e `nn.init.xavier_normal_`.

6.3.3 Pre-elaborazione degli Input

Traslare e scalare gli input (*shift and scale*) in modo che abbiano media zero e varianza unitaria rende la superficie di errore più *circolare* (meno ellittica), migliorando la velocità di convergenza. Un approccio più avanzato è la **decorrelazione** tramite PCA:

1. Eliminare le componenti principali con autovalori più piccoli (riduzione della dimensionalità).
2. Dividere le componenti rimanenti per la radice quadrata dei rispettivi autovalori (*whitening*).

Per un neurone lineare, il whitening converte la superficie di errore da un'ellissi allineata agli assi a un cerchio, rendendo ottimale la discesa del gradiente puro.

6.4 Il Metodo del Momentum

6.4.1 Intuizione Fisica

L'idea del momentum si ispira alla fisica classica: immaginiamo una pallina che rotola sulla superficie di errore. La posizione della pallina nel piano orizzontale rappresenta il vettore dei pesi. Inizialmente la pallina segue il gradiente, ma non appena acquisisce velocità smette di seguire la direzione di massima pendenza: il *momentum* la trascina nella direzione in cui si stava già muovendo.



Figura 6.3: Confronto tra SGD senza momentum (sinistra) e con momentum (destra). Senza momentum la traiettoria oscilla fortemente in direzione trasversale; con momentum le oscillazioni vengono smorzate e la discesa verso il minimo è più diretta.

I benefici principali sono due:

- **Smorzamento delle oscillazioni** nelle direzioni ad alta curvatura, combinando gradienti con segno opposto.
- **Accelerazione** nelle direzioni a gradiente piccolo ma consistente.

6.4.2 Equazioni del Momentum (SGD con Momentum)

In SGD con momentum si mantengono due quantità: il vettore dei pesi $\mathbf{w}$ e il *termine di momentum* $\mathbf{p}$ (un running average dei gradienti). Le equazioni di aggiornamento sono:

$$\mathbf{p}_{k+1} = \beta \mathbf{p}_k + \eta \nabla \mathcal{L}(X, y, \mathbf{w}_k), \quad (6.4)$$

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \gamma \mathbf{p}_{k+1}, \quad (6.5)$$

dove $\beta \in (0, 1)$ è il **coefficiente di smorzamento** (dampening factor o momentum coefficient) ed η è il learning rate. Il parametro β deve essere strettamente maggiore di zero (altrimenti si ricade nel semplice gradient descent) e strettamente minore di uno (altrimenti le quantità esplodono).

Effetto di β . Valori piccoli di β permettono di cambiare direzione più rapidamente; valori grandi producono una traiettoria più inerziale, con curve più lente ma smorzamento maggiore delle oscillazioni.

6.4.3 Momentum di Nesterov (NAG)

Il metodo standard calcola il gradiente nella posizione corrente e poi compie un grande salto. ? ha proposto una variante ispirata al metodo di Nesterov per funzioni convesse:

1. Compie un grande salto nella direzione del gradiente *accumulato precedentemente*.
2. Misura il gradiente nella nuova posizione e applica una correzione.

Le equazioni diventano:

$$\hat{\mathbf{w}}_k = \mathbf{w}_k - \beta \mathbf{p}_k, \quad (6.6)$$

$$\mathbf{p}_{k+1} = \beta \mathbf{p}_k + \eta \nabla \mathcal{L}(X, y, \hat{\mathbf{w}}_k), \quad (6.7)$$

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \gamma \mathbf{p}_{k+1}. \quad (6.8)$$

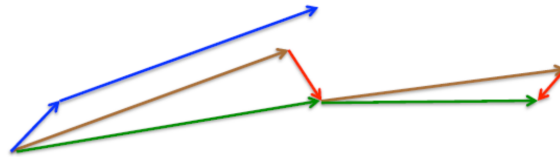


Figura 6.4: Intuizione del Nesterov Accelerated Gradient. Il vettore marrone è il salto nella direzione accumulata, il rosso è la correzione, il verde è il gradiente accumulato risultante. I vettori blu rappresentano il percorso del momentum standard.

Perché il momentum funziona? Esistono due spiegazioni complementari:

Accelerazione Con Nesterov è dimostrabile una convergenza accelerata su problemi *convessi*. Per reti neurali la prova non si applica direttamente, e l'accelerazione regge solo su superfici quadratiche. In più, l'accelerazione non funziona bene in presenza di rumore stocastico.

Noise Smoothing Ragione probabilmente più importante in pratica: il momentum è un *running average* dei gradienti. Dove SGD richiederebbe di mediare molti aggiornamenti prima di fare un passo, il momentum lo fa automaticamente, rendendo ogni singolo aggiornamento una buona approssimazione della direzione corretta.

In sintesi, sia l'accelerazione che il noise smoothing contribuiscono alle buone prestazioni del momentum SGD. Il noise smoothing evita il rimbalzamento sul "pavimento" del minimo (bottom of the valley), mentre l'accelerazione velocizza la discesa nelle regioni a bassa curvatura.

6.5 Learning Rate Adattativi Separati

6.5.1 Motivazione

In una rete multi-layer i learning rate ottimali possono variare molto tra i pesi:

- Le ampiezze dei gradienti sono spesso molto diverse tra i layer, specialmente con pesi iniziali piccoli.
- Il fan-in di un'unità determina l'entità dell'effetto di overshoot causato dalla modifica simultanea di molti pesi in ingresso.

La strategia è usare un **learning rate globale** (impostato a mano) moltiplicato per un **guadagno locale** $g_{ij}(t)$ determinato empiricamente per ogni peso.

6.5.2 Determinazione dei Guadagni Locali

Si parte da $g_{ij}(t_0) = 1$ per ogni peso. Poi, ad ogni step:

$$\Delta w_{ij} = -\epsilon g_{ij} \frac{\partial E}{\partial w_{ij}} \quad (6.9)$$

$$g_{ij}(t) = \begin{cases} g_{ij}(t-1) + 0.05 & \text{se } \frac{\partial E}{\partial w_{ij}}(t) \cdot \frac{\partial E}{\partial w_{ij}}(t-1) > 0 \\ g_{ij}(t-1) \times 0.95 & \text{altrimenti} \end{cases} \quad (6.10)$$

Aumenti additivi e decrementi moltiplicativi garantiscono che i grandi guadagni decadano rapidamente non appena iniziano le oscillazioni. I guadagni vengono limitati in un intervallo ragionevole (es. $[0.1, 10]$ oppure $[0.01, 100]$).

Analogia: AIMD nel controllo della congestione TCP

Il meccanismo di aumento additivo e decremento moltiplicativo (AIMD) è lo stesso principio usato nel controllo della congestione TCP: aumenta linearmente la finestra di congestione finché non si verifica una perdita, poi la dimezza. Questo garantisce robustezza e veloce recupero dalle oscillazioni.

Combinare con il momentum. I learning rate adattativi possono essere combinati con il momentum usando l'accordo di segno tra il gradiente corrente e la velocità del peso (Jacobs, 1989). Nota che i learning rate adattativi gestiscono solo effetti allineati agli assi (derivate parziali), mentre il momentum non dipende dall'allineamento degli assi.

6.6 rprop e rmsprop

6.6.1 rprop: Solo il Segno del Gradiente

Quando l'ampiezza del gradiente varia molto tra pesi diversi e durante l'addestramento, è difficile scegliere un unico learning rate globale. Per il **full batch learning** si può ovviare usando solo il *segno* del gradiente:

- Tutti gli aggiornamenti di peso hanno la stessa ampiezza.
- Si esce rapidamente dalle regioni a gradiente quasi nullo (plateau).

rprop combina l'uso del solo segno del gradiente con l'adattamento della dimensione del passo per ogni peso separatamente:

- Se i segni degli ultimi due gradienti *concordano*: moltiplica la dimensione del passo per 1.2.
- Altrimenti: moltiplica per 0.5.
- Limita la dimensione del passo nell'intervallo $[10^{-6}, 50]$.

Perché rprop non funziona con i mini-batch. SGD media implicitamente i gradienti su mini-batch successivi (quando il learning rate è piccolo). rprop, invece, aggiorna la dimensione del passo sulla base del *segno* in ogni mini-batch separatamente. Se un peso riceve gradiente $+0.1$ per nove mini-batch e -0.9 per uno, rprop incrementerebbe il peso nove volte e lo decremterà una sola volta, portando a un'esplosione del peso. Questo motiva la necessità di rmsprop.

6.6.2 rmsprop: Versione Mini-Batch di rprop

rmsprop mantiene una media mobile del quadrato del gradiente per ogni peso e divide il gradiente per la radice di questa media:

$$\text{MeanSquare}(w, t) = 0.9 \text{ MeanSquare}(w, t - 1) + 0.1 \left(\frac{\partial E}{\partial w} \right)^2, \quad (6.11)$$

$$\Delta w = - \frac{\eta}{\sqrt{\text{MeanSquare}(w, t)}} \frac{\partial E}{\partial w}. \quad (6.12)$$

Dividere per $\sqrt{\text{MeanSquare}(w, t)}$ normalizza il gradiente rispetto alla sua ampiezza recente, ottenendo l'effetto di rprop ma in modo compatibile con i mini-batch. I valori suggeriti da Hinton sono $\gamma = 0.9$ e $\eta = 0.001$.

6.7 Metodi Adattativi Avanzati

6.7.1 Adagrad

Adagrad adatta il learning rate a ciascun parametro, effettuando aggiornamenti più grandi per parametri rari e più piccoli per quelli frequenti. Definendo $g_{t,i} = \nabla_{\theta} J(\theta_{t,i})$, l'aggiornamento SGD standard diventa:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i}, \quad (6.13)$$

dove $G_t \in \mathbb{R}^{d \times d}$ è una matrice diagonale il cui elemento (i, i) è la **somma dei quadrati di tutti i gradienti passati** rispetto a θ_i fino al passo t , e $\epsilon \approx 10^{-8}$ evita la divisione per zero.

Vantaggi e limiti. Adagrad elimina la necessità di regolare manualmente il learning rate (default 0.01). Il limite principale è che la somma dei gradienti al quadrato cresce monotonamente: il learning rate si riduce fino a diventare infinitesimo, bloccando l'apprendimento.

6.7.2 Adadelta

Adadelta risolve il problema di Adagrad sostituendo la somma cumulativa dei quadrati del gradiente con una *media esponenzialmente decrescente*:

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma) g_t^2. \quad (6.14)$$

L'aggiornamento diventa:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\text{RMS}[\Delta\theta]_{t-1}}{\text{RMS}[g]_t} g_t, \quad (6.15)$$

dove $\text{RMS}[g]_t = \sqrt{E[g^2]_t + \epsilon}$ e $\text{RMS}[\Delta\theta]_{t-1}$ è la radice del mean square degli aggiornamenti passati (anch'esso calcolato con media esponenziale). In questo modo il learning rate viene eliminato dall'equazione e non deve essere impostato.

Relazione con rmsprop. RMSprop e Adadelta sono stati sviluppati indipendentemente nello stesso periodo per risolvere lo stesso problema. RMSprop corrisponde esattamente al primo vettore di aggiornamento di Adadelta.

6.7.3 Adam: Adaptive Moment Estimation

Adam (?) combina l'idea di rmsprop (media del quadrato dei gradienti) con quella del momentum (media dei gradienti), mantenendo stime del *primo momento* (media) e del *secondo momento non centrato* (varianza) dei gradienti:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (6.16)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2. \quad (6.17)$$

Poiché $m_0 = v_0 = \mathbf{0}$, le stime sono distorte verso zero nelle prime iterazioni. Si correggono con:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (6.18)$$

L'aggiornamento finale è:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t. \quad (6.19)$$

I valori di default suggeriti dagli autori sono $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Interpretazione. Quando i gradienti rimangono approssimativamente costanti (discesa uniforme), la varianza è quasi nulla e $v_t \approx m_t^2$, quindi $\hat{m}_t / \sqrt{\hat{v}_t} \approx 1$: il passo è dell'ordine di η . Quando i gradienti cambiano rapidamente, $\sqrt{\hat{v}_t} \gg \hat{m}_t$ e il passo diventa molto più piccolo di η . Adam adatta quindi la dimensione del passo in modo automatico per ciascun peso.

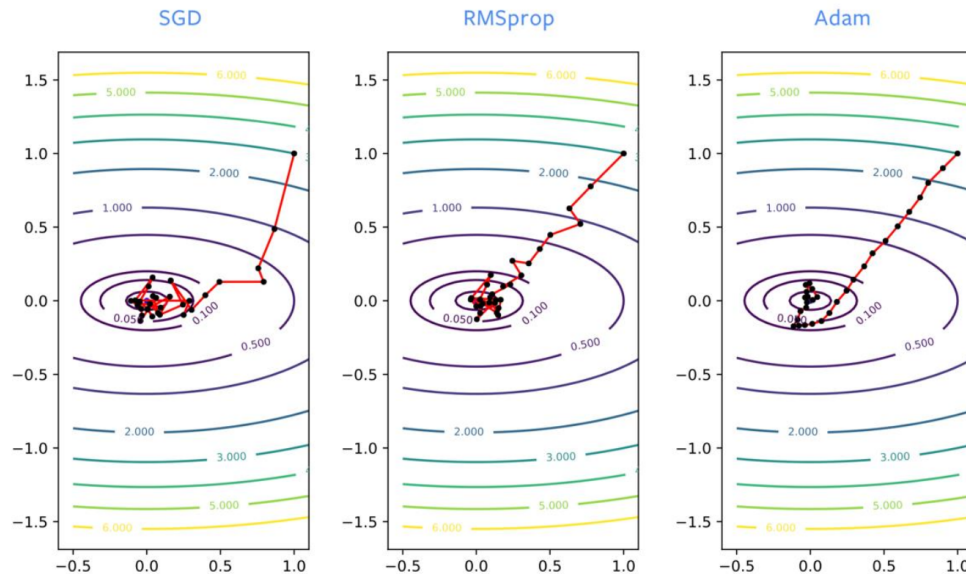


Figura 6.5: Confronto delle traiettorie di SGD, RMSprop e Adam sulla stessa superficie di errore. Adam converge in modo più diretto verso il minimo grazie alla combinazione di momentum e normalizzazione adattiva.

Limitazioni di Adam

- Su problemi semplici è dimostrabile che Adam **non converge**.
- Tende a produrre **errori di generalizzazione** maggiori rispetto a SGD, specialmente in computer vision: trovare il minimo locale più vicino (invece del minimo "piatto" favorito da SGD) porta a minori capacità di generalizzazione.
- Richiede **3 buffer** invece di 1 (pesi, m_t , v_t): per modelli da miliardi di parametri questo può diventare limitante in memoria.
- Sono necessari 2 iperparametri di momentum da regolare invece di 1.

6.7.4 AdamW: Adam con Weight Decay Corretto

Le librerie di deep learning implementano tipicamente la *L2 regularization* aggiungendo λw al gradiente prima dell'aggiornamento. Tuttavia questo non è equivalente al weight decay per gli ottimizzatori adattativi: il termine di regolarizzazione viene normalizzato da $\sqrt{\hat{v}_t}$, quindi i pesi con gradienti grandi vengono regolarizzati *meno* di quelli con gradienti piccoli. **AdamW** separa il weight decay dall'aggiornamento adattivo:

$$\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \eta_t \left(\frac{\alpha \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right) - \eta_t w \mathbf{x}_{t-1}. \quad (6.20)$$

Il termine di regolarizzazione $w \mathbf{x}_{t-1}$ è proporzionale al solo valore del peso e non viene influenzato dalla normalizzazione adattiva, replicando il comportamento del weight decay in SGD e migliorando la generalizzazione.

6.7.5 Lion: EvoLved Sign Momentum

Lion (?) è un ottimizzatore scoperto tramite *program search* (ricerca evolutiva sullo spazio dei programmi). Richiede meno memoria di Adam (mantiene solo il momentum, non il secondo momento) e produce aggiornamenti di ampiezza uniforme grazie all'operazione `sign`:

Listing 6.1: Pseudocodice di Lion.

```

1 def train(weight, gradient, momentum, lr):
2     update = interp(gradient, momentum, beta1) # (1-b1)*m +
        b1*g
3     update = sign(update)
4     momentum = interp(gradient, momentum, beta2)
5     update = update + weight * lambda_wd # weight
        decay
6     update = update * lr
7     return update, momentum

```

Le prestazioni di Lion crescono con la dimensione del mini-batch e richiedono un learning rate più piccolo rispetto ad Adam a causa della norma maggiore prodotta dalla funzione `sign`.

6.8 Metodi di Secondo Ordine

6.8.1 Riduzione dell'Errore e Rapporto Gradiente/Curvatura

La riduzione massima dell'errore percorrendo una direzione **d** dipende dal **rapporto gradiente/curvatura**. Una buona direzione ha un rapporto elevato anche se il gradiente è piccolo. Per trovare queste direzioni si può usare la matrice di curvatura (Hessiana):

$$H_{i,j} = \frac{\partial^2 f}{\partial x_i \partial x_j}. \quad (6.21)$$

Gradiente Vettore delle derivate prime di un campo scalare.

Jacobiano Matrice dei gradienti per le componenti di un campo vettoriale ($J_{i,j} = \partial F_i / \partial x_j$).

Hessiana Matrice delle derivate miste di secondo ordine di un campo scalare.

6.8.2 Metodo di Newton

Il metodo di Newton moltiplica il vettore del gradiente per l'inversa della matrice di curvatura H :

$$\Delta \mathbf{w} = -\epsilon H(\mathbf{w})^{-1} \frac{dE}{d\mathbf{w}}. \quad (6.22)$$

Su una superficie quadratica reale converge al minimo in un unico passo. Il problema pratico è che con un milione di pesi la matrice di curvatura ha un trilione di termini: **inverirla è computazionalmente intrattabile**.

Matrici di curvatura. Ogni elemento della matrice specifica come il gradiente in una direzione cambia spostandosi in un'altra. I termini fuori diagonale corrispondono alle *torsioni* della superficie di errore. La ragione per cui la discesa del gradiente produce zig-zag è che aggiornare un peso muta il gradiente di tutti gli altri.

6.8.3 Hessian-Free Optimization e Gradiente Coniugato

Il metodo **Hessian-Free** (HF) approssima la matrice di curvatura e, assumendo che quell'approssimazione sia corretta, minimizza l'errore usando il **gradiente coniugato** (CG). Poi aggiorna l'approssimazione e minimizza di nuovo.

Gradiente Coniugato. Invece di invertire H in un solo passo, CG usa una sequenza di passi ognuno dei quali minimizza l'errore lungo una direzione:

- Ogni nuova direzione è *coniugata* alle precedenti: muoversi in essa non altera i gradienti nelle direzioni già ottimizzate.
- Dopo N passi è garantita la convergenza al minimo di una superficie quadratica N -dimensionale.
- Applicato a superfici non quadratiche (CG non lineare) funziona comunque bene nella pratica.

L'ottimizzatore HF usa CG sulla superficie quadratica approssimata (dove CG è ottimale). Per le RNN è importante aggiungere un vincolo che impedisce di modificare troppo le attività nascoste da un'iterazione all'altra.

6.9 Riepilogo dei Metodi di Ottimizzazione

Tabella 6.1: Guida alla scelta dell'ottimizzatore.

Scenario	Ottimizzatore	Note
Dataset piccolo ($\leq 10k$) / bassa ridondanza	Full-batch (CG, L-BFGS, rprop)	Convergenza precisa
Dataset grande e ridondante	Mini-batch SGD + Momentum	Default per CV
Gradient sparso / NLP / embedding	Adam / AdamW	Adattività per feature rare
Modelli grandi, generalizzazione critica	AdamW o SGD + momentum	AdamW corregge weight decay
Memoria limitata, batch grande	Lion	Un solo buffer di stato

Regole operative.

1. Inizializzare i pesi con **Xavier/Glorot** (sigmoide/tanh) o **He** (ReLU); mai inizializzare a zero tutti i pesi hidden.

2. Normalizzare gli input (media zero, varianza unitaria) o usare Batch Normalization per accelerare la convergenza.
3. Per reti grandi preferire **AdamW** come default; passare a SGD con momentum e cosine annealing per massimizzare la generalizzazione.
4. Non abbassare il learning rate troppo presto: attendere che la loss si sia stabilizzata su un plateau.
5. Usare mini-batch bilanciati tra le classi (cfr. Sezione 6.2.1) e limitare la dimensione del mini-batch tenendo conto che batch più grandi richiedono learning rate più alti (linear scaling rule).
6. Per i learning rate adattativi limitare sempre i guadagni locali in un intervallo ragionevole e usare mini-batch grandi per ridurre il rumore nelle stime dei segni del gradiente.

Come sottolinea [Bishop and Bishop \(2024\)](#), non esiste una ricetta universale perché le reti neurali differiscono enormemente (reti molto profonde, reti ricorrenti, reti wide-shallow) e i task pongono requisiti diversi (accuratezza dei pesi, presenza di feature rare, ...). La scelta dell'ottimizzatore va quindi fatta considerando congiuntamente architettura, dataset e obiettivo di generalizzazione.

Capitolo 7

Normalization Layers

I **layer di normalizzazione** sono componenti ormai indispensabili nelle reti neurali profonde. Senza di essi, addestrare reti molto profonde è difficoltoso: i gradienti esplodono o svaniscono, i tempi di convergenza si allungano e la scelta del learning rate diventa critica. Questo capitolo introduce il problema del *covariate shift*, la soluzione più nota (Batch Normalization), le varianti sviluppate per contesti diversi e una discussione critica su *perché* la normalizzazione funziona davvero (Bishop and Bishop, 2024).

7.1 Covariate Shift e Internal Covariate Shift

Il **covariate shift** è un cambiamento nella distribuzione degli input di un sistema di apprendimento: tipicamente la distribuzione durante il test differisce da quella durante il training. Questo fenomeno danneggia le prestazioni e viene normalmente affrontato con tecniche di *domain adaptation*.

Nelle reti neurali profonde esiste un fenomeno analogo ma interno: l'**internal covariate shift**. Durante il training, gli aggiornamenti dei pesi nei layer inferiori modificano continuamente la distribuzione degli input per i layer superiori. I layer successivi devono quindi adattarsi non solo a nuovi dati, ma a distribuzioni che cambiano ad ogni step di ottimizzazione, rallentando significativamente il training.

7.2 Batch Normalization

7.2.1 Idea di Base

La **Batch Normalization** (BN) (?) è un metodo di normalizzazione applicato ai layer interni della rete. Normalmente gli input alla rete vengono normalizzati nell'intervallo $[0, 1]$ o $[-1, 1]$ oppure portati a media zero e varianza unitaria. BN estende questa pratica ai layer intermedi, eseguendo un'*operazione di whitening* sulle attivazioni ad ogni step.

7.2.2 Perché l'Approccio Naïf Non Funziona

L'approccio banale sarebbe: addestrare su un batch, aggiornare i parametri, poi normalizzare le attivazioni. Questo però **non funziona**: porta a un'esplosione dei bias mentre i parametri di distribuzione (media e varianza) rimangono invariati. Il motivo è che l'ottimizzazione con discesa del gradiente non tiene conto della normalizzazione che avverrà al passo successivo, ignorando sistematicamente il suo effetto.

7.2.3 La Soluzione: un Layer di Normalizzazione Differenziabile

La soluzione proposta da ? è aggiungere un **layer esplicito** che esegua la normalizzazione come parte del grafo computazionale, in modo che il gradiente possa “vedere” la normalizzazione e fare aggiustamenti se necessario.

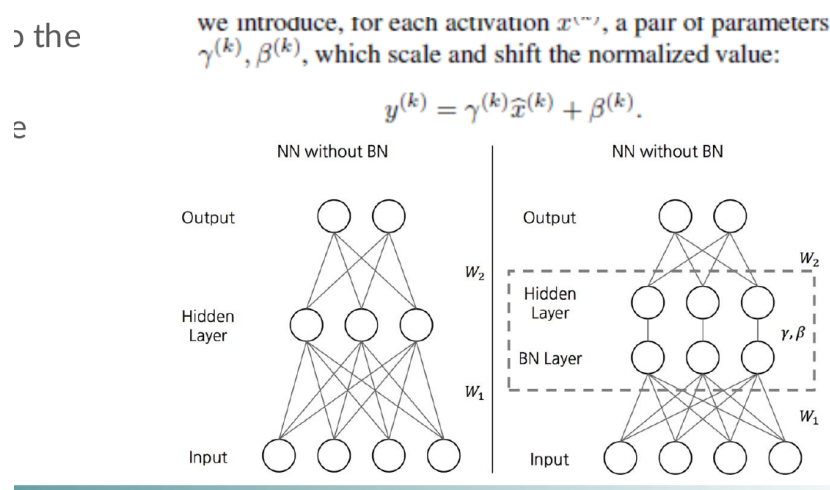


Figura 7.1: Confronto tra una rete senza BN (sinistra) e con BN (destra). Il BN Layer, inserito tra il layer lineare e quello successivo, introduce i parametri apprendibili γ e β che scalano e traslano le attivazioni normalizzate.

Algoritmo formale. Dato un mini-batch $\mathcal{B} = \{x_1, \dots, x_m\}$ e i parametri apprendibili γ, β , il layer BN calcola:

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ media del mini-batch} \quad (7.1)$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ varianza del mini-batch} \quad (7.2)$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalizzazione (z-score)} \quad (7.3)$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scala e trasla} \quad (7.4)$$

dove $\epsilon \approx 10^{-5}$ evita la divisione per zero.

```

Input: Values of  $x$  over a mini-batch:  $\mathcal{B} = \{x_{1..m}\}$ ;
          Parameters to be learned:  $\gamma, \beta$ 
Output:  $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$ 

 $\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$  // mini-batch mean
 $\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$  // mini-batch variance
 $\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$  // normalize
 $y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i)$  // scale and shift

```

Figura 7.2: Pseudocodice dell'algoritmo di Batch Normalization come presentato nel paper originale (?). Le quattro operazioni (media, varianza, normalizzazione, scale-and-shift) costituiscono il forward pass del layer BN.

7.2.4 Parametri Apprendibili γ e β

Senza γ e β , l'output del layer BN avrebbe sempre media 0 e deviazione standard 1, limitando la capacità rappresentativa della rete. I parametri γ (fattore di scala) e β (bias) permettono alla rete di operare su qualsiasi range di valori.

Un aspetto cruciale è che γ e β **non annullano la normalizzazione**: sono parametri appresi per discesa del gradiente e risultano molto più stabili di μ e σ (che cambiano ad ogni mini-batch). Il layer ha persino la capacità di imparare la funzione identità (per “de-normalizzare” se necessario), ma lo fa solo se il task lo richiede.

7.3 Vantaggi della Batch Normalization

7.3.1 Benefici Teorici (Motivazione Originale)

Riduzione dell'Internal Covariate Shift La motivazione originale del paper. Con BN, ogni neurone tende ad avere un'attivazione approssimativamente gaussiana (solitamente non attivo, a volte parzialmente attivo, raramente molto attivo). Il covariate shift è indesiderabile perché i layer successivi devono continuamente adattarsi a cambiamenti nel *tipo* di distribuzione, non solo nei parametri.

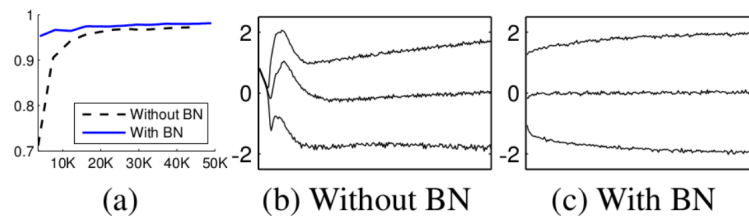
Nota critica

Questa motivazione è stata confutata sperimentalmente. Ricerche successive hanno dimostrato che BN non riduce significativamente l'internal covariate shift. I veri benefici hanno spiegazioni diverse (cfr. Sezione 7.6).

Riduzione di vanishing/exploding gradients Con attivazioni distribuite normalmente, il segnale del gradiente si propaga in modo più stabile tra i layer. Senza BN, attivazioni basse in un layer possono propagarsi a cascata producendo attivazioni sempre più piccole nei layer successivi.

7.3.2 Benefici Pratici

- **Riduce i tempi di training:** meno problemi di gradienti anomali $\Rightarrow$ convergenza più rapida.
- **Riduce il bisogno di regolarizzazione** (dropout, L2): le medie e varianze calcolate sul batch introducono rumore stocastico che impedisce alla rete di memorizzare le risposte corrette per ogni esempio (effetto implicito di regolarizzazione).
- **Permette learning rate più alti:** minor pericolo di esplosione del gradiente consente di usare η più grandi senza instabilità.
- **Abilita funzioni di attivazione saturanti in reti profonde:** la normalizzazione impedisce alle attivazioni di finire nelle zone di saturazione della sigmoide (valori molto alti o molto bassi), dove il gradiente è quasi nullo.



↓ applied to MNIST (a), and activations of a randomly selected neuron over time

Figura 7.3: Risultati sperimentali su MNIST (?). (a) Accuratezza durante il training: con BN si raggiunge convergenza più rapida e accuratezza finale più alta. (b) Senza BN: l'attivazione di un neurone casuale oscilla ampiamente nel tempo. (c) Con BN: le attivazioni si stabilizzano presto e rimangono nella stessa distribuzione per tutto il training.

7.4 Posizionamento del Layer BN

Il BN layer viene tipicamente inserito **tra il layer lineare (o convolutivo) e la funzione di attivazione**. Questa posizione permette di normalizzare le attivazioni prima che vengano passate alla non-linearità, evitando la saturazione.

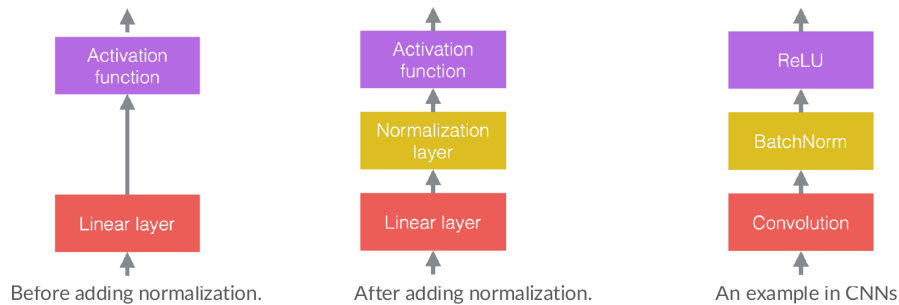


Figura 7.4: Posizionamento del layer di normalizzazione nell'architettura. Sinistra: rete senza normalizzazione (Linear $\rightarrow$ Activation). Centro: rete con normalizzazione (Linear $\rightarrow$ Norm $\rightarrow$ Activation). Destra: esempio tipico in una CNN (Convolution $\rightarrow$ BatchNorm $\rightarrow$ ReLU).

Fase di inferenza. Durante il training, μ e σ^2 vengono calcolati sul mini-batch corrente. Durante l'inferenza, non si può usare lo stesso approccio (il batch potrebbe contenere un solo esempio). Si usano invece le **statistiche mobili** (running mean e running variance) accumulate durante il training:

$$\hat{\mu} \leftarrow (1 - \alpha) \hat{\mu} + \alpha \mu_B, \quad \hat{\sigma}^2 \leftarrow (1 - \alpha) \hat{\sigma}^2 + \alpha \sigma_B^2. \quad (7.5)$$

In PyTorch questo comportamento è gestito automaticamente con `model.train()` e `model.eval()`.

7.5 Varianti di Normalizzazione

La scelta di *quali* elementi del tensore usare per calcolare media e varianza dà origine a diverse varianti. Per un mini-batch di N immagini di altezza H , larghezza W e C canali:

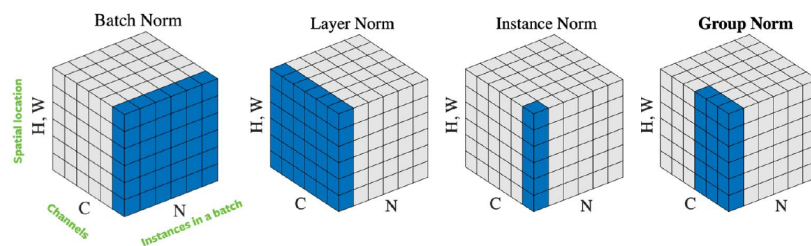


Figura 7.5: Le quattro principali operazioni di normalizzazione. In blu sono evidenziati gli elementi usati per calcolare media e varianza per ogni singola statistica. Assi: N (istanze nel batch), C (canali), $H \times W$ (posizioni spaziali).

7.5.1 Batch Normalization (BN)

Normalizza **lungo la dimensione del batch** N : per ogni canale c , calcola media e varianza su tutti gli elementi (n, c, h, w) al variare di n, h, w . Produce statistiche stabili per batch grandi ma è inadatta a batch piccoli (le stime di μ e σ diventano rumorose) o a sequenze di lunghezza variabile.

$$\mu_c = \frac{1}{N \cdot H \cdot W} \sum_{n,h,w} x_{n,c,h,w}, \quad \sigma_c^2 = \frac{1}{N \cdot H \cdot W} \sum_{n,h,w} (x_{n,c,h,w} - \mu_c)^2. \quad (7.6)$$

7.5.2 Layer Normalization (LN)

Normalizza **lungo i canali** per ogni singola istanza: per ogni campione n , calcola media e varianza su tutti i canali e le posizioni spaziali. Non dipende dal batch, quindi funziona perfettamente con batch di dimensione 1 ed è la scelta standard nei **Transformer** (dove le sequenze hanno lunghezza variabile).

$$\mu_n = \frac{1}{C \cdot H \cdot W} \sum_{c,h,w} x_{n,c,h,w}. \quad (7.7)$$

In PyTorch: `nn.LayerNorm`.

7.5.3 Instance Normalization (IN)

Normalizza **per canale per istanza**: media e varianza sono calcolate su $H \times W$ per ogni coppia (n, c) indipendentemente. Utile in **style transfer** e generazione di immagini, dove la normalizzazione deve essere specifica per ogni immagine e ogni feature map.

$$\mu_{n,c} = \frac{1}{H \cdot W} \sum_{h,w} x_{n,c,h,w}. \quad (7.8)$$

In PyTorch: `nn.InstanceNorm2d`.

7.5.4 Group Normalization (GN)

Compromesso tra LN e IN: divide i C canali in G gruppi e normalizza ogni gruppo separatamente per ogni istanza. Non dipende da N e mantiene buone prestazioni anche con batch piccoli (es. $N = 1$ o $N = 2$ tipici in object detection con immagini ad alta risoluzione).

$$\mu_{n,g} = \frac{1}{(C/G) \cdot H \cdot W} \sum_{c \in \text{gruppo } g, h, w} x_{n,c,h,w}. \quad (7.9)$$

In PyTorch: `nn.GroupNorm`. Con $G = C$ si ottiene Instance Norm; con $G = 1$ si ottiene Layer Norm.

Tabella 7.1: Confronto delle varianti di normalizzazione.

Metodo	Calcola su	Dipende da N ?	Uso tipico	PyTorch
Batch Norm	N, H, W per canale	Sì	CNN classifica	BatchNorm2d
Layer Norm	C, H, W per istanza	No	Transformer, RNN	LayerNorm
Instance Norm	H, W per (n,c)	No	Style transfer	InstanceNorm2d
Group Norm	gruppo di canali per istanza	No	Detection, segm.	GroupNorm

7.6 Perché la Normalizzazione Funziona Davvero?

Nonostante la normalizzazione funzioni bene in pratica, le ragioni della sua efficacia sono ancora oggetto di dibattito. La motivazione originale (riduzione dell’internal covariate shift) è stata confutata sperimentalmente. I meccanismi attualmente più accreditati sono tre:

Effetto di ottimizzazione Le reti con layer di normalizzazione sono più facili da ottimizzare e permettono l’uso di learning rate più alti. La normalizzazione *liscia* la superficie di perdita, rendendola più uniforme e riducendo la necessità di seguire direzioni di gradiente complesse.

Effetto di regolarizzazione Le stime di media e varianza calcolate su un mini-batch sono **rumorose** (cambiano da batch a batch). Questo rumore aggiuntivo agisce come un regolarizzatore implicito, migliorando la generalizzazione in modo simile al dropout. La rete non può memorizzare valori fissi perché la normalizzazione li cambia ad ogni step.

Riduzione della sensibilità all’inizializzazione dei pesi Grazie alla normalizzazione, l’inizializzazione dei pesi diventa meno critica: la distribuzione delle attivazioni viene corretta automaticamente al primo forward pass.

Come conseguenza pratica, la normalizzazione permette di essere più “distratti” nella progettazione: si possono combinare quasi qualsiasi blocco architetturale e avere buone probabilità di convergenza senza dover analizzare il condizionamento della rete.

7.7 Riepilogo e Linee Guida Pratiche

Regole operative.

1. Per **CNN su immagini con batch ≥ 16** : usare `nn.BatchNorm2d` dopo ogni layer conv, prima della ReLU. Ricordare di chiamare `model.eval()` durante l’inferenza per usare le running statistics.
2. Per **Transformer e modelli sequence-to-sequence**: usare `nn.LayerNorm`; BN è inadatta a sequenze di lunghezza variabile e batch piccoli.
3. Per **object detection / segmentazione semantica** con batch piccoli ($N \leq 4$): preferire `nn.GroupNorm` (tipicamente $G = 32$) che non degrada al calare di N .
4. Per **style transfer e GAN**: `nn.InstanceNorm2d` normalizza per immagine, preservando le statistiche di stile specifiche per ciascun esempio.

5. Non usare BN dopo layer con batch size 1 o in presenza di sequenze molto corte: le stime di μ e σ saranno inaffidabili.
6. Ricordarsi che BN introduce un bias implicito per ciascun canale tramite β : quando BN segue un layer lineare/conv con bias, il bias del layer precedente è ridondante e può essere disabilitato (`bias=False`).

Come sottolinea [Bishop and Bishop \(2024\)](#), i layer di normalizzazione sono oggi parte integrante di quasi ogni architettura di successo. La loro semplicità implementativa, unita all'impatto significativo su velocità di convergenza e generalizzazione, li rende uno strumento imprescindibile nel toolkit del Deep Learning moderno.

Capitolo 8

Reti Neurali Convoluzionali

Le **Reti Neurali Convoluzionali** (Convolutional Neural Networks, CNN o ConvNet) sono una classe di architetture progettate specificamente per elaborare dati strutturati su una griglia, come le immagini (griglia 2D di pixel). La loro idea fondamentale risale ai lavori di Fukushima sulla gerarchia semplice/complessa della corteccia visiva e alla prima CNN sviluppata da Yann LeCun a Toronto tra il 1988 e il 1989, successivamente consolidata nel celebre lavoro su LeNet-5 per il riconoscimento di cifre scritte a mano (Bishop and Bishop, 2024).

8.1 Limiti delle Reti Completamente Connesse sulle Immagini

Nelle reti *fully connected* classiche, ogni neurone riceve in ingresso tutti i valori del volume di input. Questo approccio **non scala** alle immagini reali:

- Un'immagine CIFAR-10 di dimensioni $32 \times 32 \times 3$ genera già $32 \cdot 32 \cdot 3 = 3072$ pesi per un singolo neurone del primo strato nascosto.
- Un'immagine di dimensioni più realistiche, ad esempio $200 \times 200 \times 3$, porterebbe a $200 \cdot 200 \cdot 3 = 120\,000$ pesi per neurone.
- Con molti neuroni i parametri si accumulano rapidamente, determinando **overfitting** e un costo computazionale proibitivo.

La connettività totale è dunque uno spreco: pixel vicini sono altamente correlati, e le feature visive utili (bordi, texture, forme) compaiono in posizioni diverse dell'immagine ma con la stessa struttura locale.

8.2 Volumi 3D e Connettività Locale

8.2.1 Organizzazione tridimensionale

A differenza di una rete fully connected, i layer di una ConvNet organizzano le proprie unità in **tre dimensioni**: larghezza (W), altezza (H) e profondità (D). Il termine

profondità qui si riferisce alla terza dimensione del *volume di attivazione* (per esempio i 3 canali RGB di un'immagine), *non* al numero di layer della rete.

8.2.2 Connettività locale

Per input ad alta dimensionalità è impraticabile collegare ogni unità a tutti i valori del volume precedente. La soluzione è connettere ogni unità soltanto a una **regione locale** dell'input. L'estensione spaziale di questa connessione è un iperparametro chiamato **campo recettivo** (receptive field) dell'unità, che coincide con la dimensione del filtro.

Due proprietà fondamentali:

1. **Connessione locale in 2D**: lungo larghezza e altezza ogni unità vede solo una piccola finestra dell'input.
2. **Connessione piena in profondità**: lungo la dimensione di profondità la connessione si estende all'intera profondità del volume di input (ad esempio tutti e 3 i canali RGB).

8.3 Il Layer di Convoluzione

8.3.1 Operazione di convoluzione

Il layer convoluzionale fa scorrere (*slide*) un filtro $\mathbf{w}$ di dimensioni $F \times F \times D$ su tutte le posizioni spaziali del volume di input. In ogni posizione viene calcolato un **prodotto scalare** tra il filtro e il corrispondente blocco dell'input, producendo un singolo scalare di attivazione:

$$a = \mathbf{w}^\top \mathbf{x} + b, \quad (8.1)$$

dove $\mathbf{x}$ è il vettore di $F \cdot F \cdot D$ valori del blocco corrente e b è il bias del filtro. Per un filtro $5 \times 5 \times 3$ il prodotto scalare ha dimensione $5 \cdot 5 \cdot 3 = 75$.

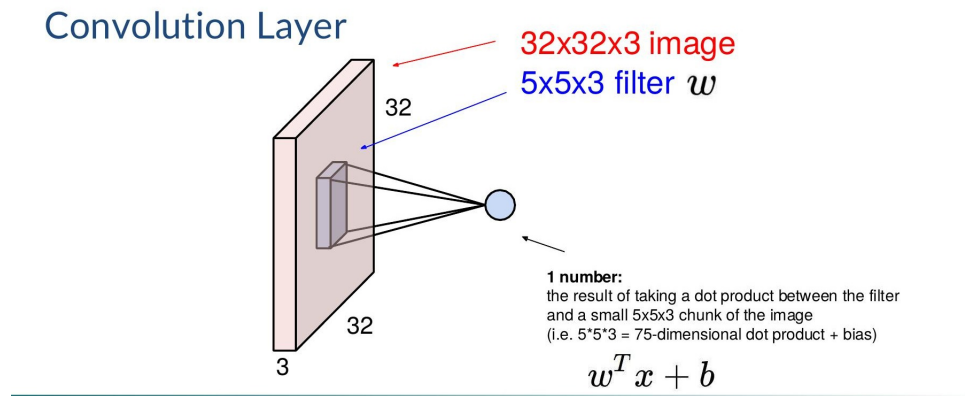


Figura 8.1: Convoluzione di un filtro $5 \times 5 \times 3$ su un'immagine $32 \times 32 \times 3$: ogni neurone di output calcola il prodotto scalare 75-dimensionale $\mathbf{w}^\top \mathbf{x} + b$ tra il filtro e il blocco locale dell'immagine.

Il layer si chiama *convoluzionale* perché l'operazione è formalmente legata alla convoluzione discreta tra due segnali:

$$(f * g)[x, y] = \sum_{n_1=-\infty}^{+\infty} \sum_{n_2=-\infty}^{+\infty} f[n_1, n_2] \cdot g[x - n_1, y - n_2], \quad (8.2)$$

ovvero una moltiplicazione elemento per elemento e somma tra il filtro e il segnale (immagine).

8.3.2 Mappe di attivazione

Facendo scorrere un singolo filtro su tutto il volume di input si ottiene una **mappa di attivazione** (activation map) bidimensionale. Se si usano K filtri si ottengono K mappe, che vengono impilate lungo la dimensione di profondità per formare il volume di output.

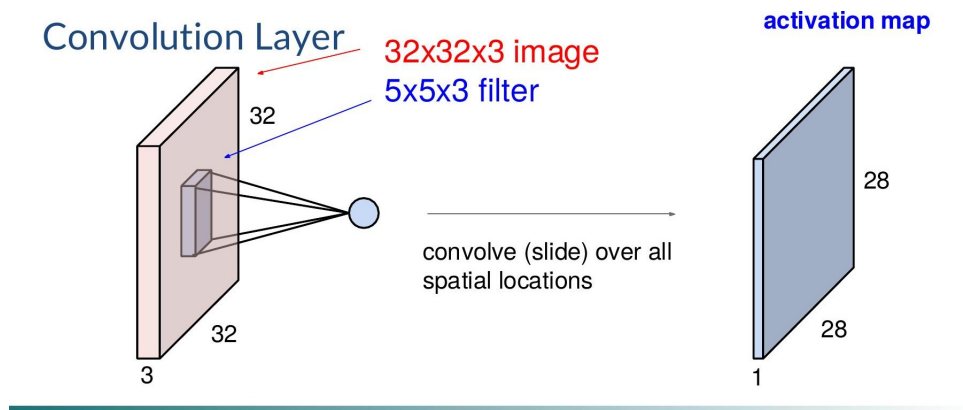


Figura 8.2: Un singolo filtro $5 \times 5 \times 3$ applicato sull'immagine $32 \times 32 \times 3$ produce una mappa di attivazione $28 \times 28 \times 1$.

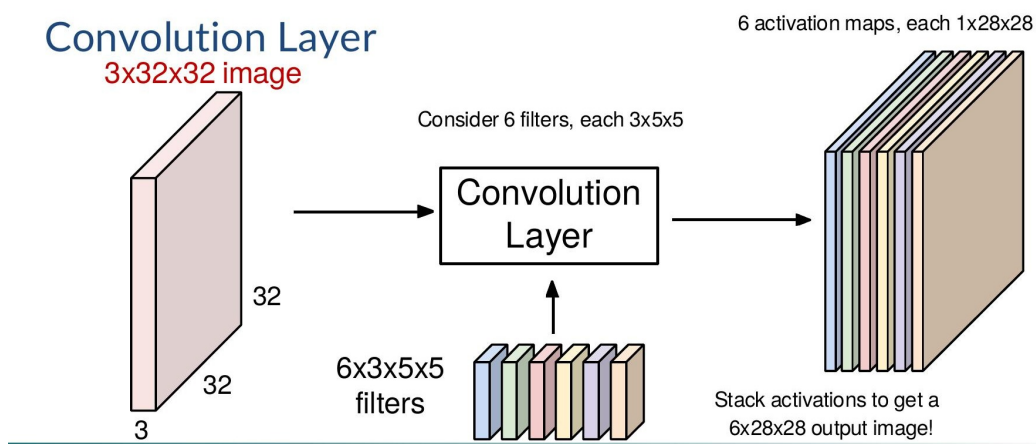


Figura 8.3: Con 6 filtri $3 \times 5 \times 5$, il layer convoluzionale produce 6 mappe di attivazione che vengono impilate per ottenere un output $6 \times 28 \times 28$.

Esempio. Per un'immagine $32 \times 32 \times 3$ con un filtro $5 \times 5 \times 3$ e stride 1, la mappa di attivazione risultante ha dimensioni $28 \times 28 \times 1$. Con 6 filtri si ottiene un volume $28 \times 28 \times 6$.

In generale, con K filtri di dimensione $F \times F \times D_{\text{in}}$:

- ogni filtro ha $F \cdot F \cdot D_{\text{in}} + 1$ parametri (incluso il bias);
- il volume di output ha profondità K ;
- il numero totale di parametri del layer è $K \cdot (F \cdot F \cdot D_{\text{in}} + 1)$.

8.3.3 Dimensioni spaziali: stride e zero-padding

La dimensione spaziale dell'output dipende da tre iperparametri:

Stride S Passo con cui il filtro scorre sull'input. Stride maggiore produce output più piccoli e sottocampiona la rappresentazione.

Zero-padding P Aggiunta di zeri intorno al bordo dell'input, che permette di controllare la dimensione dell'output.

Dimensione del filtro F Controlla il campo recettivo locale.

La formula generale per la dimensione spaziale dell'output è:

$$W_{\text{out}} = \frac{W_{\text{in}} + 2P - F}{S} + 1, \quad (8.3)$$

e analogamente per H_{out} . Se il risultato non è intero, la combinazione (F, S, P) è incompatibile con l'input dato.

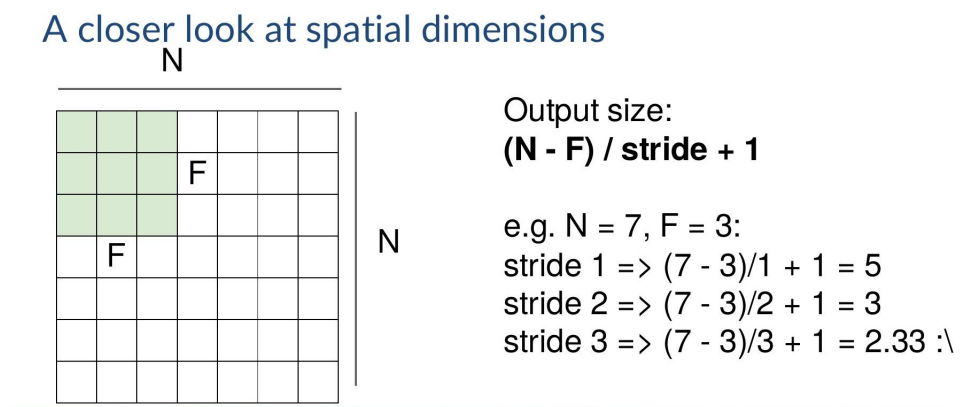


Figura 8.4: Formula per la dimensione dell'output $(N - F)/\text{stride} + 1$ con esempi numerici: $N = 7, F = 3$ con stride 1 ($\rightarrow 5$), stride 2 ($\rightarrow 3$), stride 3 (risultato non intero $\Rightarrow$ non valido).

Zero-padding per preservare la dimensione. È comune usare stride 1, filtri di dimensione $F \times F$ e zero-padding pari a $P = (F - 1)/2$: questo preserva la dimensione spaziale dell'input (formula aggiornata: $(N + 2P - F)/S + 1$). Per esempio, $F = 3 \Rightarrow P = 1$; $F = 5 \Rightarrow P = 2$; $F = 7 \Rightarrow P = 3$.

In practice: Common to zero pad the border

0	0	0	0	0	0		
0							
0							
0							
0							

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

(recall:)

$$(N + 2P - F) / \text{stride} + 1$$

Figura 8.5: Zero-padding di 1 pixel intorno a un input 7×7 : con filtro 3×3 e stride 1, l'output è ancora 7×7 (formula: $(7 + 2 \cdot 1 - 3) / 1 + 1 = 7$).

8.3.4 Convoluzione 1×1

Un filtro di dimensione $1 \times 1 \times D_{\text{in}}$ esegue un prodotto scalare D_{in} -dimensionale in ogni posizione spaziale senza alcuna interazione spaziale. Questo tipo di convoluzione è utile per:

- **Riduzione di dimensionalità** lungo la profondità (collo di bottiglia), senza modificare la risoluzione spaziale.
- Aggiungere nonlinearità puntuali tra i layer convoluzionali.

8.4 Campi Recettivi

Il **campo recettivo** di un'unità è la regione dell'input originale che influenza il suo valore di attivazione. Con un kernel di dimensione K , ogni elemento dell'output dipende da una regione $K \times K$ del layer precedente. Ogni convoluzione successiva aggiunge $K - 1$ alla dimensione del campo recettivo; con L layer:

$$\text{RF} = 1 + L \cdot (K - 1). \quad (8.4)$$

Receptive Fields

Each successive convolution adds $K - 1$ to the receptive field size
 With L layers the receptive field size is $1 + L * (K - 1)$

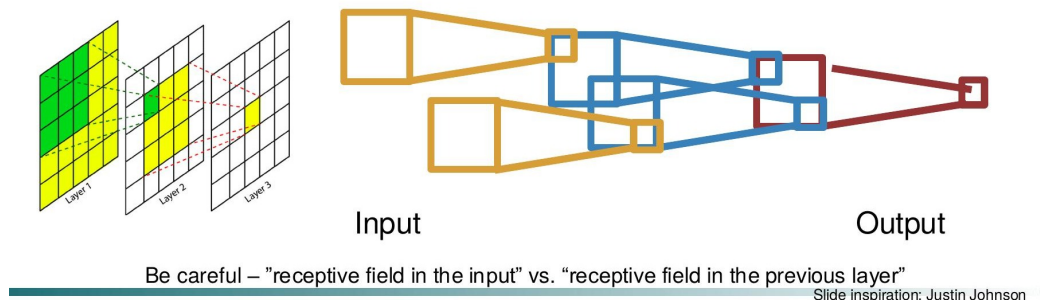


Figura 8.6: Campi recettivi attraverso tre layer convoluzionali successivi: ogni layer espande la porzione dell'input originale vista da ciascuna unità. Con L layer e kernel K , il campo recettivo totale sull'input è $1 + L(K - 1)$. Nota: “receptive field in the input” vs. “receptive field in the previous layer” sono concetti distinti.

Problema per immagini grandi. Per immagini grandi è necessario un numero molto elevato di layer affinché ogni unità dell'output “veda” l'intera immagine. La **soluzione** è il sottocampionamento (*downsampling*) all'interno della rete, ottenuto con:

- **Convoluzione con stride** > 1 : riduce direttamente la risoluzione spaziale.
- **Pooling**: layer dedicati al sottocampionamento (vedi Sezione 8.6).

8.5 Cosa Imparano i Filtri Convoluzionali?

I filtri del primo layer fungono da *template* locali sull'immagine. Dopo l'addestramento su grandi dataset, i filtri del primo layer imparano a rilevare feature di basso livello come bordi orientati in varie direzioni, colori opposti e frequenze spaziali diverse (in AlexNet, 64 filtri $3 \times 11 \times 11$). Nei layer successivi le mappe combinano le risposte precedenti, rilevando pattern di complessità crescente: angoli al secondo layer, parti di oggetti al terzo, interi oggetti agli strati più profondi.

8.6 Il Layer di Pooling

Il layer di pooling serve a **ridurre la risoluzione spaziale** delle mappe di attivazione, rendendo la rappresentazione più compatta e introducendo una certa **invarianza traslazionale**. Opera in modo *indipendente* su ciascuna mappa.

8.6.1 Max Pooling

Il **Max Pooling** è la variante più comune: data una finestra di dimensioni $P_H \times P_W$ con stride S , si prende il massimo dei valori nella finestra.

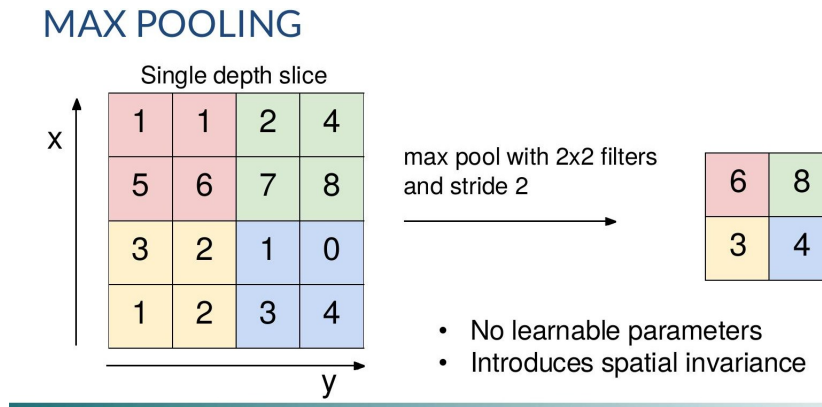


Figura 8.7: Max pooling 2×2 con stride 2 su una singola mappa 4×4 : ogni quadrante 2×2 viene sostituito dal suo valore massimo, dimezzando la risoluzione spaziale. Nessun parametro apprendibile; introduce invarianza spaziale locale.

In formule, il max pooling 2×2 con stride 2 sull'esempio delle slide produce:

$$\begin{pmatrix} 1 & 1 & 2 & 4 \\ 5 & 6 & 7 & 8 \\ 3 & 2 & 1 & 0 \\ 1 & 2 & 3 & 4 \end{pmatrix} \xrightarrow{\text{MaxPool } 2 \times 2, s=2} \begin{pmatrix} 6 & 8 \\ 3 & 4 \end{pmatrix}. \quad (8.5)$$

Proprietà chiave: nessun parametro apprendibile; invarianza spaziale locale (piccole traslazioni non cambiano l'output); riduzione spaziale tipicamente di un fattore 2.

8.6.2 Altre varianti di pooling

Average Pooling Calcola la media dei valori nella finestra.

Global Average Pooling (GAP) Riduce ogni mappa a un unico scalare (media globale). Usato in GoogLeNet e ResNet come alternativa ai layer FC finali.

LP-Norm Pooling $(\sum_i x_i^p)^{1/p}$; per $p \rightarrow \infty$ si ottiene il max pooling.

8.7 Il Layer Fully Connected

Dopo un certo numero di layer CONV/ReLU/POOL, le mappe di attivazione vengono **appiattite** (*flatten*) in un vettore unidimensionale e passate a uno o più **layer fully connected** (FC), dove ogni neurone è connesso a tutti i valori dell'input. Questi layer eseguono la classificazione vera e propria combinando le feature estratte dai layer convoluzionali. Il layer finale produce un punteggio per ciascuna delle C classi, tipicamente seguito da **softmax** per ottenere una distribuzione di probabilità.

8.8 Architettura Tipica di una ConvNet

Una ConvNet classica è costruita impilando ripetutamente layer CONV+ReLU, seguiti periodicamente da layer di pooling e, alla fine, da layer FC:

$$\underbrace{[(\text{CONV} \rightarrow \text{ReLU})^{\times N} \rightarrow \text{POOL?}]^{\times M}}_{\text{estrazione di feature}} \longrightarrow \underbrace{(\text{FC} \rightarrow \text{ReLU})^{\times K} \rightarrow \text{Softmax}}_{\text{classificazione}} \quad (8.6)$$

dove tipicamente $N \lesssim 5$, M è grande, $0 \leq K \leq 2$.

Trend moderni.

- Filtri piccoli (3×3) in stack profondi al posto di filtri grandi (due 3×3 hanno lo stesso campo recettivo di un 5×5 ma con meno parametri e più nonlinearità).
- Eliminazione progressiva del pooling, sostituito da convoluzioni con stride 2.
- Architetture come **ResNet** e **GoogLeNet** sfidano il paradigma sequenziale con connessioni residue e moduli Inception.

8.8.1 Esempio di stack convoluzionale

Tabella 8.1: Evoluzione delle dimensioni dei volumi in una ConvNet di esempio su un'immagine $32 \times 32 \times 3$.

Layer	Filtri / Operazione	Output	Param.
Input	–	$32 \times 32 \times 3$	0
CONV1+ReLU	6 filtri 5×5 , $s = 1$, $P = 0$	$28 \times 28 \times 6$	456
CONV2+ReLU	10 filtri 5×5 , $s = 1$, $P = 0$	$24 \times 24 \times 10$	1510
MaxPool	2×2 , $s = 2$	$12 \times 12 \times 10$	0
FC+ReLU	120 unità	120	173 640
Softmax	10 classi	10	1210

8.9 Proprietà Fondamentali delle ConvNet

8.9.1 Parameter sharing

In un layer convoluzionale, lo **stesso filtro** (con gli stessi pesi) viene applicato in *tutte* le posizioni spaziali dell'input. Questo si basa sull'assunzione che una feature utile in una posizione sia utile anche in un'altra (*equivarianza traslazionale*). Il vantaggio pratico è una drastica riduzione del numero di parametri.

8.9.2 Equivarianza traslazionale

La convoluzione è **equivariante per traslazione**: traslare l'input causa una traslazione corrispondente nella mappa di attivazione. Formalmente, se T_Δ è una traslazione di Δ posizioni:

$$f(T_\Delta(\mathbf{x})) = T_\Delta(f(\mathbf{x})). \quad (8.7)$$

Il pooling con stride > 1 introduce invece **invarianza approssimata**: una traslazione di 1 pixel nell'input diventa 0.5 pixel dopo un pooling 2×2 .

8.9.3 Connessioni sparse

Ogni unità dell'output dipende solo da una piccola regione locale dell'input invece che da tutti i pixel, riducendo la complessità computazionale e favorendo la generalizzazione.

8.10 Riepilogo dei Componenti Principali

Tabella 8.2: Riepilogo dei layer fondamentali di una ConvNet.

Layer	Funzione	Iperparametri principali
CONV	Estrazione di feature locali tramite prodotti scalari con filtri appresi	F, K, S, P
ReLU	Nonlinearità puntuale $\max(0, x)$	–
POOL (Max/Avg)	Sottocampionamento spaziale	Finestra $P_H \times P_W, S$
FC	Classificazione globale	Numero di unità
Softmax	Distribuzione di probabilità sulle classi	–

Regole pratiche.

1. Preferire filtri piccoli (3×3) in stack profondi.
2. Usare zero-padding $P = (F - 1)/2$ per preservare la risoluzione nei layer CONV con stride 1.
3. Il pooling 2×2 con stride 2 è la scelta più comune; il global average pooling è preferibile nei classificatori moderni.
4. Verificare sempre che l'equazione (8.3) dia un intero.
5. Aumentare la profondità (più filtri per layer) man mano che la risoluzione spaziale diminuisce, per mantenere la capacità rappresentativa della rete.

Come sottolinea [Bishop and Bishop \(2024\)](#), la scelta dell'architettura convoluzionale è guidata da considerazioni sul campo recettivo, dalla capacità del modello rispetto alla dimensione del dataset e dal costo computazionale del training e dell'inference.

Capitolo 9

Reti Ricorrenti

I dati del mondo reale sono spesso intrinsecamente *sequenziali*: la posizione futura di un oggetto in moto, le parole di un testo, le note di una melodia o i fotogrammi di un video dipendono da ciò che è accaduto *prima*. Le reti neurali standard (feed-forward) ignorano questa struttura temporale: ricevono un input di dimensione fissa e producono un output indipendente da ogni elaborazione precedente. Le **reti neurali ricorrenti** (RNN) colmano questa lacuna introducendo un meccanismo di *memoria implicita* che trasmette informazioni da un passo temporale al successivo (Bishop and Bishop, 2024).

9.1 Motivazione: Modellare Sequenze

9.1.1 Il Problema della Modellazione Sequenziale

Consideriamo il compito di predire la prossima parola in una frase. Un primo approccio naïve consiste nell'usare una finestra fissa di k parole precedenti come input a una rete feed-forward (*fixed-window approach*). Questo schema soffre però di tre limitazioni fondamentali:

Dipendenze a lungo termine Una finestra fissa non può catturare contesti lontani nel tempo. Frase come “*France is where I grew up, but I now live in Boston. I speak fluent _____*” richiedono informazioni che si trovano decine di parole prima.

Mancato rispetto dell'ordine La rappresentazione a *bag of words* codifica solo la presenza/assenza di parole: non distingue “*The food was good, not bad at all*” da “*The food was bad, not good at all*”.

Assenza di condivisione dei parametri Con una finestra grande ogni posizione ha i propri pesi separati; ciò impedisce di trasferire le conoscenze apprese in una posizione alle altre.

Le RNN risolvono questi problemi mantenendo uno **stato nascosto** $\mathbf{h}_t$ che riassume tutta la storia della sequenza fino al passo t .

9.1.2 Rappresentazione Numerica del Linguaggio

Le reti neurali richiedono input numerici; le parole devono essere quindi trasformate in vettori. Il processo si articola in tre fasi: costruire un **vocabolario** (corpus di tutte le parole), assegnare un indice a ogni parola (**indicizzazione**), e infine convertire l'indice in un vettore di dimensione fissa tramite **embedding**.

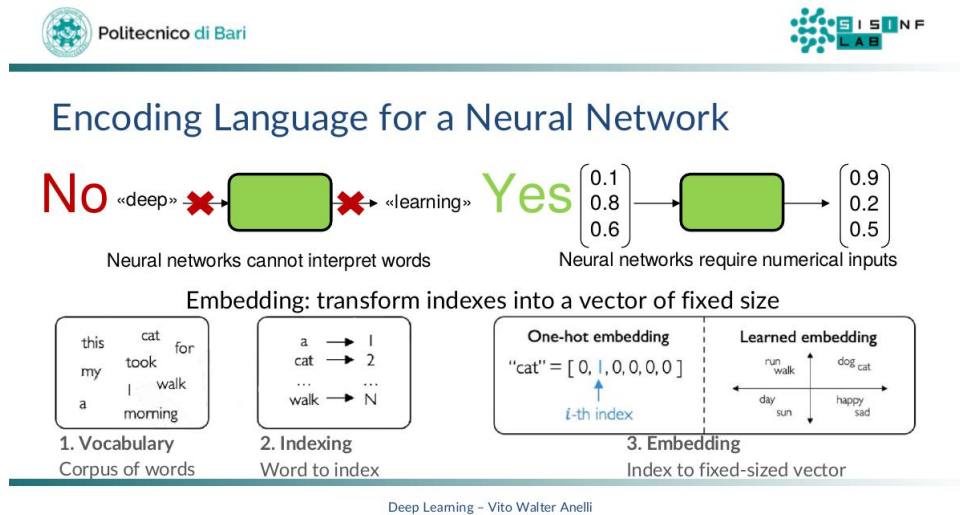


Figura 9.1: Codifica del linguaggio per una rete neurale. Sinistra: le parole non possono essere date direttamente in input. Centro: one-hot encoding (sparso, non apprende similarità semantiche). Destra: learned embedding (denso, cattura relazioni tra parole nello spazio vettoriale).

L'**one-hot encoding** rappresenta la parola i come un vettore binario di lunghezza $|V|$ con un solo 1 nella posizione i ; è semplice ma non codifica alcuna nozione di similarità semantica. Il **learned embedding** (es. Word2Vec, GloVe) apprende durante l'addestramento una matrice $E \in \mathbb{R}^{|V| \times d}$ che proietta ogni indice in un vettore denso di dimensione d , catturando relazioni semantiche come $\text{king} - \text{man} + \text{woman} \approx \text{queen}$.

9.2 Reti Neurali Ricorrenti (RNN)

9.2.1 Neuroni con Ricorrenza

Una rete feed-forward calcola $\hat{\mathbf{y}}_t = f(\mathbf{x}_t)$ indipendentemente per ogni passo temporale. Una **RNN** invece aggiunge una connessione ricorrente dallo stato nascosto al passo precedente:

$$\hat{\mathbf{y}}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}). \quad (9.1)$$

L'equazione di **aggiornamento dello stato nascosto** e di **produzione dell'output** di una RNN vanilla sono:

$$\mathbf{h}_t = \tanh(W_{hh}^T \mathbf{h}_{t-1} + W_{xh}^T \mathbf{x}_t), \quad (9.2)$$

$$\hat{\mathbf{y}}_t = W_{hy}^T \mathbf{h}_t, \quad (9.3)$$

dove $W_{hh} \in \mathbb{R}^{d_h \times d_h}$, $W_{xh} \in \mathbb{R}^{d_x \times d_h}$ e $W_{hy} \in \mathbb{R}^{d_h \times d_y}$ sono le matrici dei pesi, *condivise tra tutti i passi temporali*. Questa condivisione è la chiave dell'efficienza parametrica delle RNN.

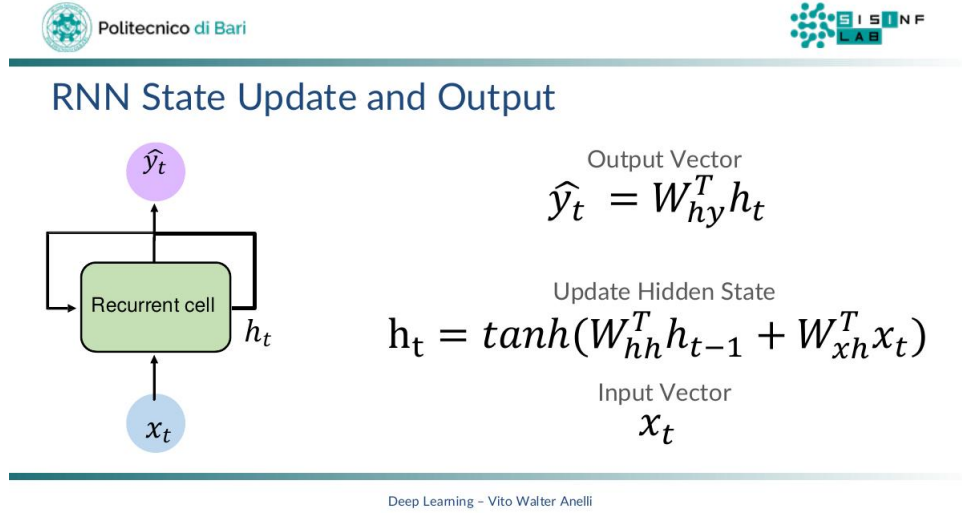


Figura 9.2: Cella ricorrente: sinistra, la rappresentazione compatta con il loop di ricorrenza; destra, le equazioni di aggiornamento dello stato nascosto $\mathbf{h}_t$ e di calcolo dell'output $\hat{\mathbf{y}}_t$.

Listing 9.1: RNN minima in PyTorch.

```

1 import torch.nn as nn
2
3 rnn = nn.RNN(input_size=d_x, hidden_size=d_h,
4               num_layers=1, batch_first=True)
5 # output: (batch, seq_len, d_h), h_n: (1, batch, d_h)
6 output, h_n = rnn(x, h_0)
```

9.2.2 Grafo Computazionale nel Tempo

Una RNN può essere visualizzata come una rete feed-forward molto profonda ottenuta *srotolando* (*unrolling*) la ricorrenza nel tempo. Ad ogni passo temporale si inserisce un livello che condivide i pesi con tutti gli altri. La loss totale è la somma delle loss ai singoli istanti:

$$\mathcal{L} = \sum_{t=0}^T \mathcal{L}_t(\hat{\mathbf{y}}_t, \mathbf{y}_t). \quad (9.4)$$

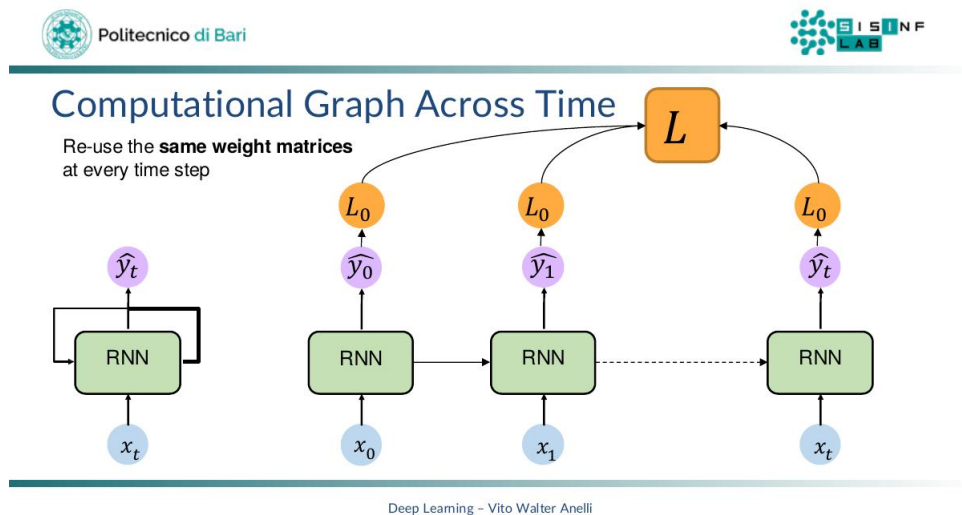


Figura 9.3: Grafo computazionale di una RNN srotolata nel tempo. Le stesse matrici dei pesi W_{hh} , W_{xh} , W_{hy} vengono riusate a ogni passo; la loss $\mathcal{L}$ è la somma delle loss parziali $\mathcal{L}_0, \mathcal{L}_1, \dots$.

9.2.3 Architetture Sequenziali

A seconda del rapporto tra lunghezza dell'input e dell'output si distinguono:

One-to-One Rete vanilla senza ricorrenza.

Many-to-One Una sequenza in ingresso produce un singolo output (es. classificazione del sentiment).

One-to-Many Un singolo input produce una sequenza in uscita (es. generazione di musica o di testo).

Many-to-Many Sequenza in ingresso, sequenza in uscita (es. traduzione automatica, etichettatura POS).

9.3 Backpropagation Through Time (BPTT)

9.3.1 Algoritmo

Poiché la RNN srotolata è equivalente a una rete feed-forward con pesi condivisi, la si può addestrare con la normale backpropagation applicata al grafo computazionale nel tempo: è il cosiddetto algoritmo di **Backpropagation Through Time (BPTT)** (Bishop and Bishop, 2024).

- Il *forward pass* scorre la sequenza da $t = 0$ a $t = T$, costruendo uno stack delle attività nascoste $\mathbf{h}_0, \dots, \mathbf{h}_T$.
- Il *backward pass* disimpila le attività e calcola i gradienti dell'errore a ogni passo temporale.

- I gradienti rispetto a W_{hh} , W_{xh} , W_{hy} vengono sommati su tutti i passi prima dell'aggiornamento.

Formalmente, il gradiente dell'errore rispetto allo stato nascosto al passo t dipende da tutti i passi successivi:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t+1}} \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t}. \quad (9.5)$$

9.3.2 Il Problema dei Gradienti che Svaniscono o Esplodono

Il calcolo del gradiente rispetto allo stato nascosto iniziale $\mathbf{h}_0$ richiede il prodotto di T fattori W_{hh} (moltiplicazione matriciale ripetuta). Questo genera due patologie simmetriche:

Gradienti che svaniscono Se i valori singolari di W_{hh} sono < 1 , il prodotto converge a zero esponenzialmente: le informazioni distanti nel tempo non riescono più a influenzare i pesi.

Gradienti che esplodono Se i valori singolari sono > 1 , il prodotto diverge: l'addestramento diventa instabile.

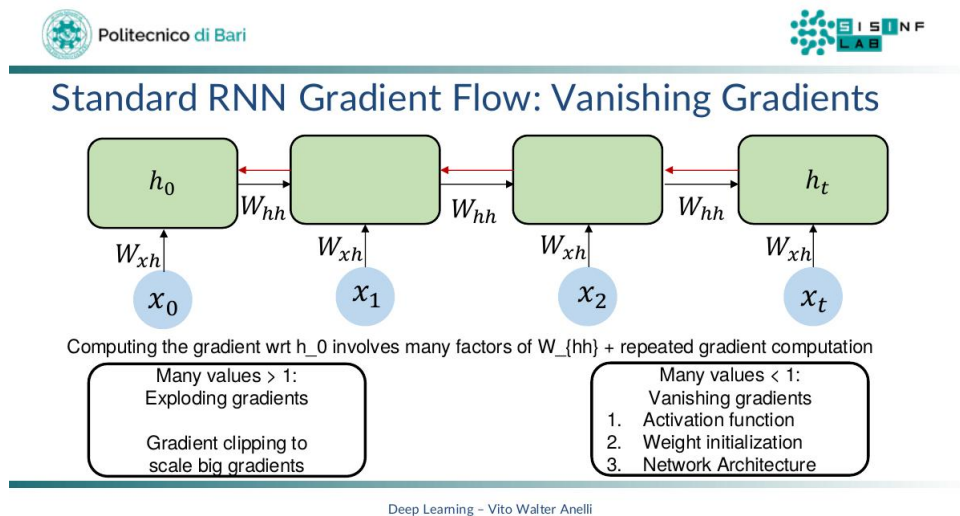


Figura 9.4: Flusso del gradiente in una RNN standard. Le frecce rosse mostrano come il gradiente debba attraversare molte moltiplicazioni per W_{hh} per raggiungere $\mathbf{h}_0$. Se $\|W_{hh}\| < 1$: vanishing; se $\|W_{hh}\| > 1$: exploding.

Soluzioni pratiche.

- **Gradient clipping**: si riscalano i gradienti quando la loro norma supera una soglia θ : $\nabla \leftarrow \frac{\theta}{\|\nabla\|} \nabla$ se $\|\nabla\| > \theta$. Mitiga l'esplosione.
- **Inizializzazione della matrice W_{hh}** come matrice ortogonale (valori singolari = 1): rallenta il collasso.

- **Architetture con gate** (LSTM, GRU): soluzione strutturale al problema del vanishing (Sezione 9.4).

Nota sul backward pass lineare. A differenza del forward pass — dove funzioni di attivazione come tanh saturano e *comprimono* i valori — il backward pass è localmente *lineare*: raddoppiare i gradienti all'uscita raddoppia tutti i gradienti a ritroso nella rete. Questo amplifica la fragilità numerica del BPTT su sequenze lunghe.

9.4 Long Short-Term Memory (LSTM)

9.4.1 Motivazione: La Memoria Analogica

? propongono di risolvere il problema del vanishing gradient introducendo una **cella di memoria** con connessioni lineari di auto-rinforzo (peso di auto-collegamento uguale a 1) e **porte** (*gates*) che controllano selettivamente la scrittura, la lettura e la cancellazione delle informazioni.

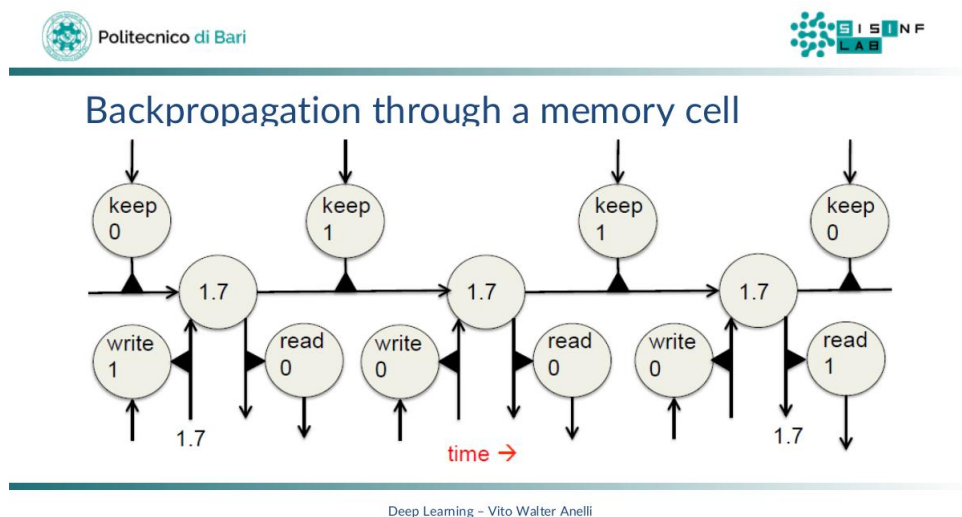


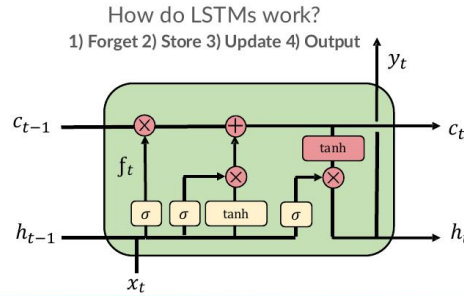
Figura 9.5: Cella di memoria analogica: il valore 1.7 viene scritto quando il gate $write = 1$, mantenuto finché $keep = 1$, e letto quando $read = 1$. La backpropagation attraversa questo circuito perché le porte logistiche hanno derivate ben definite.

9.4.2 Architettura della Cella LSTM

Una cella LSTM mantiene due vettori di stato distinti:

- $\mathbf{c}_t \in \mathbb{R}^{d_h}$: **cell state**, la memoria a lungo termine.
- $\mathbf{h}_t \in \mathbb{R}^{d_h}$: **hidden state**, la memoria a breve termine e l'output della cella.

Long Short Term Memory (LSTMs)



Deep Learning - Vito Walter Anelli

Figura 9.6: Schema completo della cella LSTM. Il cell state $\mathbf{c}_t$ scorre orizzontalmente subendo solo operazioni element-wise (moltiplicazione e addizione), garantendo un flusso del gradiente non interrotto durante la BPTT.

Il comportamento della cella si articola in quattro fasi.

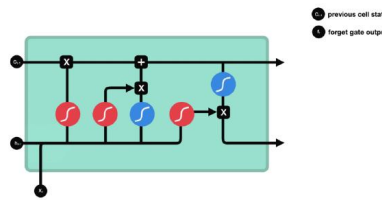
Forget Gate (Cancellazione)

Il forget gate decide quali informazioni eliminare dallo stato della cella:

$$\mathbf{f}_t = \sigma \left(W_f \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} + \mathbf{b}_f \right), \quad (9.6)$$

dove σ è la sigmoide. Un valore $f_t^{(i)} \approx 0$ cancella l' i -esima componente del cell state; $f_t^{(i)} \approx 1$ la conserva.

Forget gate



This gate decides what information should be thrown away or kept. Information from the previous hidden state and information from the current input is passed through the sigmoid function. Values come out between 0 and 1. The closer to 0 means to forget, and the closer to 1 means to keep.

Deep Learning - Vito Walter Anelli

Figura 9.7: Forget gate: le attivazioni sigmoidee (in rosso) determinano quali informazioni del cell state precedente vengono eliminate (moltiplicazione per $\mathbf{f}_t$).

Input Gate (Memorizzazione)

Due sub-gate cooperano per aggiungere nuova informazione al cell state:

$$\mathbf{i}_t = \sigma \left(W_i \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} + \mathbf{b}_i \right), \quad (9.7)$$

$$\tilde{\mathbf{c}}_t = \tanh \left(W_c \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} + \mathbf{b}_c \right). \quad (9.8)$$

$\mathbf{i}_t$ (gate sigmoid) seleziona quali componenti aggiornare; $\tilde{\mathbf{c}}_t$ (gate tanh) genera il valore candidato da scrivere.

Aggiornamento del Cell State

Il cell state viene aggiornato combinando la cancellazione e la scrittura selettiva:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (9.9)$$

dove $\odot$ indica il prodotto element-wise (Hadamard).

Output Gate (Uscita)

L'output gate filtra il cell state per produrre il nuovo hidden state:

$$\mathbf{o}_t = \sigma \left(W_o \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} + \mathbf{b}_o \right), \quad (9.10)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \quad (9.11)$$

9.4.3 Flusso del Gradiente nelle LSTM

Il punto cruciale delle LSTM è che la backpropagation da $\mathbf{c}_t$ a $\mathbf{c}_{t-1}$ richiede *solo operazioni element-wise* (moltiplicazione per $\mathbf{f}_t$ e addizione); non c'è alcuna moltiplicazione matriciale lungo il percorso del cell state. Di conseguenza:

- Il gradiente scorre quasi indisturbato per centinaia di passi temporali (*uninterrupted gradient flow*).
- Il forget gate impara a modulare il decay del gradiente in modo *adattivo*: quando $\mathbf{f}_t \approx \mathbf{1}$ il gradiente viene interamente preservato; quando $\mathbf{f}_t \approx \mathbf{0}$ la dipendenza dal passato viene eliminata.

Listing 9.2: LSTM in PyTorch.

```

1 import torch.nn as nn
2
3 lstm = nn.LSTM(input_size=d_x, hidden_size=d_h,
4                 num_layers=n_layers, batch_first=True)
```

```

5 # h_0, c_0: (n_layers, batch, d_h)
6 output, (h_n, c_n) = lstm(x, (h_0, c_0))

```

9.5 Gated Recurrent Unit (GRU)

9.5.1 Architettura

La **Gated Recurrent Unit** (?) è una semplificazione delle LSTM che elimina il cell state separato e usa solo due porte, riducendo il numero di parametri e spesso raggiungendo prestazioni comparabili.

Reset gate. Il reset gate $\mathbf{r}_t$ controlla quanta memoria del passato viene dimenticata nel calcolo del candidato:

$$\mathbf{r}_t = \sigma(W_r [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_r). \quad (9.12)$$

Update gate. Il update gate $\mathbf{z}_t$ determina in che misura il candidato $\tilde{\mathbf{h}}_t$ sostituisce il vecchio hidden state:

$$\mathbf{z}_t = \sigma(W_z [\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_z). \quad (9.13)$$

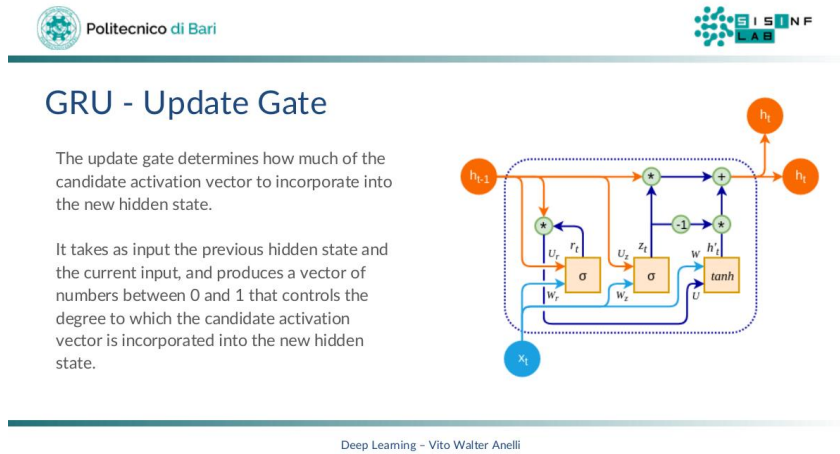


Figura 9.8: GRU: update gate $\mathbf{z}_t$ (a destra nel diagramma). Il vettore di output è una media pesata tra il vecchio stato e il candidato, con il bilanciamento controllato da $\mathbf{z}_t$.

Candidato e aggiornamento.

$$\tilde{\mathbf{h}}_t = \tanh(W [\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}), \quad (9.14)$$

$$\mathbf{h}_t = (\mathbf{1} - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad (9.15)$$

L'equazione (9.15) mostra che la GRU interpola linearmente tra lo stato vecchio e quello candidato: quando $\mathbf{z}_t \approx \mathbf{1}$ l'intero hidden state viene rimpiazzato; quando

$\mathbf{z}_t \approx \mathbf{0}$ lo stato viene mantenuto intatto (effetto simile a quello dell'LSTM con forget gate chiuso e input gate aperto).

9.5.2 Confronto LSTM vs GRU

Tabella 9.1: Confronto tra LSTM e GRU.

Caratteristica	LSTM	GRU
Numero di porte	3 (forget, input, output)	2 (reset, update)
Vettori di stato	2 ($\mathbf{h}_t, \mathbf{c}_t$)	1 ($\mathbf{h}_t$)
Parametri per cella	$4 \times (d_h^2 + d_h d_x + d_h)$	$3 \times (d_h^2 + d_h d_x + d_h)$
Vanishing gradient	Risolto (cell state lineare)	Risolto (interpolazione)
Prestazioni tipiche	Migliori su sequenze lunghissime	Comparabili, più veloci

9.6 Bi-LSTM e Architetture Avanzate

9.6.1 LSTM Bidirezionale

Una **Bi-LSTM** elabora la sequenza in entrambe le direzioni temporali (sinistra→destra e destra→sinistra) e concatena i due hidden state. Questo consente all'output al passo t di dipendere sia dal contesto passato che da quello futuro. Formalmente:

$$\vec{\mathbf{h}}_t = \text{LSTM}_{\text{fwd}}(\mathbf{x}_t, \vec{\mathbf{h}}_{t-1}), \quad (9.16)$$

$$\overleftarrow{\mathbf{h}}_t = \text{LSTM}_{\text{bwd}}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1}), \quad (9.17)$$

$$\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]. \quad (9.18)$$

Le Bi-LSTM trovano ampio impiego in NLP per task come Named Entity Recognition (NER), Part-of-Speech tagging e relazione semantica, dove il contesto futuro è disponibile al momento dell'inferenza. Non sono adatte alla generazione autoregressiva (la direzione futura non è disponibile durante il decoding).

9.6.2 Encoder–Decoder per la Traduzione Automatica

Il paradigma **sequence-to-sequence** (Seq2Seq) usa due RNN separate:

Encoder Legge la sequenza sorgente e la comprime in un vettore di contesto fisso $\mathbf{c} = \mathbf{h}_T^{\text{enc}}$.

Decoder Genera la sequenza target autoregressivamente, condizionato su $\mathbf{c}$: $\mathbf{h}_t^{\text{dec}} = f(\mathbf{h}_{t-1}^{\text{dec}}, \hat{y}_{t-1}, \mathbf{c})$.

Il vettore di contesto fisso costituisce un **collo di bottiglia** informativo: per sequenze molto lunghe non riesce a codificare tutta l'informazione rilevante. Questo limite motiva l'introduzione dei **meccanismi di attenzione**, che permettono al decoder di accedere direttamente a tutti gli hidden state dell'encoder pesandoli in modo apprendibile (?).

9.7 Applicazioni delle RNN

Generazione di musica La rete riceve caratteri di notazione musicale e predice il prossimo carattere (schema many-to-many con teacher forcing durante l'addestramento).

Classificazione del sentiment Una LSTM legge una sequenza di parole e l'ultimo hidden state viene passato a un classificatore lineare (many-to-one).

Traduzione automatica Architettura Seq2Seq encoder-decoder, idealmente potenziata con attenzione.

Predizione di sequenze fisiche Stima della traiettoria di oggetti in moto come predizione della prossima posizione di una pallina.

Modellazione del linguaggio Calcolo della distribuzione di probabilità $P(w_t \mid w_1, \dots, w_{t-1})$ alla base di modelli linguistici e sistemi di completamento automatico.

9.8 Riepilogo e Linee Guida Pratiche

Tabella 9.2: Guida alla scelta dell'architettura ricorrente.

Scenario	Architettura	Note
Sequenze corte, baseline rapida	RNN vanilla	Instabile su $T > 20$
Dipendenze a lungo termine	LSTM	Standard de facto pre-Transformer
Risorse computazionali limitate	GRU	Meno parametri, buona efficienza
Contesto bidirezionale disponibile	Bi-LSTM	NER, POS tagging
Traduzione, summarization	Seq2Seq + Attenzione	Precursore dei Transformer

Consigli operativi.

1. Usare **gradient clipping** (norma soglia ≈ 5) per tutte le RNN vanilla; nelle LSTM è meno critico ma consigliabile.
2. Inizializzare W_{hh} come matrice **ortogonale** per stabilizzare le prime fasi dell'addestramento.
3. Preferire le **LSTM** alle RNN vanilla se la sequenza supera i 20–30 passi; passare alle **GRU** per ridurre la memoria necessaria.
4. Per l'NLP, usare un **learned embedding** pre-addestrato (es. GloVe, FastText) come layer di input: riduce drasticamente i tempi di convergenza.
5. Le RNN possono essere sostituite dai **Transformer** per molti task di sequenza, ma rimangono preferibili quando si ha memoria limitata, si lavora su dispositivi edge o quando la lunghezza della sequenza è molto variabile (Bishop and Bishop, 2024).
6. Per il Seq2Seq, integrare sempre un **meccanismo di attenzione**: rompe il collo di bottiglia del vettore di contesto fisso e migliora significativamente la qualità delle traduzioni lunghe.

Bibliografia

- Mistral AI. Mixtral of experts. <https://arxiv.org/abs/2401.04088>, 2023.
- Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer Nature, 2024. ISBN 978-3-031-45467-7. URL <https://www.bishopbook.com>. Freely available online at <https://www.bishopbook.com>.
- Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
- Yann LeCun and Alfredo Canziani. NYU deep learning SP21 – DS-GA 1008. <https://atcold.github.io/NYU-DLSP21/>, 2021. Spring 2021, NYU Center for Data Science.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.